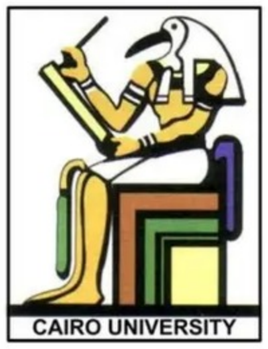


MICROCONTROLLERS AND INTERFACING USING ARM PROCESSOR

Lecture 1

Prof. Yasser Mostafa Kadah

Course Goals

- At the end of this course, you should be able to:
  - Understand the architecture of one of the most popular microcontroller (MCU) families
  - Use an integrated development environment (IDE) to program and debug an MCU
  - Program an MCU using Assembly and C languages
  - Understand and use peripherals integrated into an MCU
  - Interface an MCU to simple external components
  - Use timers in various modes
  - Understand and use interrupts
  - Understand and use analog to digital converters (ADC) and digital to analog converters (DAC)
  - Communicate using a serial interface (if time allows)

Course Prerequisites

- A course in Electric Circuits that includes understanding basic electronic components such as resistors, capacitors, diodes and transistors

- A course in basic digital logic design that includes logic gates and Boolean arithmetic

- Ability to program in a high-level programming language such as C or C++

Microprocessors and Microcontrollers

- Microprocessor: general-purpose CPU
  - Emphasis is on flexibility and performance
  - Generic user-interface such as keyboard, mouse, etc.
  - Used in a PC, PDA, cell phone, etc.

- Microcontroller: microprocessor + memory on a single chip
  - Emphasis is on size and cost reduction
  - The user interface is tailored to the application, such as the buttons on a TV remote control
  - Used in a digital watch, TV remote control, car and many common day-to-day appliances

Microcontroller Architectures

- Microcontroller architecture refers to the internal hardware organization of a microcontroller
- Each hardware architecture has its own set of software instructions called assembly language that allows programming of the microcontroller
- Some of the popular microcontroller architectures
  - Intel 8051
  - Zilog Z80
  - Atmel AVR
  - Microchip PIC
  - ARM

ARM Processors

- ARM: Advanced RISC Machines
  - Previously "Acorn RISC Machines"
  - ARM1: 1985
  - Now: ARM11, ARM Cortex A, ARM Cortex R, ARM Cortex M

STM32 Platform

STM32 Product Lines

Common core peripherals and architecture:

- Communication peripherals: USART, SPI, I²C
- Multiple general-purpose timers
- Integrated reset and brown-out warning
- Multiple DMA
- 2x watchdogs Real-time clock
- Integrated regulator PLL and clock circuit
- External memory interface (FSMC)
- Up to 3x 12-bit DAC
- Up to 4x 12-bit ADC (Up to 5 MSPS)
- Main oscillator and 32 kHz oscillator
- Low-speed and high-speed internal RC oscillators
- -40 to +85 °C and up to 105 °C operating temperature range
- Low voltage 2.0 to 3.6 V or 1.65/1.7 to 3.6 V (depending on series)
- Temperature sensor

STM32 F4 series - High performance with DSP (STM32F405/415/407/417)

168 MHz Cortex-M4 with DSP and FPU	Up to 192-Kbyte SRAM	Up to 1-Mbyte Flash	2x USB 2.0 OTG FS/HS	3-phase MC timer	2x CAN 2.0B	SDIO 2x I²S audio Camera IF	Ethernet IEEE 1588	Crypto/ hash processor and RNG	STM32 F4

STM32 F3 series - Mixed-signal with DSP (STM32F302/303/313/372/373/383)

72 MHz Cortex-M4 with DSP and FPU	Up to 48-Kbyte SRAM & CCM-SRAM	Up to 256-Kbyte Flash	USB 2.0 FS	2x 3-phase MC timer (144 MHz)	CAN 2.0B	Up to 7x comparator	3x 16-bit ΣΔ ADC	4x PGA	STM32 F3

STM32 F2 series - High performance (STM32F205/215/207/217)

120 MHz Cortex-M3 CPU	Up to 128-Kbyte SRAM	Up to 1-Mbyte Flash	2x USB 2.0 OTG FS/HS	3-phase MC timer	2x CAN 2.0B	SDIO 2x I²S audio Camera IF	Ethernet IEEE 1588	Crypto/ hash processor and RNG	STM32 F2

STM32 F1 series - Mainstream - 5 product lines (STM32F100/101/102/103 and 105/107)

+

Up to 72 MHz Cortex-M3 CPU	Up to 96-Kbyte SRAM	Up to 1-Mbyte Flash	USB 2.0 OTG FS	3-phase MC timer	Up to 2x CAN 2.0B	SDIO 2x I²S audio	Ethernet IEEE 1588	STM32 F1

STM32 F0 series – Entry level (STM32F050/051)

48 MHz Cortex-M0 CPU	Up to 8-Kbyte SRAM	Up to 64-Kbyte Flash	3-phase MC timer	Comparator	CEC	STM32 F0

STM32 L1 series - Ultra-low-power (STM32L151/152/162)

32 MHz Cortex-M3 CPU	Up to 48-Kbyte SRAM	Up to 384-Kbyte Flash	USB FS device	Up to 12-Kbyte EEPROM	LCD 8x40 4x44	Comparator	BOR MSI VScal	AES 128-bit	STM32 L1

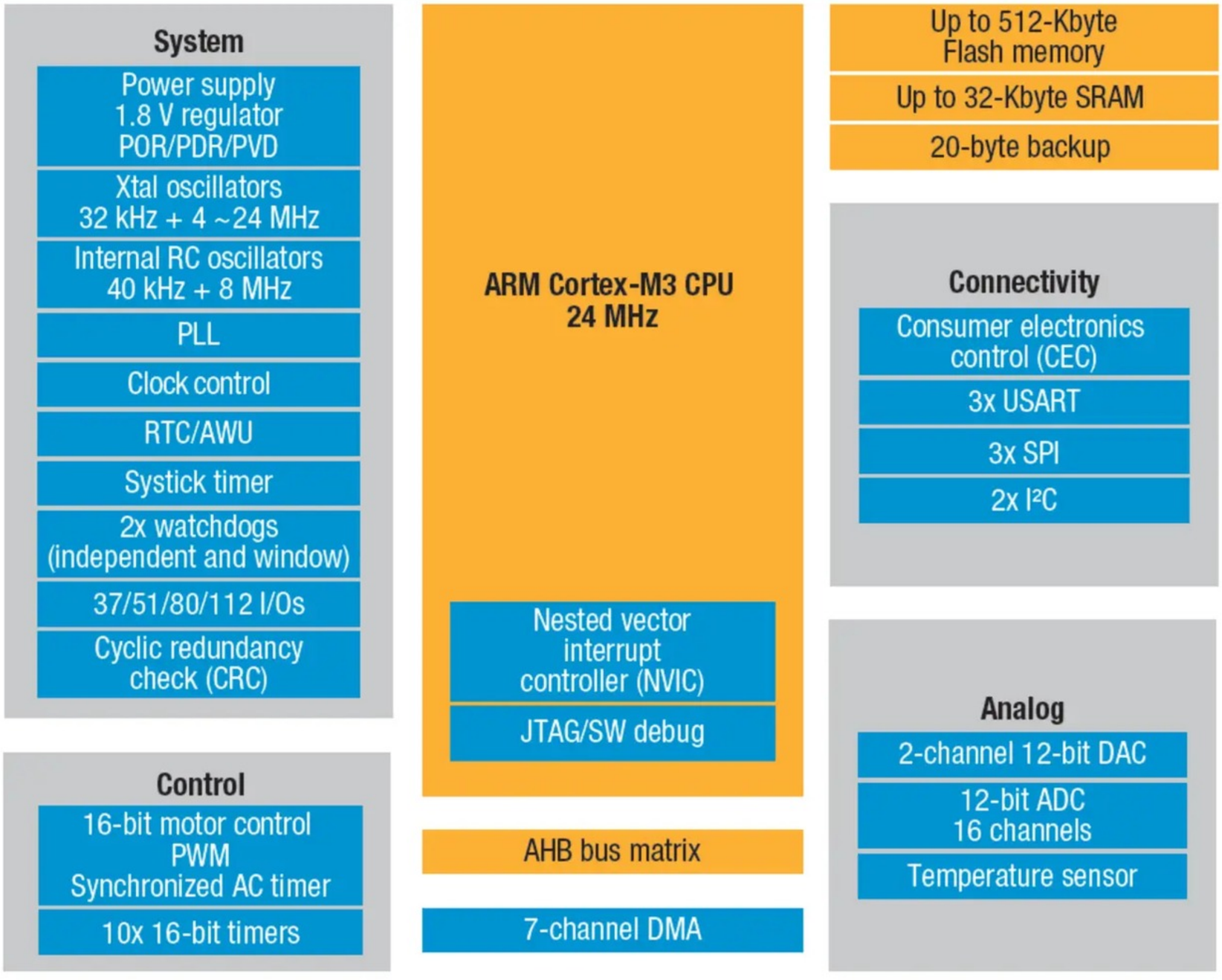
STM32 W series - Wireless (STM32W108)

24 MHz Cortex-M3 CPU	Up to 16-Kbyte SRAM	Up to 256-Kbyte Flash	2.4 GHz IEEE 802.15.4 Transceiver	Lower MAC Digital baseband	AES 128-bit	STM32 W

Abbreviations:

FS:	Full speed
HS:	High speed
MC:	Motor control
MSI:	Multi-speed internal oscillator
RNG:	Random number generator
SDIO:	Secure digital input/output
VScal:	Voltage scaling
FPU:	Floating point unit
DSC:	Digital signal controller

STM32 Value Line Block Diagram



Abbreviations:

AWU:	Auto wake up from halt	PDR:	Power-down reset	RTC:	Real-time clock
BOR:	Brown-out reset	POR:	Power-on reset	SPI:	Serial peripheral interface
I²C:	Inter integrated circuit	PVD:	Programmable voltage detector	USART:	Universal sync/async receiver transmitter

STM32 Value Line Devices

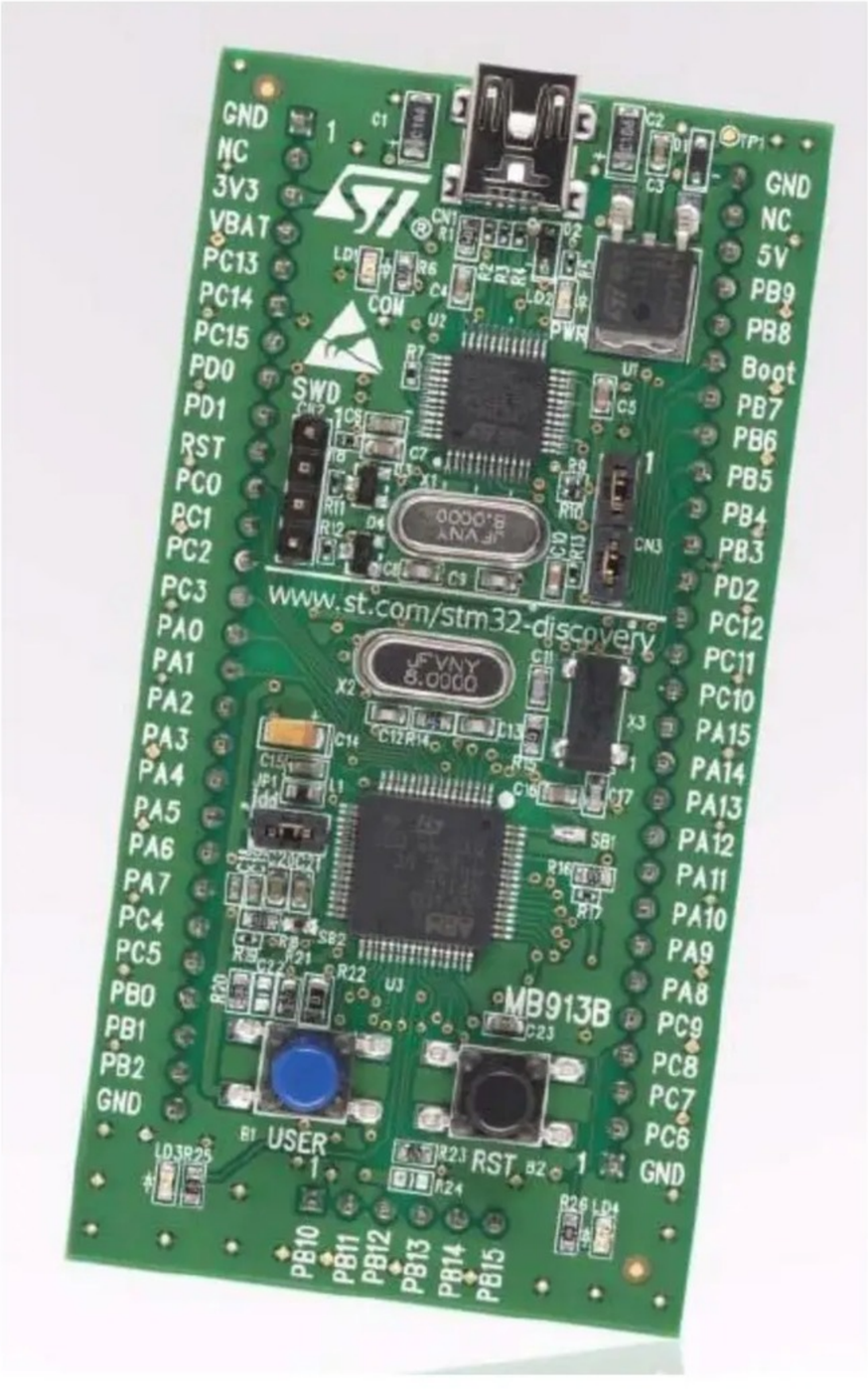
Part number	Package	Flash size (Kbytes)	Internal RAM size (Kbytes)	Timer functions		ADC	DAC	I/Os	Serial interface	Supply voltage (V)
				16-bit	Others					
STM32F100 Value line - 24 MHz CPU										
STM32F100C4	LQFP48 (7x7)	16	4	6x16-bit		10x12-bit	2x12-bit	37	1xSPI,1xI²C, CEC, 2xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100C6	LQFP48 (7x7)	32	4	6x16-bit		10x12-bit	2x12-bit	37		2 to 3.6
STM32F100C8	LQFP48 (7x7)	64	8	7x16-bit		10x12-bit	2x12-bit	37	2xSPI, 2xI²C, CEC, 3xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100CB	LQFP48 (7x7)	128	8	7x16-bit		10x12-bit	2x12-bit	37		2 to 3.6
STM32F100R4	LQFP64 (10x10), TFBGA6 (5x5)	16	4	6x16-bit		16x12-bit	2x12-bit	51		2 to 3.6
STM32F100R6	LQFP64 (10x10), TFBGA64 (5x5), Unsawn wafer V.I. 100%	32	4	6x16-bit		16x12-bit	2x12-bit	51	1xSPI, 1xI²C, CEC, 2xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100R8	LQFP64 (10x10), TFBGA64 (5x5)	64	8	7x16-bit		16x12-bit	2x12-bit	51		2 to 3.6
STM32F100RB	LQFP64 (10x10), TFBGA64 (5x5)	128	8	7x16-bit	2xWDG, RTC, 24-bit down counter, 2x16-bit basic timers	16x12-bit	2x12-bit	51	2xSPI, 2xI²C, CEC, 3xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100RC	LQFP64 (10x10)	256	24	11x16-bit		16x12-bit	2x12-bit	51		2 to 3.6
STM32F100RD	LQFP64 (10x10)	384	32	11x16-bit		16x12-bit	2x12-bit	51	3xSPI, 2xI²C, CEC, 5xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100RE	LQFP64 (10x10)	512	32	11x16-bit		16x12-bit	2x12-bit	51		2 to 3.6
STM32F100V8	LQFP100 (14x14)	64	8	7x16-bit		16x12-bit	2x12-bit	80	2xSPI, 2xI²C, CEC, 3xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100VB	LQFP100 (14x14)	128	8	7x16-bit		16x12-bit	2x12-bit	80		2 to 3.6
STM32F100VC	LQFP100 (14x14)	256	24	11x16-bit		16x12-bit	2x12-bit	80		2 to 3.6
STM32F100VD	LQFP100 (14x14)	384	32	11x16-bit		16x12-bit	2x12-bit	80		2 to 3.6
STM32F100VE	LQFP100 (14x14)	512	32	11x16-bit		16x12-bit	2x12-bit	80	3xSPI, 2xI²C, CEC, 5xUSART (IrDa, ISO 7816)	2 to 3.6
STM32F100ZC	LQFP144 (20x20)	256	24	11x16-bit		16x12-bit	2x12-bit	112		2 to 3.6
STM32F100ZD	LQFP144 (20x20)	384	32	11x16-bit		16x12-bit	2x12-bit	112		2 to 3.6
STM32F100ZE	LQFP144 (20x20)	512	32	11x16-bit		16x12-bit	2x12-bit	112		2 to 3.6

STM32 Value line Discovery Kit

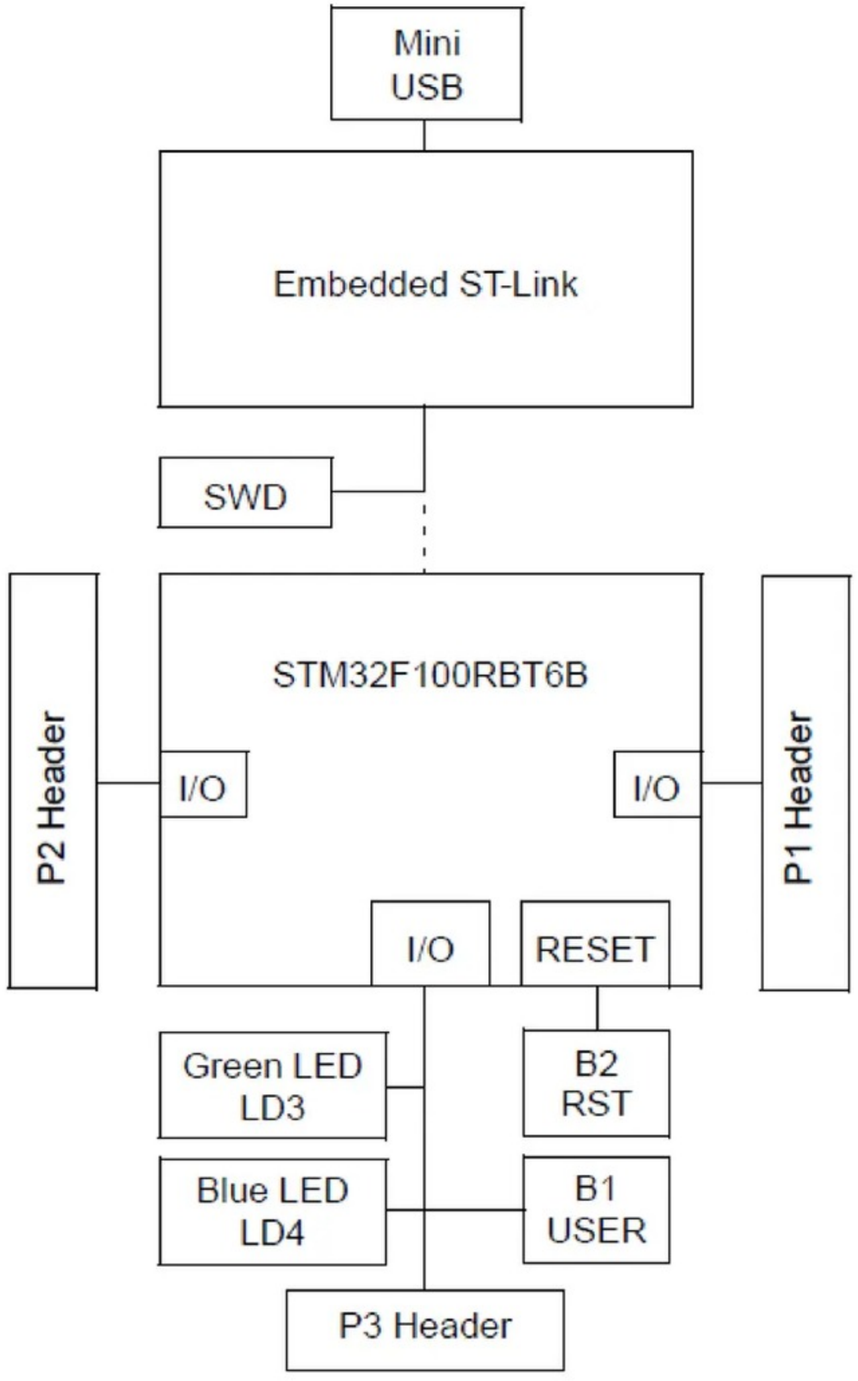
- The cheapest and quickest way to discover the STM32 Value line family
- Embedded ST-LINK or ST-LINK/V2 included to debug applications
- Works with several popular IDEs
  - MDK-ARM (Keil)
  - EWARM (IAR)

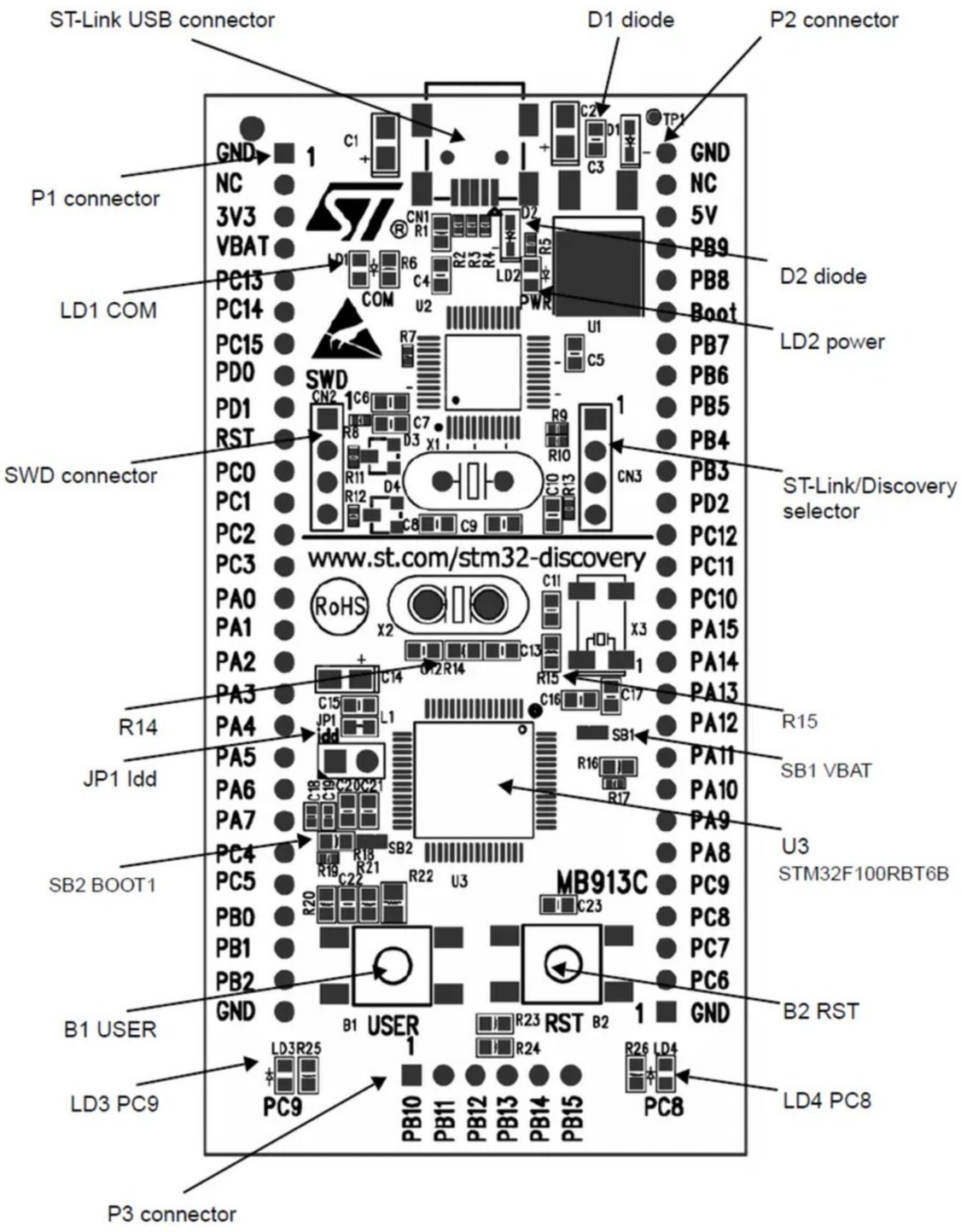
Kit Features

- STM32F100RBT6B microcontroller, 128 KB Flash, 8 KB RAM in 64-pin LQFP
- On-board ST-Link (for programming and debugging)
- Two red LEDs; LD1 for USB communication, LD2 for 3.3 volts power on
- Designed to be powered by USB or an external supply of 5 V or 3.3 V
- Can supply target application with 5 volts and 3 volts
- Two user LEDs, LD3 and LD4 (green and blue)
- Two push buttons (User and Reset)
- Extension header for all QFP64 I/Os for quick connection to prototyping board or easy probing

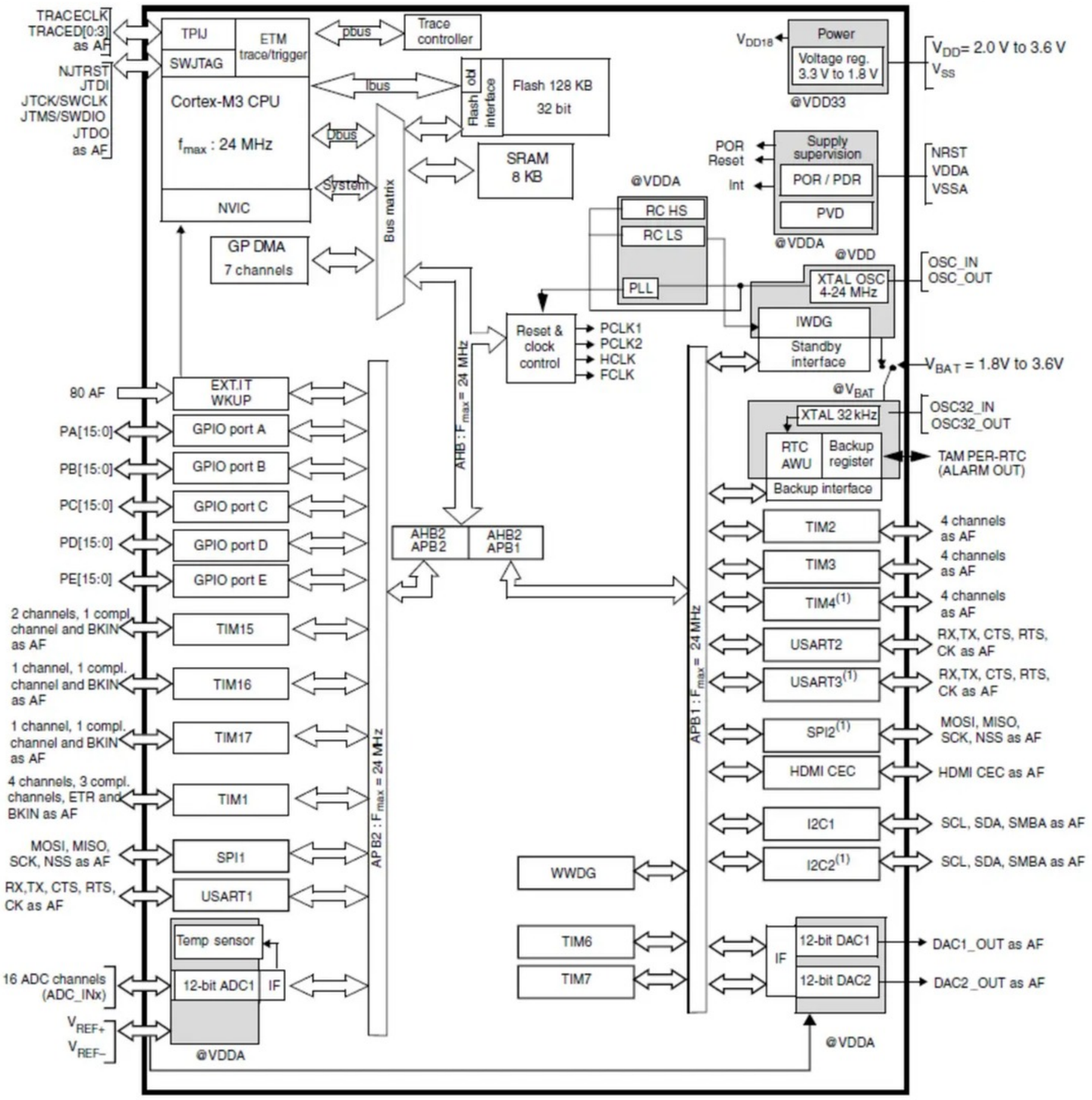


Kit Hardware Block Diagram





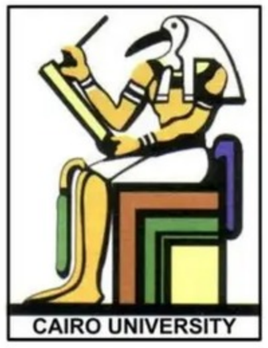
STM32F100RBT6B Microcontroller



Assignments

- Explain the differences between MIPS and DMIPS.
- Compare the major ARM processor families available today from the points of view of processing power, cost, applications. (Hint: focus on ARM11 and ARM Cortex series processors)
- Install the MDK-ARM Evaluation Version and compile and link the kit's demo software available from ST web site.
- Revision of C programming.

- Next week: Lab demonstration of kit and development tools



STM32 MICROCONTROLLER

Lecture 2

Prof. Yasser Mostafa Kadah

Harvard and von Neumann Architectures

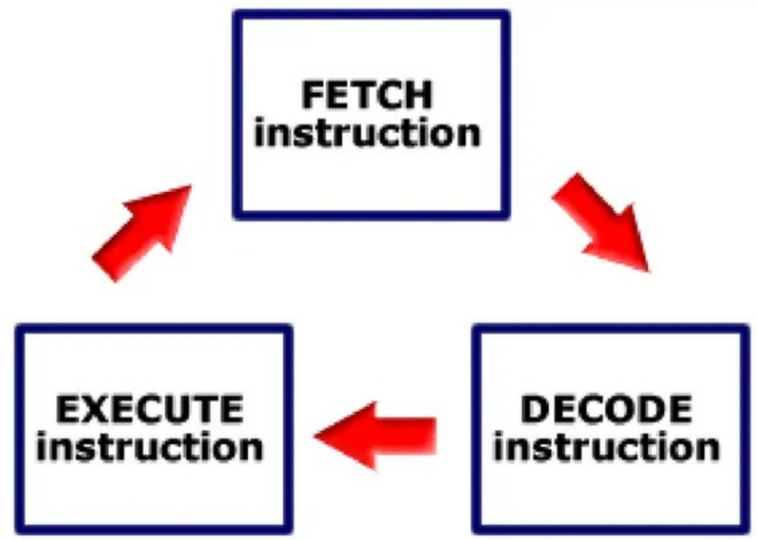
- Harvard Architecture—a type of computer architecture where the instructions (program code) and data are stored in separate memory spaces
  - Example: Intel 8051 architecture

- von Neumann Architecture—another type of computer architecture where the instructions and data are stored in the same memory space
  - Example: ARM, Intel x86 architecture (Intel Pentium, AMD Athlon, etc.)

Instruction Execution Cycle

- Fetch operation—retrieves an instruction from the location in code memory pointed to by the program counter (PC)

- Execute operation—executes the instruction that was fetched during the fetch operation. In addition to executing the instruction, the CPU also adds the appropriate number to the PC to point it to the next instruction to be fetched.



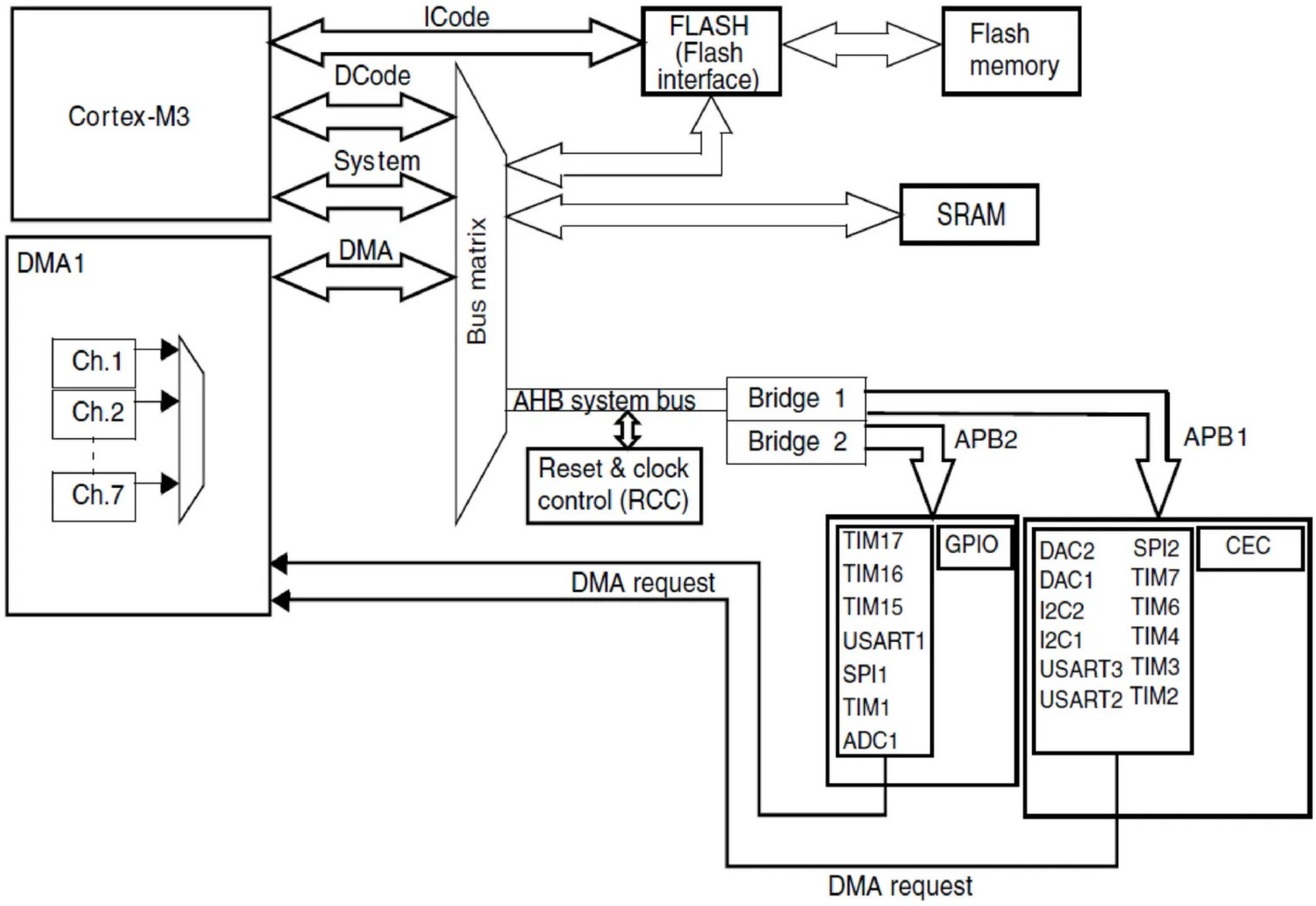
Microcontroller Architectures

- Microcontroller architecture refers to the internal hardware organization of a microcontroller
- Each hardware architecture has its own set of software instructions called assembly language that allows programming of the microcontroller
- Some of the popular microcontroller architectures
  - Intel 8051
  - Zilog Z80
  - Atmel AVR
  - Microchip PIC
  - ARM

Memory and Bus Architecture

- Three masters:
  - Cortex™-M3 core DCode bus (D-bus) and System bus (S-bus)
  - GP-DMA1 (general-purpose DMA)
- Three slaves:
  - Internal SRAM
  - Internal Flash memory
  - AHB to APB bridges (AHB to APBx), which connect all the APB peripherals

System Architecture



Definitions

- ICode bus
  - Connects the instruction bus of the Cortex™-M3 core to the Flash memory instruction interface. Instruction fetches are performed on this bus.
- DCode bus
  - Connects the DCode bus (literal load and debug access) of the Cortex™-M3 core to the Flash memory data interface
- System bus
  - Connects the system bus of the Cortex™-M3 core (peripherals bus) to a bus matrix which manages the arbitration between the core and the DMA

Definitions

- DMA bus
  - Connects the AHB master interface of the DMA to the bus matrix which manages the access of CPU DCode and DMA to the SRAM, Flash memory and peripherals
- Bus matrix
  - Manages the access arbitration between the core system bus and the DMA master bus. The arbitration uses a round robin algorithm
- AHB/APB bridges (APB)
  - The two AHB/APB bridges provide full synchronous connections between the AHB and the two APB buses
  - APB buses operate at full speed (up to 24 MHz)

Memory Organization

- Program memory, data memory, registers and I/O ports are organized within the same linear 4-Gbyte address space
- Bytes are coded in memory in *little endian* format
  - The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte, the most significant
- Addressable memory space is divided into 8 main blocks, each of 512 MB

Memory Map

Boundary address	Peripheral	Bus
0x4002 3000 - 0x4002 33FF	CRC	AHB
0x4002 2400 - 0x4002 2FFF	Reserved	
0x4002 2000 - 0x4002 23FF	Flash memory interface	
0x4002 1400 - 0x4002 1FFF	Reserved	
0x4002 1000 - 0x4002 13FF	Reset and clock control RCC	
0x4002 0400 - 0x4002 0FFF	Reserved	
0x4002 0000 - 0x4002 03FF	DMA1	
0x4001 4C00 - 0x4001 FFFF	Reserved	APB2
0x4001 4800 - 0x4001 4BFF	TIM17 timer	
0x4001 4400 - 0x4001 47FF	TIM16 timer	
0x4001 4000 - 0x4001 43FF	TIM15 timer	
0x4001 3C00 - 0x4001 3FFF	Reserved	
0x4001 3800 - 0x4001 3BFF	USART1	
0x4001 3400 - 0x4001 37FF	Reserved	
0x4001 3000 - 0x4001 33FF	SPI1	
0x4001 2C00 - 0x4001 2FFF	TIM1 timer	
0x4001 2800 - 0x4001 2BFF	Reserved	
0x4001 2400 - 0x4001 27FF	ADC1	
0x4001 1C00 - 0x4001 23FF	Reserved	
0x4001 1800 - 0x4001 1BFF	GPIO Port E	
0x4001 1400 - 0x4001 17FF	GPIO Port D	
0x4001 1000 - 0x4001 13FF	GPIO Port C	
0x4001 0C00 - 0x4001 0FFF	GPIO Port B	
0x4001 0800 - 0x4001 0BFF	GPIO Port A	
0x4001 0400 - 0x4001 07FF	EXTI	
0x4001 0000 - 0x4001 03FF	AFIO	

Boundary address	Peripheral	Bus
0x4000 7C00 - 0x4000 FFFF	Reserved	APB1
0x4000 7800 - 0x4000 7BFF	CEC	
0x4000 7400 - 0x4000 77FF	DAC	
0x4000 7000 - 0x4000 73FF	Power control PWR	
0x4000 6C00 - 0x4000 6FFF	Backup registers (BKP)	
0x4000 5C00 - 0x4000 6BFF	Reserved	
0x4000 5800 - 0x4000 5BFF	I2C2	
0x4000 5400 - 0x4000 57FF	I2C1	
0x4000 4C00 - 0x4000 53FF	Reserved	
0x4000 4800 - 0x4000 4BFF	USART3	
0x4000 4400 - 0x4000 47FF	USART2	
0x4000 3C00 - 0x4000 3FFF	Reserved	
0x4000 3800 - 0x4000 3BFF	SPI2	
0x4000 3400 - 0x4000 37FF	Reserved	
0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)	
0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)	
0x4000 2800 - 0x4000 2BFF	RTC	
0x4000 1800 - 0x4000 27FF	Reserved	
0x4000 1400 - 0x4000 17FF	TIM7 timer	
0x4000 1000 - 0x4000 13FF	TIM6 timer	
0x4000 0C00 - 0x4000 0FFF	Reserved	
0x4000 0800 - 0x4000 0BFF	TIM4 timer	
0x4000 0400 - 0x4000 07FF	TIM3 timer	
0x4000 0000 - 0x4000 03FF	TIM2 timer	

Memory Organization

- Embedded SRAM
  - The STM32F100xx features up to 32 Kbytes of static SRAM. It can be accessed as bytes, half-words (16 bits) or full words (32 bits). The SRAM start address is 0x2000 0000
- Bit banding
  - The Cortex™-M3 memory map includes two bit-band regions. These regions map each word in an alias region of memory to a bit in a bit-band region of memory. Writing to a word in the alias region has the same effect as a read-modify-write operation on the targeted bit in the bit-band region.
  - In the STM32F100xx, both peripheral registers and SRAM are mapped in a bit-band region. This allows single bit-band write and read operations to be performed.

Memory Organization

- Bit-Banding Mapping formula:

  bit_word_addr = bit_band_base + (byte_offset x 32) + (bit_number × 4)

  - *bit_word_addr* is the address of the word in the alias memory region that maps to the targeted bit
  - *bit_band_base* is the starting address of the alias region
  - *byte_offset* is the number of the byte in the bit-band region that contains the targeted bit
  - *bit_number* is the bit position (0-7) of the targeted bit

Memory Organization

- Bit Banding Example
  - Mapping bit 2 of the byte located at SRAM address 0x2000 0300 in the alias region is done as follows:

    0x2200 6008 = 0x2200 0000 + (0x300*32) + (2*4).

  - Writing to address 0x2200 6008 has the same effect as a read-modify-write operation on bit 2 of the byte at SRAM address 0x2000 0300
  - Reading address 0x2200 6008 returns the value (0x01 or 0x00) of bit 2 of the byte at SRAM address 0x2000 0300 (0x01: bit set; 0x00: bit cleared)

Memory Organization

- Embedded Flash memory organization

Block	Name	Base addresses	Size (bytes)
Main memory	Page 0	0x0800 0000 - 0x0800 03FF	1 Kbyte
	Page 1	0x0800 0400 - 0x0800 07FF	1 Kbyte
	Page 2	0x0800 0800 - 0x0800 0BFF	1 Kbyte
	Page 3	0x0800 0C00 - 0x0800 0FFF	1 Kbyte
	Page 4	0x0800 1000 - 0x0800 13FF	1 Kbyte
	.	.	.
	.	.	.
	.	.	.
	Page 31	0x0800 7C00 - 0x0800 8000	1 Kbyte
Information block	System memory	0x1FFF F000 - 0x1FFF F7FF	2 Kbytes
	Option Bytes	0x1FFF F800 - 0x1FFF F80F	16
Flash memory interface registers	FLASH_ACR	0x4002 2000 - 0x4002 2003	4
	FLASH_KEYR	0x4002 2004 - 0x4002 2007	4
	FLASH_OPTKEYR	0x4002 2008 - 0x4002 200B	4
	FLASH_SR	0x4002 200C - 0x4002 200F	4
	FLASH_CR	0x4002 2010 - 0x4002 2013	4
	FLASH_AR	0x4002 2014 - 0x4002 2017	4
	Reserved	0x4002 2018 - 0x4002 201B	4
	FLASH_OBR	0x4002 201C - 0x4002 201F	4
	FLASH_WRPR	0x4002 2020 - 0x4002 2023	4

Boot Configuration

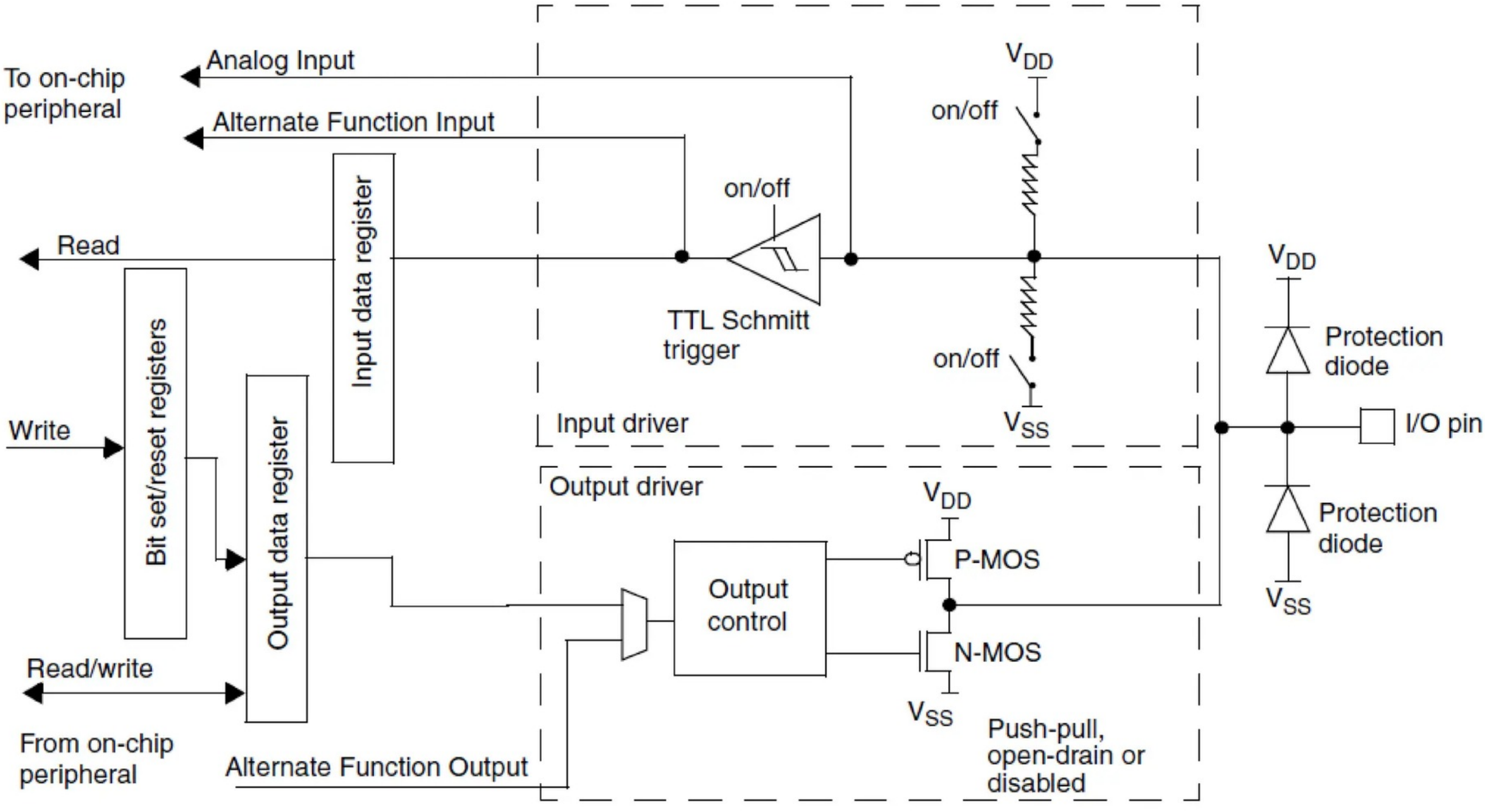
- Values on the BOOT pins are latched on the 4th rising edge of SYSCLK after a reset
  - It is up to the application to set the BOOT1 and BOOT0 pins after reset to select the required boot mode.
  - BOOT pins also resampled when exiting the Standby mode and hence must be kept in the required boot mode in the Standby mode
  - After this startup delay has elapsed, the CPU fetches the top-of-stack value from address 0x0000 0000, then starts code execution from the boot memory starting from 0x0000 0004.

Boot mode selection pins		Boot mode	Aliasing
BOOT1	BOOT0		
x	0	Main Flash memory	Main Flash memory is selected as the boot space
0	1	System memory	System memory is selected as the boot space
1	1	Embedded SRAM	Embedded SRAM is selected as the boot space

General-Purpose I/Os (GPIOs)

- Each of the general-purpose I/O ports has:
  - Two 32-bit configuration registers (GPIOx_CRL, GPIOx_CRH)
  - Two 32-bit data registers (GPIOx_IDR, GPIOx_ODR)
  - 32-bit set/reset register (GPIOx_BSRR)
  - 16-bit reset register (GPIOx_BRR)
  - 32-bit locking register (GPIOx_LCKR)
- Each port bit of GPIOs can be individually configured by software in several modes:
  - Input floating
  - Input pull-up
  - Input-pull-down
  - Analog
  - Output open-drain
  - Output push-pull
  - Alternate function push-pull
  - Alternate function open-drain

Basic Structure of GPIO Bit



GPIO Bit Configuration Table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register
General purpose output	Push-pull	0	0	01 10 11 See Table on following page		0 or 1
	Open-drain		1			0 or 1
Alternate Function output	Push-pull	1	0			don't care
	Open-drain		1			don't care
Input	Analog	0	0	00		don't care
	Input floating		1			don't care
	Input pull-down	1	0			0
	Input pull-up					1

- Note: During and just after reset, the alternate functions are not active and the I/O ports are configured in Input Floating mode (CNFx[1:0]=01b, MODEx[1:0]=00b)

GPIO Configuration: Output MODE Bits

MODE[1:0]	Meaning
00	Reserved
01	Max. output speed 10 MHz
10	Max. output speed 2 MHz
11	Max. output speed 50 MHz

GPIO Operation

- During and just after reset, the alternate functions are not active and the I/O ports are configured in Input Floating mode (CNFx[1:0]=01b, MODEx[1:0]=00b)
- When configured as output, the value written to the Output Data register (GPIOx_ODR) is output on the I/O pin.
  - It is possible to use the output driver in Push-Pull mode or Open-Drain mode (only the N-MOS is activated when outputting 0).
- The Input Data register (GPIOx_IDR) captures the data present on the I/O pin at every APB2 clock cycle
- All GPIO pins have an internal weak pull-up and weak pull-down which can be activated or not when configured as input

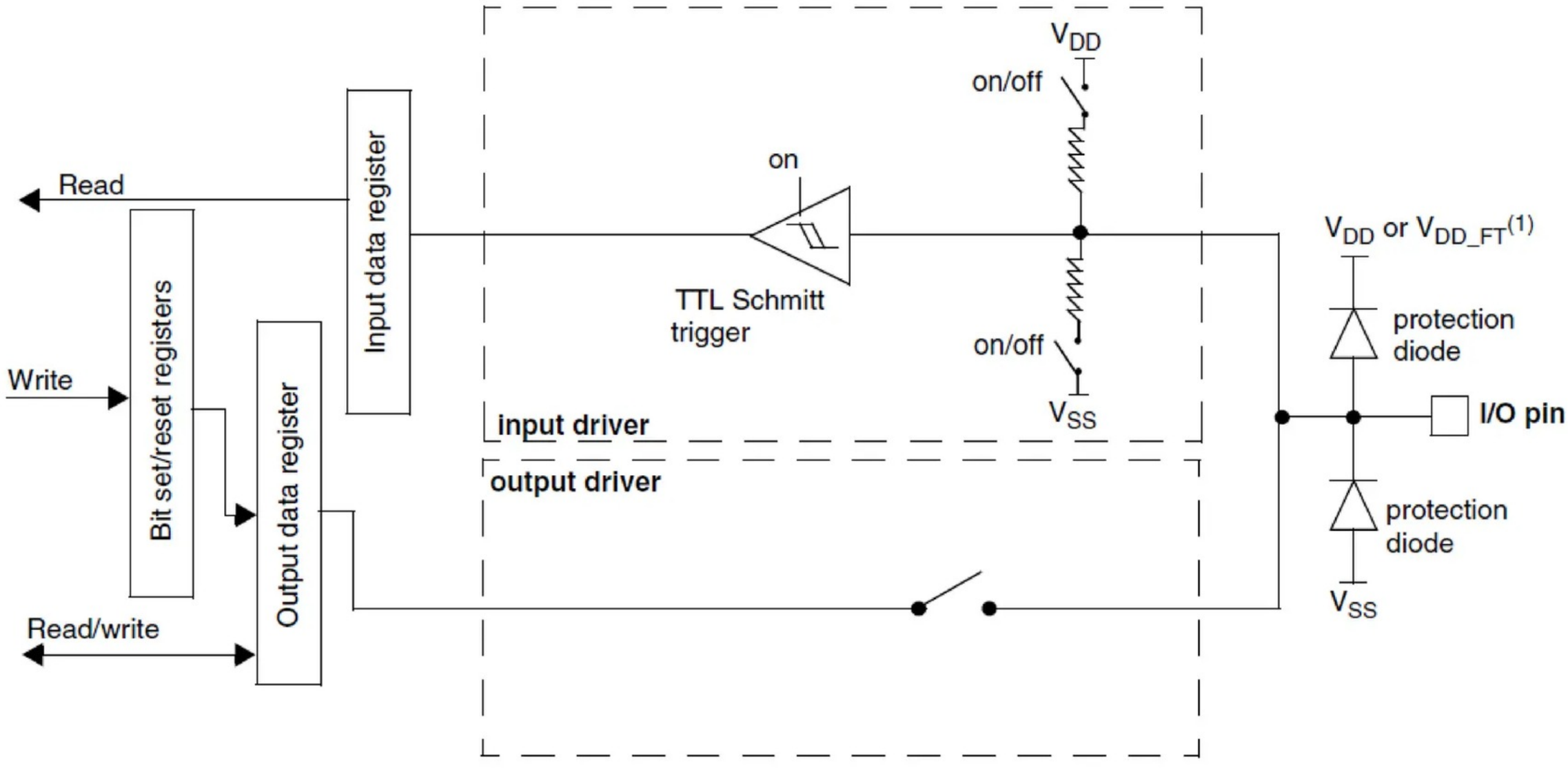
GPIO Atomic Bit Set or Reset

- Atomic Read/Modify access
  - No interruption in the middle to cause errors
- Atomic operations ensure that the desired change is not interrupted resulting in partial set/reset of GPIOs
- There is no need for the software to disable interrupts when programming the GPIOx_ODR at bit level: it is possible to modify only one or several bits in a single atomic APB2 write access
- This is achieved by programming to '1' the Bit Set/Reset Register (GPIOx_BSRR, or for reset only GPIOx_BRR) to select the bits you want to modify.
  - Unselected bits will not be modified

Input Configuration

- When the I/O Port is programmed as Input:
  - The Output Buffer is disabled
  - The Schmitt Trigger Input is activated
  - The weak pull-up and pull-down resistors are activated or not depending on input configuration (pull-up, pull-down or floating)
  - The data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle
  - A read access to the Input Data Register obtains the I/O State

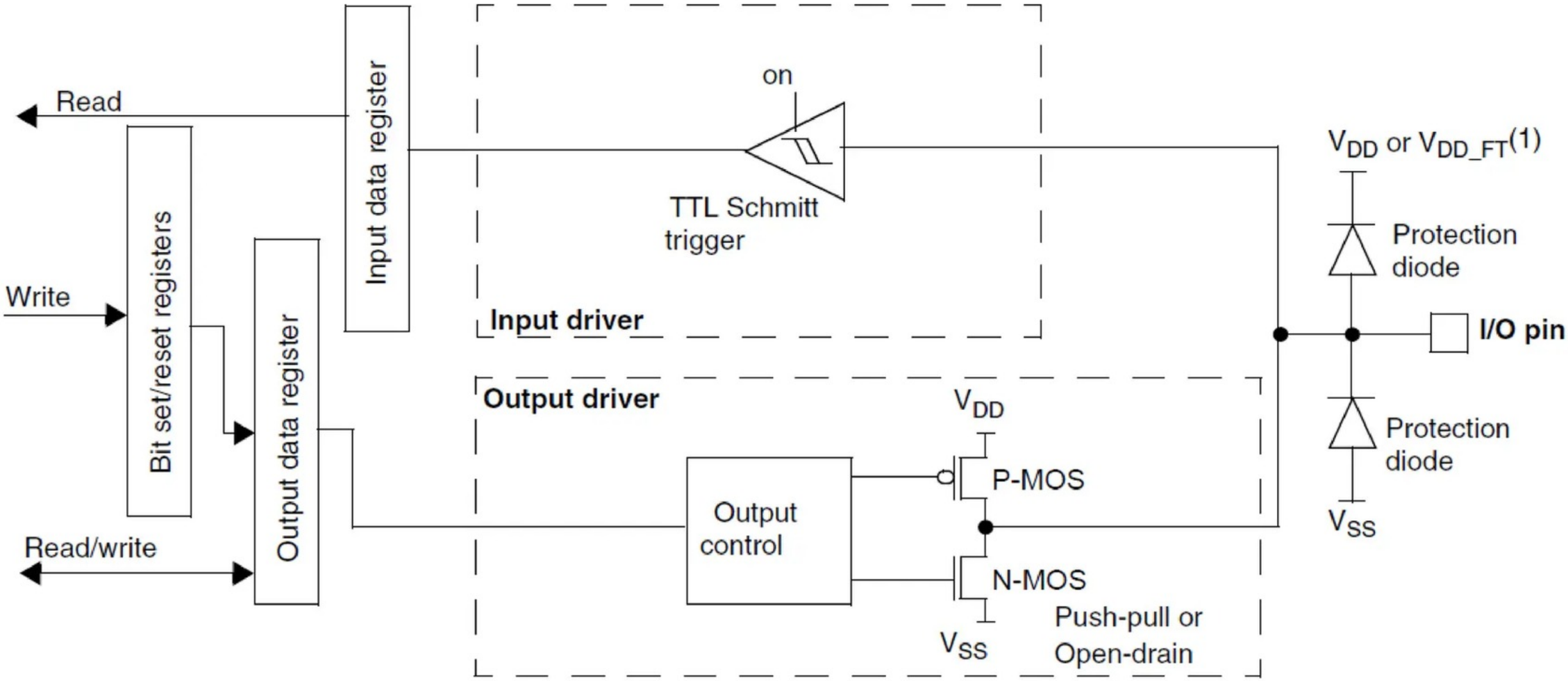
Input Configuration



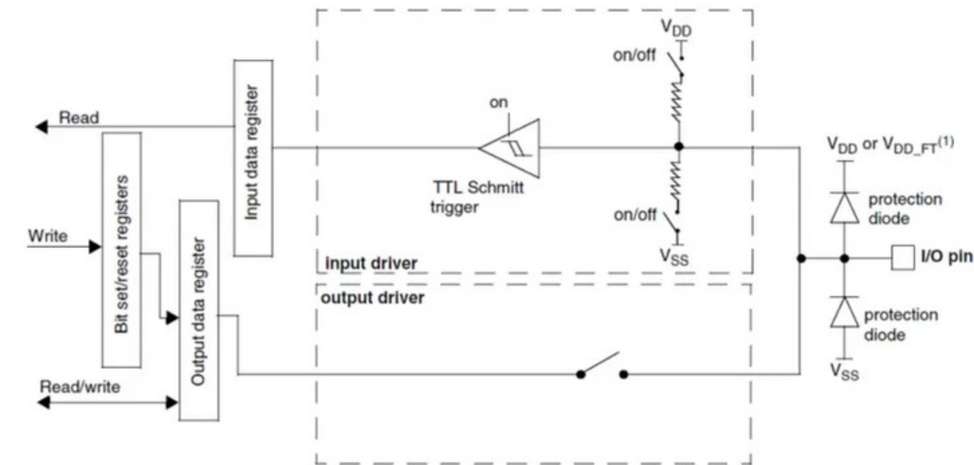
Output Configuration

- When the I/O Port is programmed as Output:
  - The Output Buffer is enabled:
    - Open Drain Mode: A “0” in the Output register activates the N-MOS while a “1” in the Output register leaves the port in Hi-Z. (the P-MOS is never activated)
    - Push-Pull Mode: A “0” in the Output register activates the N-MOS while a “1” in the Output register activates the P-MOS
  - The Schmitt Trigger Input is activated.
  - The weak pull-up and pull-down resistors are disabled.
  - The data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle
  - Read access to Input Data Register gets the I/O state in open drain mode
  - Read access to Output Data register gets last written value in Push-Pull mode

Output Configuration



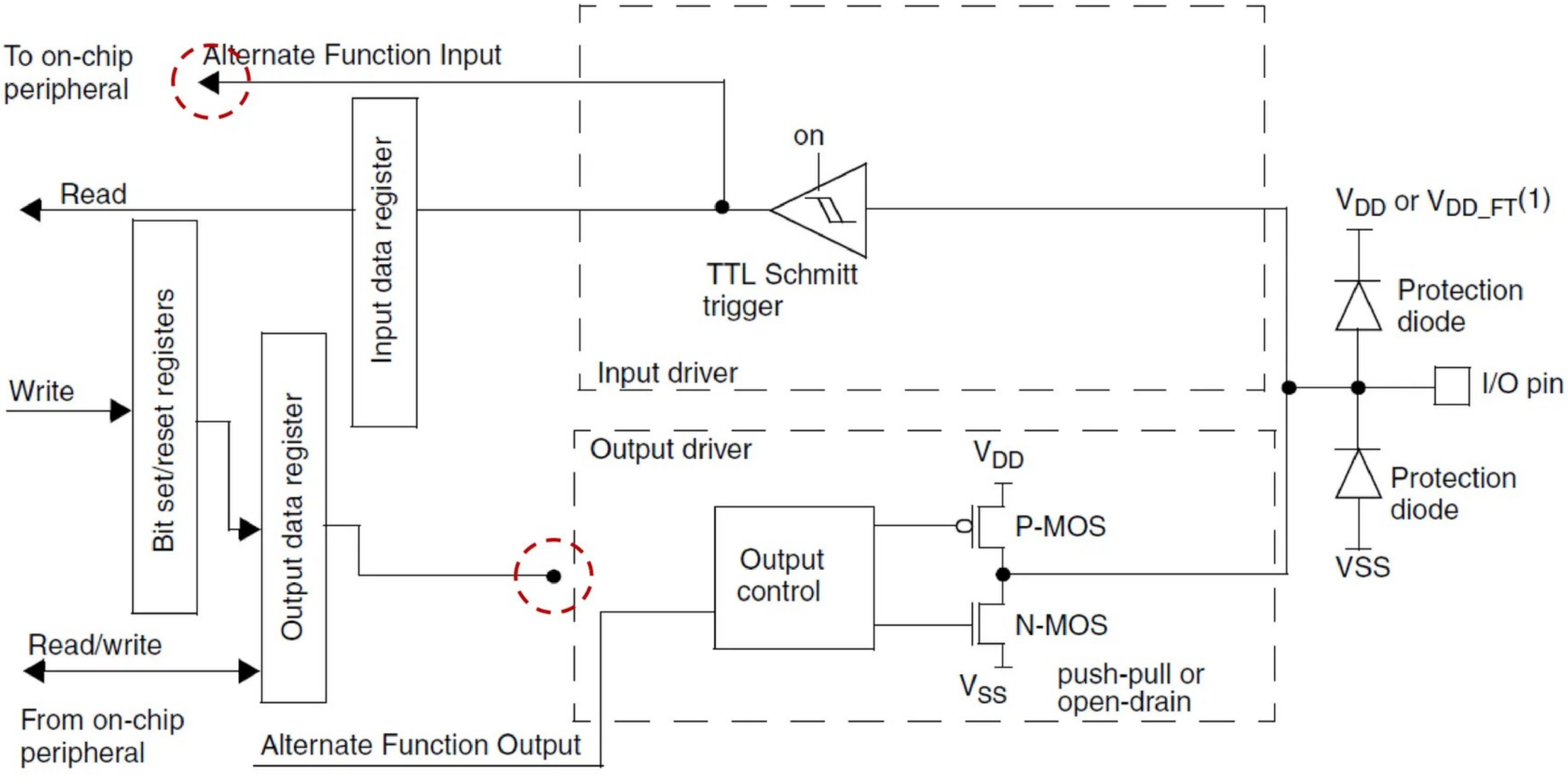
Compare to input configuration



Alternate Function Configuration

- When the I/O Port is programmed as Alternate Function:
  - The Output Buffer is turned on in Open Drain or Push-Pull configuration
  - The Output Buffer is driven by the signal coming from the peripheral (alternate function out)
  - The Schmitt Trigger Input is activated
  - The weak pull-up and pull-down resistors are disabled
  - The data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle
  - A read access to the Input Data Register gets the I/O state in open drain mode
  - A read access to the Output Data register gets the last written value in Push-Pull mode

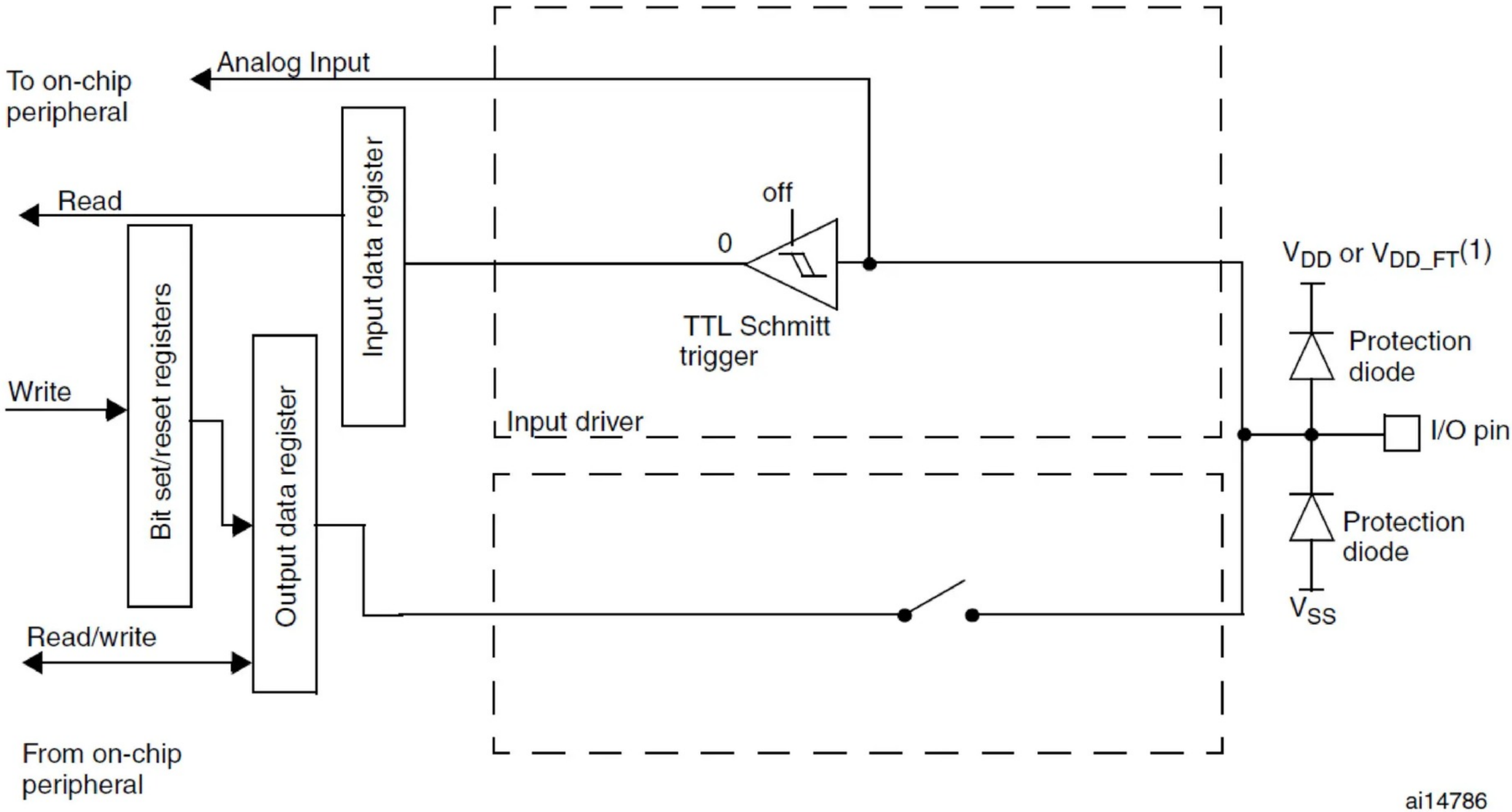
Alternate Function Configuration



Analog Configuration

- When the I/O Port is programmed as Analog configuration:
  - The Output Buffer is disabled.
  - The Schmitt Trigger Input is de-activated providing zero consumption for every analog value of the I/O pin. The output of the Schmitt Trigger is forced to a constant value (0).
  - The weak pull-up and pull-down resistors are disabled.
  - Read access to the Input Data Register gets the value “0”.

Analog Configuration



GPIO Registers

7.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

7.2.2 Port configuration register high (GPIOx_CRH) (x=A..G)

Address offset: 0x04

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

GPIO Registers

7.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Address offset: 0x08h

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

7.2.4 Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

GPIO Registers

7.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

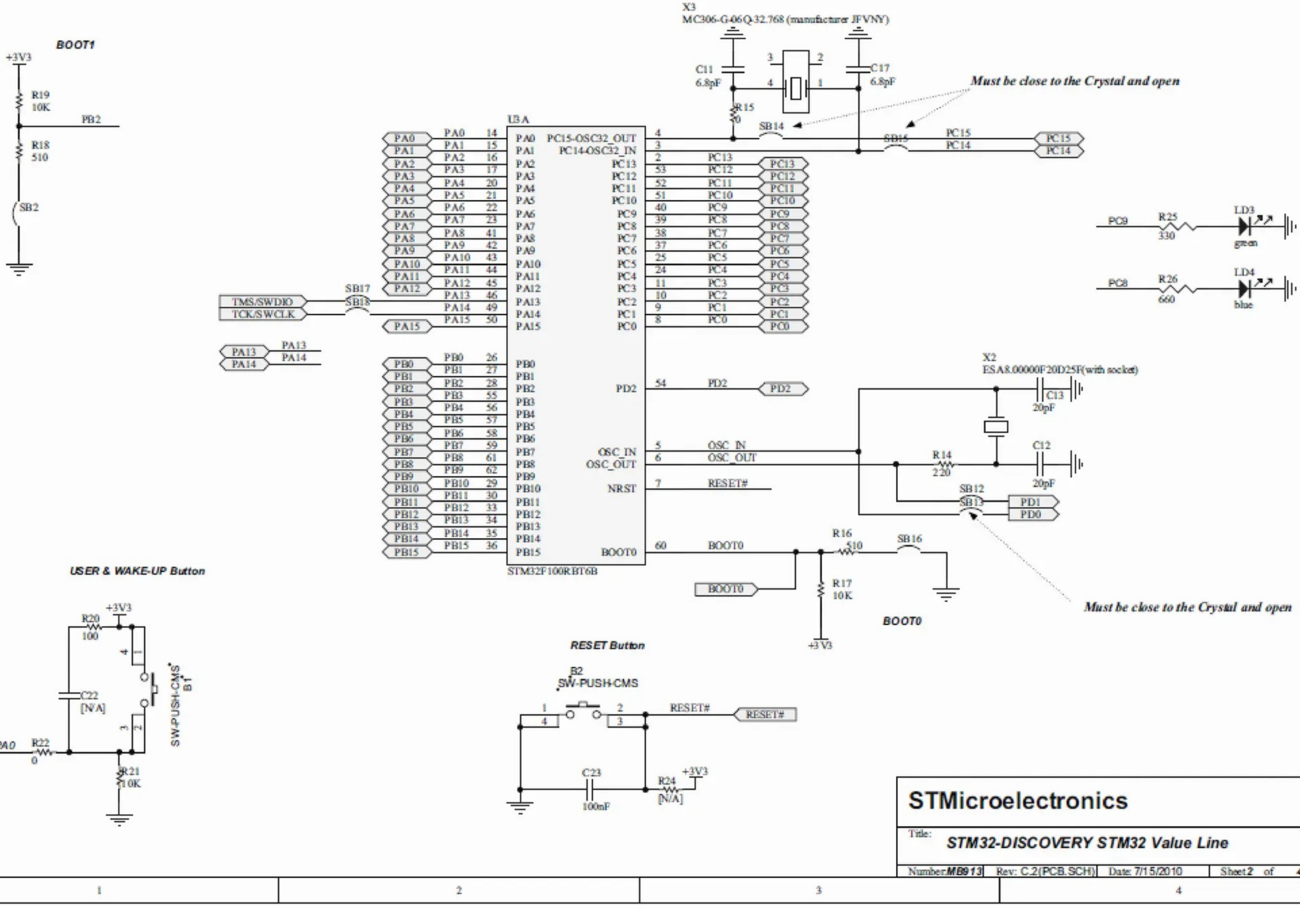
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

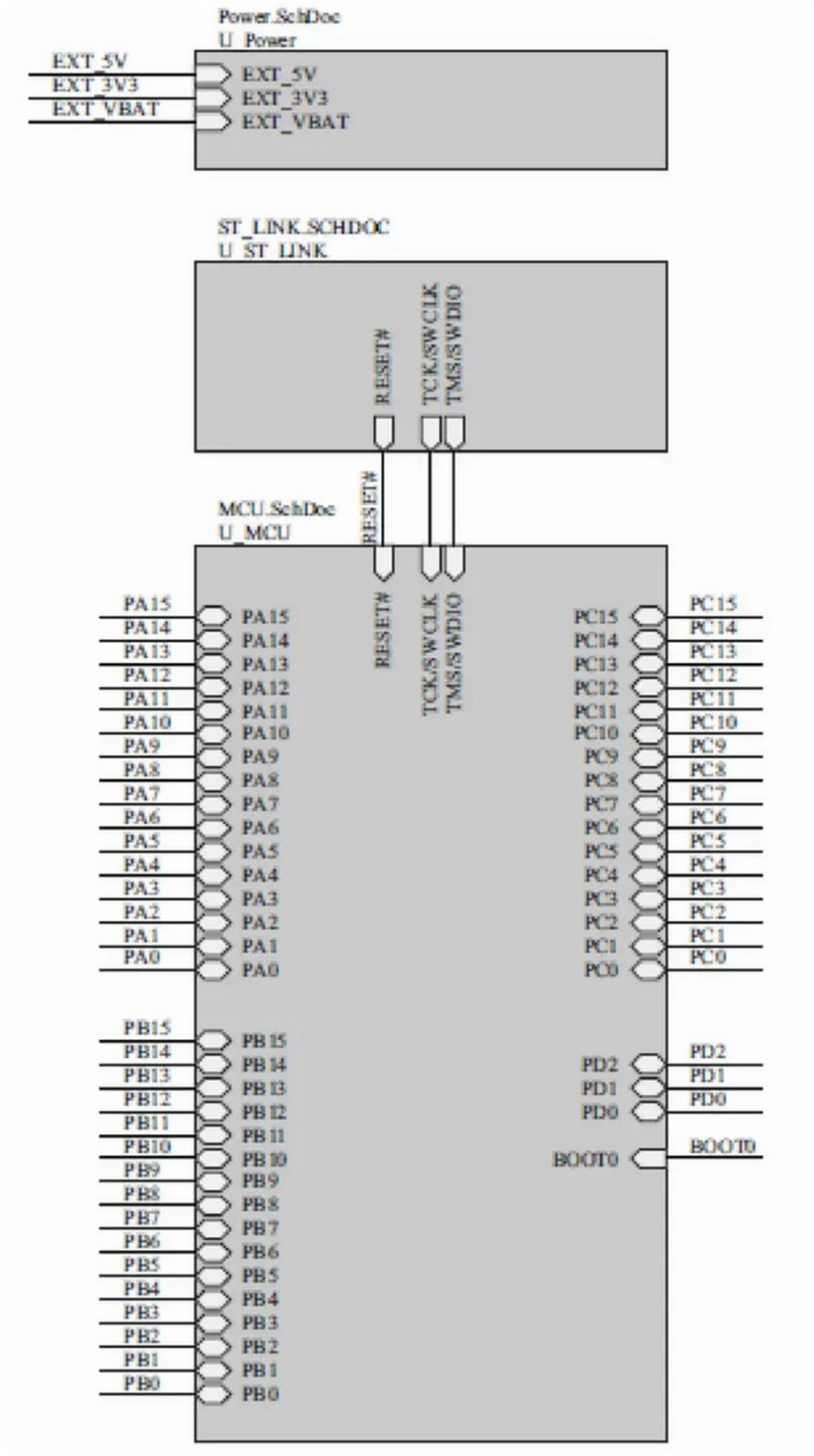
7.2.6 Port bit reset register (GPIOx_BRR) (x=A..G)

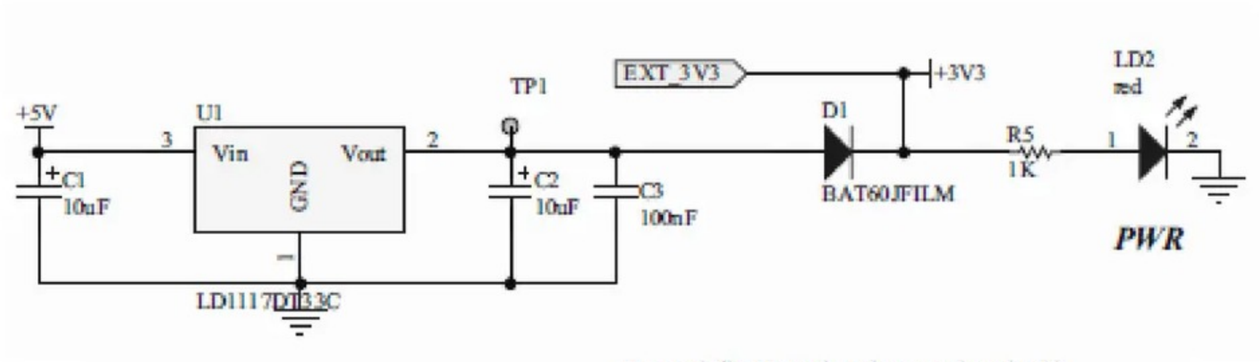
Address offset: 0x14

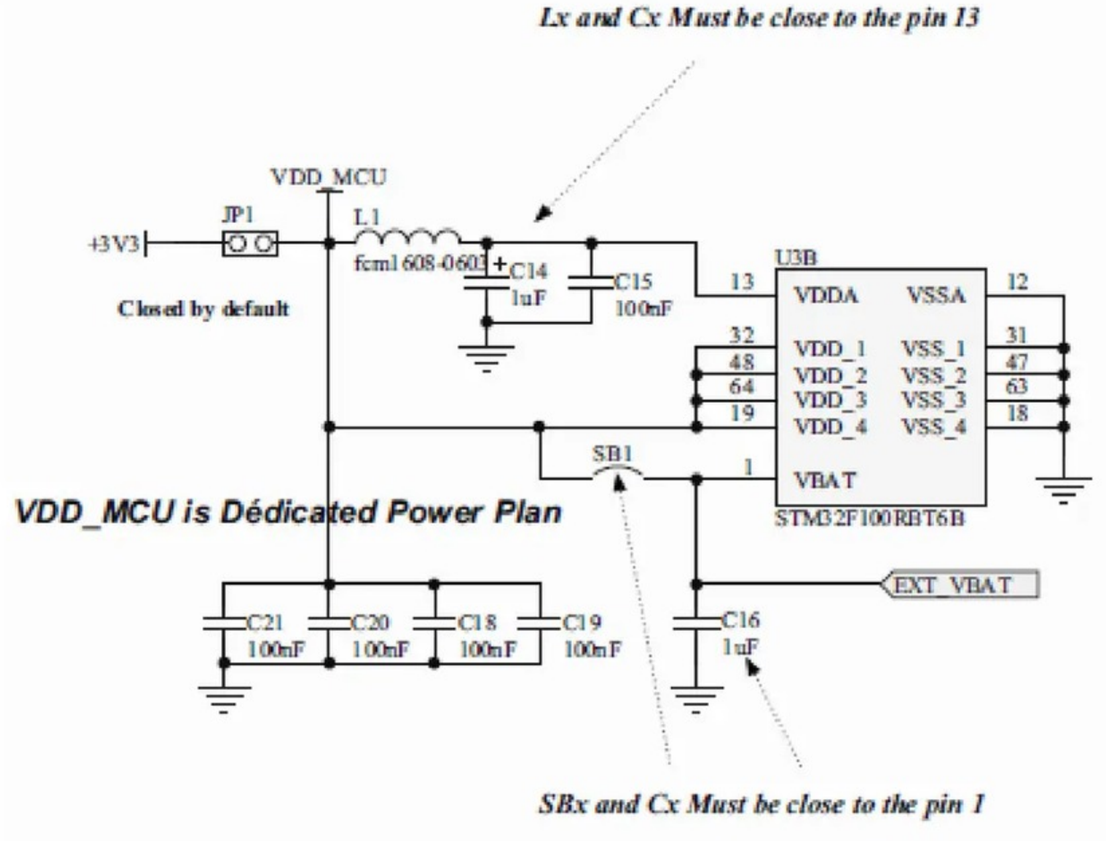
Reset value: 0x0000 0000

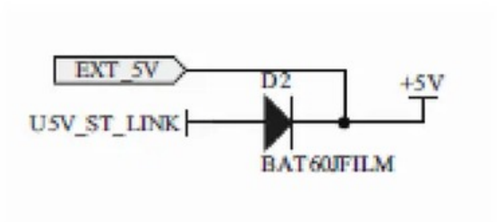
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

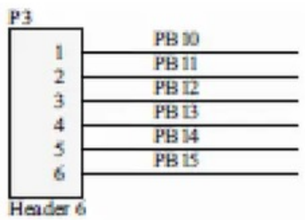

BOOT1
+3V3
R19
10K
PB2
R18
510
SB2
X3
MC306-G-06Q-32.768 (manufacturer JFVNY)
C11
6.8pF
C17
6.8pF
R15
0
Must be close to the Crystal and open
SB14
SB15
PC15
PC14
U3A
PA0
PA1
PA2
PA3
PA4
PA5
PA6
PA7
PA8
PA9
PA10
PA11
PA12
PA13
PA14
PA15
PB0
PB1
PB2
PB3
PB4
PB5
PB6
PB7
PB8
PB9
PB10
PB11
PB12
PB13
PB14
PB15
PC15-OSC32_OUT
PC14-OSC32_IN
PC13
PC12
PC11
PC10
PC9
PC8
PC7
PC6
PC5
PC4
PC3
PC2
PC1
PC0
PD2
OSC_IN
OSC_OUT
NRST
BOOT0
STM32F100RBT6B
SB17
SB18
TMS/SWDIO
TCK/SWCLK
LD3
green
LD4
blue
R25
330
R26
660
X2
ESA8.00000F20D25F(with socket)
C13
20pF
C12
20pF
R14
220
SB12
SB13
R16
510
SB16
R17
10K
BOOT0
RESET#
USER & WAKE-UP Button
R20
100
C22
[N/A]
R22
0
R21
10K
SW-PUSH-CMS
B1
RESET Button
B2
C23
100nF
R24
[N/A]
Must be close to the Crystal and open
STMicroelectronics
Title:
STM32-DISCOVERY STM32 Value Line
Number:MB913
Rev: C.2(PCB.SCH)
Date: 7/15/2010
Sheet 2 of 4

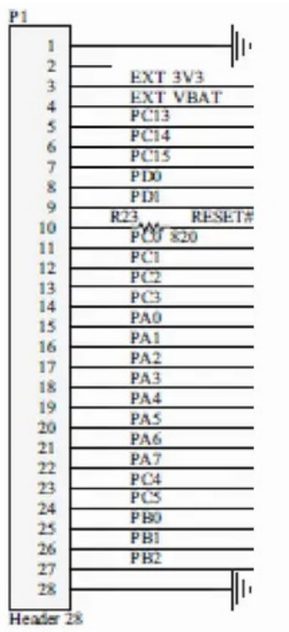

Power.SchDoc
U_Power
EXT_5V
EXT_3V3
EXT_VBAT
ST_LINK.SCHDOC
U_ST_LINK
RESET#
TCK/SWCLK
TMS/SWDIO
MCU.SchDoc
U_MCU
PA15 PA14 PA13 PA12 PA11 PA10 PA9 PA8 PA7 PA6 PA5 PA4 PA3 PA2 PA1 PA0
PB15 PB14 PB13 PB12 PB11 PB10 PB9 PB8 PB7 PB6 PB5 PB4 PB3 PB2 PB1 PB0
PC15 PC14 PC13 PC12 PC11 PC10 PC9 PC8 PC7 PC6 PC5 PC4 PC3 PC2 PC1 PC0
PD2 PD1 PD0
BOOT0

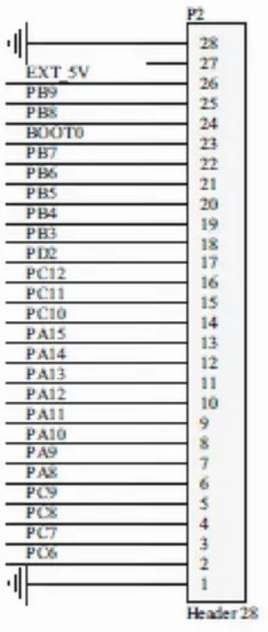

+5V
U1
Vin
GND
Vout
LD1117DT33C
C1
10uF
C2
10uF
C3
100nF
TP1
EXT_3V3
+3V3
D1
BAT60JFILM
R5
1K
LD2
red
PWR


Lx and Cx Must be close to the pin 13
VDD_MCU
JP1
+3V3
Closed by default
L1
fcml608-0603
C14
1uF
C15
100nF
U3B
VDDA VSSA
VDD_1 VSS_1
VDD_2 VSS_2
VDD_3 VSS_3
VDD_4 VSS_4
VBAT
STM32F100RBT6B
SB1
VDD_MCU is Dédicated Power Plan
C21
100nF
C20
100nF
C18
100nF
C19
100nF
C16
1uF
EXT_VBAT
SBx and Cx Must be close to the pin 1


EXT_5V
D2
+5V
U5V_ST_LINK
BAT60JFILM

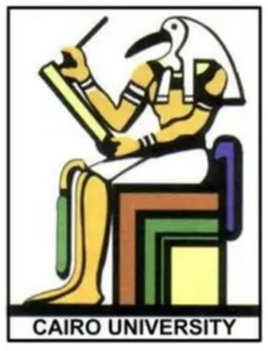

P3
PB10 PB11 PB12 PB13 PB14 PB15
Header 6


P1
EXT_3V3
EXT_VBAT
PC13 PC14 PC15 PD0 PD1
R23 820
RESET#
PC0 PC1 PC2 PC3 PA0 PA1 PA2 PA3 PA4 PA5 PA6 PA7 PC4 PC5 PB0 PB1 PB2
Header 28


P2
EXT_5V
PB9 PB8 BOOT0 PB7 PB6 PB5 PB4 PB3 PD2 PC12 PC11 PC10 PA15 PA14 PA13 PA12 PA11 PA10 PA9 PA8 PC9 PC8 PC7 PC6
Header 28

Assignments

- ARM Project #1 (To start this week)



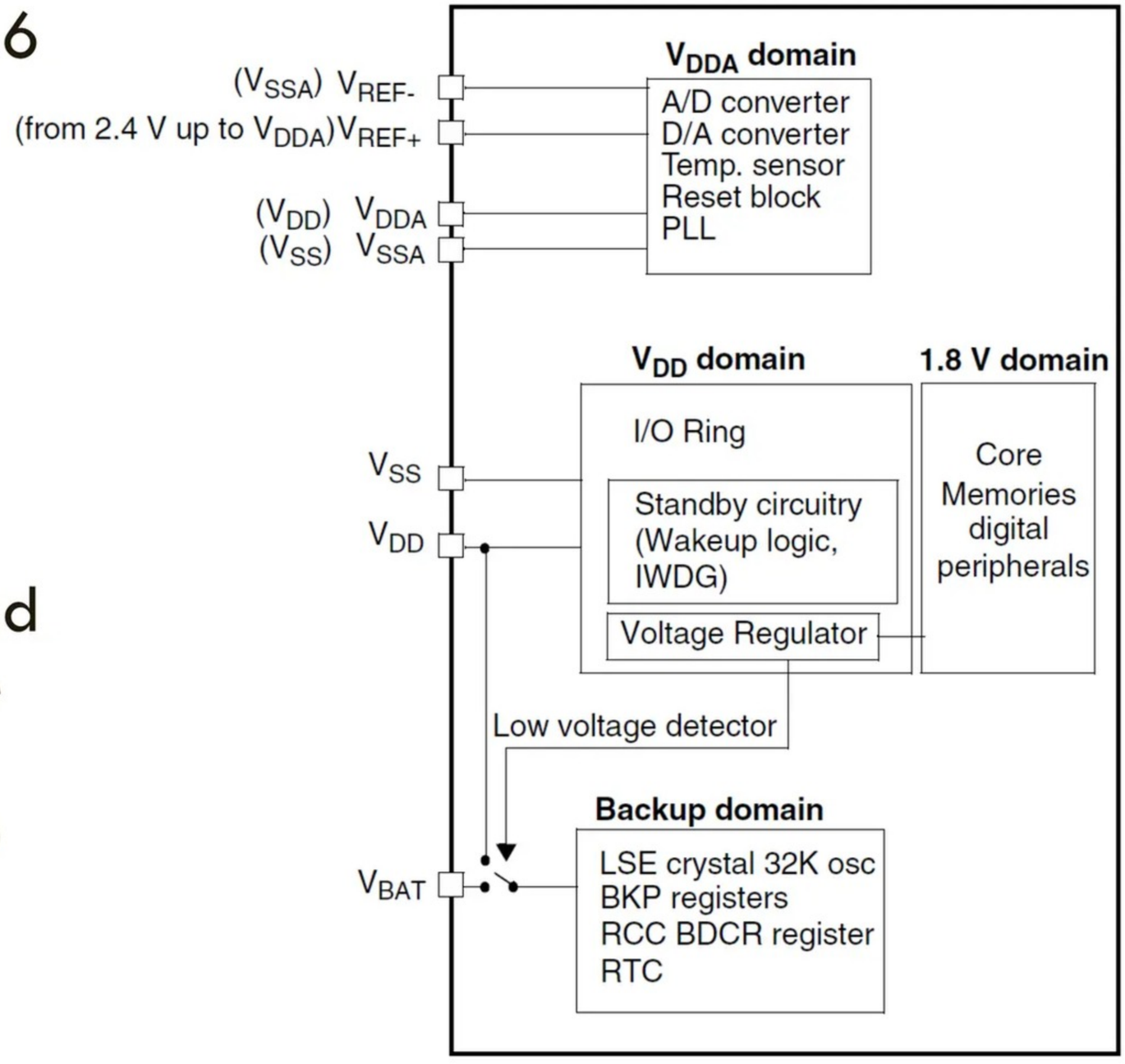
STM32 MICROCONTROLLER

Lecture 3

Prof. Yasser Mostafa Kadah

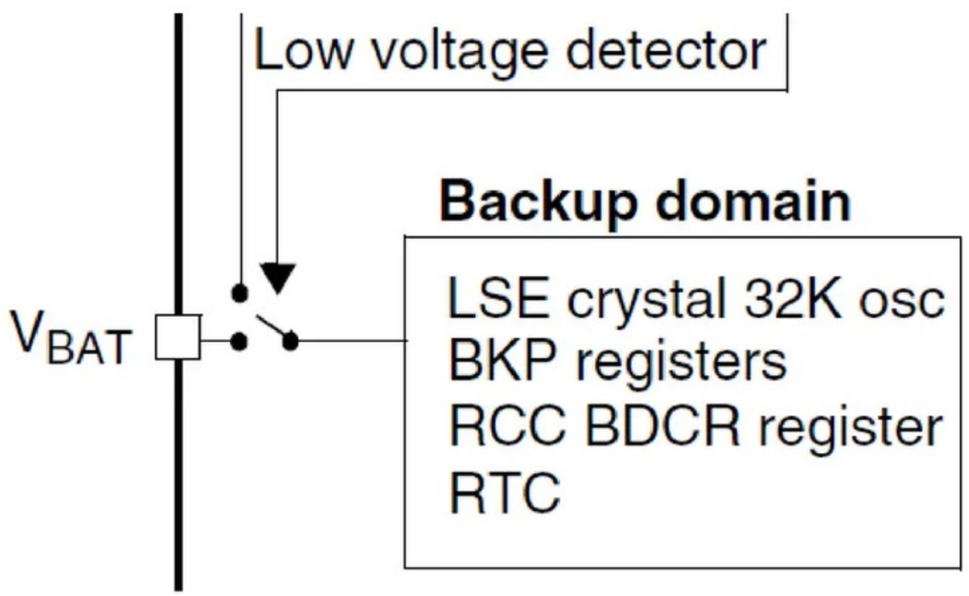
Power Supplies

- Device requires a 2.0:3.6 V operating voltage supply (VDD)
- Embedded regulator is used to supply the internal 1.8 V digital power
- Real-time clock (RTC) and backup registers can be powered from VBAT voltage when main VDD supply is powered off



Battery Backup Domain

- To retain the content of the Backup registers and supply the RTC function when VDD is turned off, VBAT pin can be connected to an optional standby voltage supplied by a battery or by another source
  - The VBAT pin powers the RTC unit, the LSE oscillator and the PC13 to PC15 IOs, allowing the RTC to operate even when the main digital supply (VDD) is turned off
- The switch to the VBAT supply is controlled by the Power Down Reset embedded in the Reset block

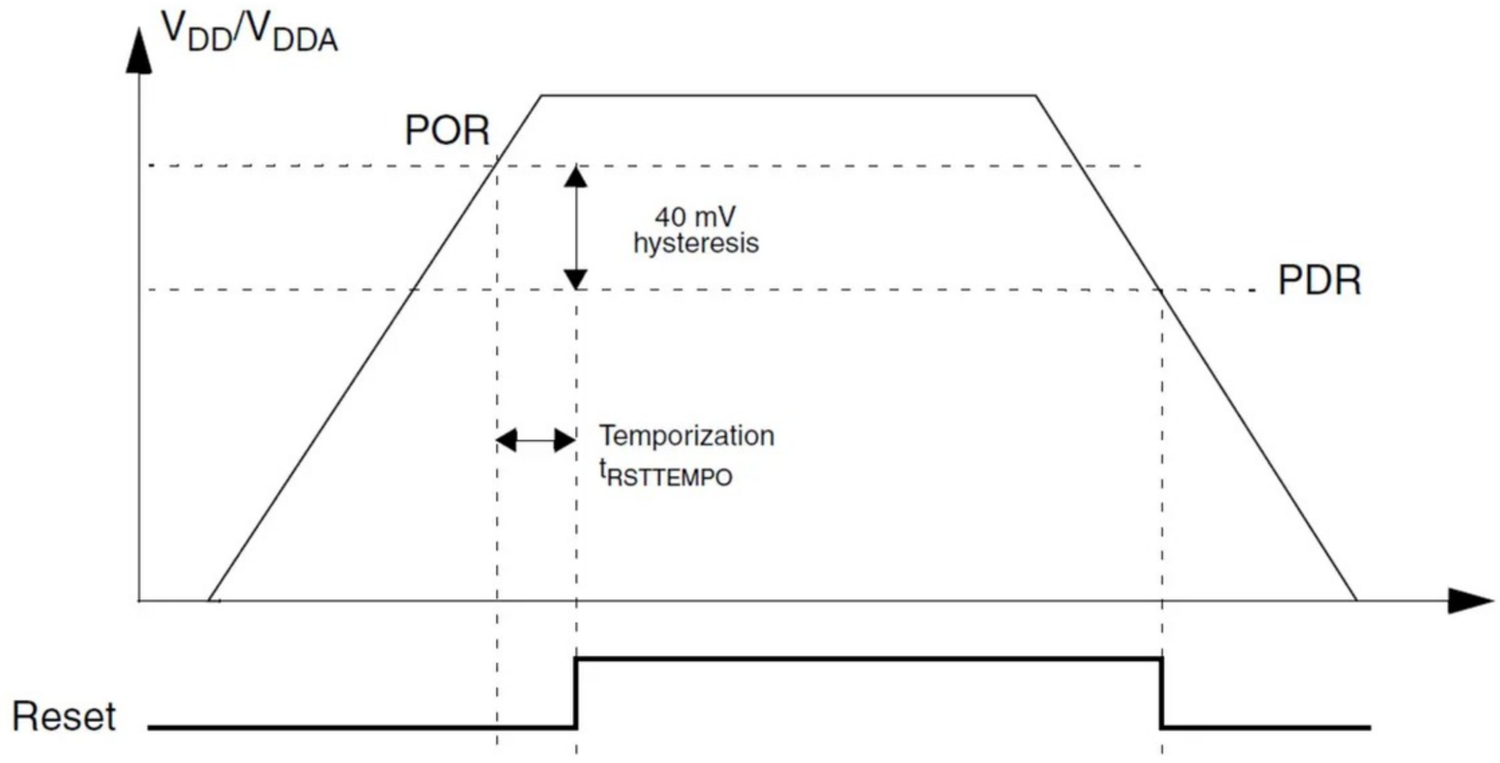


Voltage Regulator

- The voltage regulator is always enabled after Reset and works in three different modes depending on the application modes:
  - ***Run mode:*** the regulator supplies full power to the 1.8 V domain (core, memories and digital peripherals)
  - ***Stop mode:*** the regulator supplies low-power to the 1.8 V domain, preserving contents of registers and SRAM
  - ***Standby Mode:*** the regulator is powered off. The contents of the registers and SRAM are lost except for the Standby circuitry and the Backup Domain

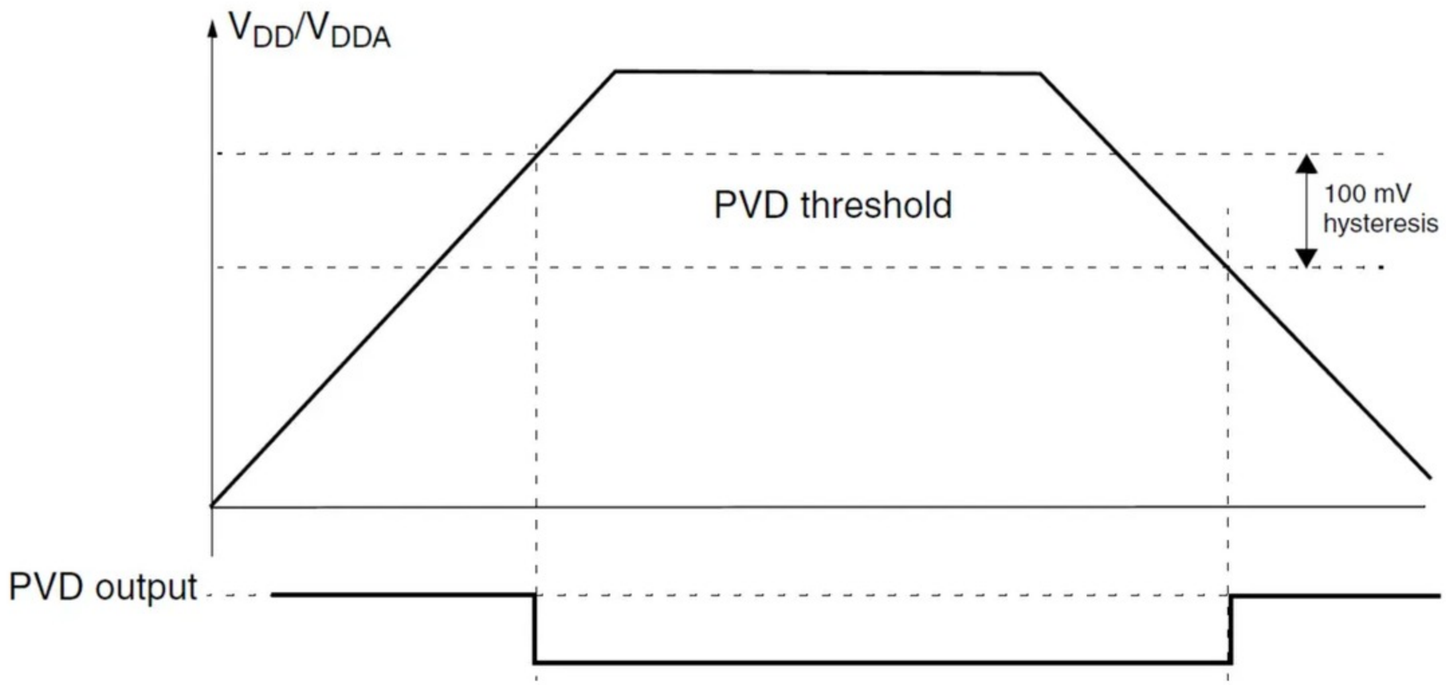
Power Supply Supervisor

- Power on reset (POR)/power down reset (PDR)
  - Integrated POR/PDR circuitry that allows proper operation starting from/down to 2 V
  - Device remains in Reset mode when V_{DD}/V_{DDA} is below a specified threshold, $V_{POR/PDR}$, without the need for an external reset circuit



Power Supply Supervisor

- Programmable voltage detector (PVD)
  - Monitors the VDD/VDDA power supply by comparing it to a threshold selected by the PLS[2:0] bits in the *Power control register (PWR_CR)*
  - PVD is enabled by setting the PVDE bit
  - A PVDO flag is available, in *Power control/status register (PWR_CSR)*, to indicate if V_{DD}/V_{DDA} is higher or lower than the PVD threshold
  - This event is internally connected to EXTI and can generate interrupt



Low-Power Modes

- By default, microcontroller is in Run mode after a Reset
- Low power modes are available to save power when the CPU need not to be kept running (e.g., while waiting for an external event)
- It is up to the user to select the mode that gives the best compromise between low-power consumption, short startup time and available wakeup sources.
- The STM32F100xx devices feature three low-power modes:
  - Sleep mode (CPU clock off, all peripherals including Cortex-M3 core peripherals like NVIC, SysTick, etc. are kept running)
  - Stop mode (all clocks are stopped)
  - Standby mode (1.8V domain powered-off)
- In addition, power consumption in Run mode can be reduced by:
  - Slowing down the system clocks
  - Gating the clocks to the APB and AHB peripherals when they are unused.

Low-Power Modes

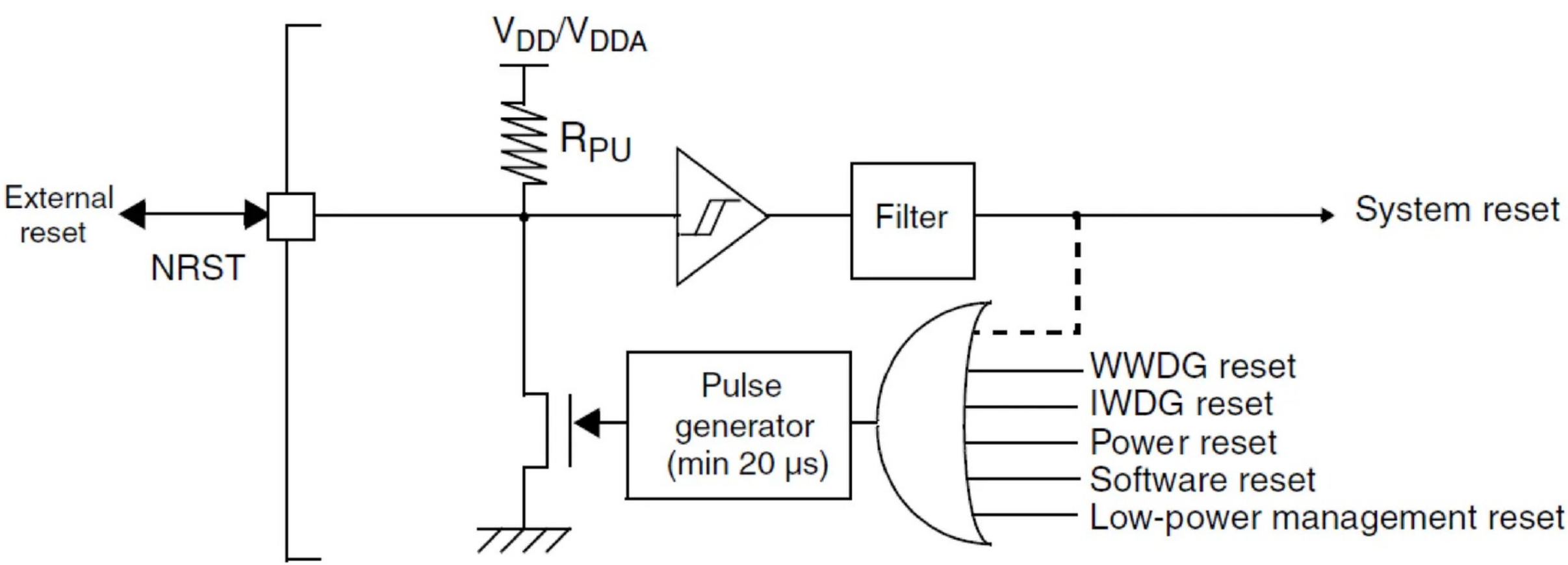
Mode name	Entry	wakeup	Effect on 1.8V domain clocks	Effect on V_{DD} domain clocks	Voltage regulator
Sleep (Sleep now or Sleep-on - exit)	WFI	Any interrupt	CPU clock OFF no effect on other clocks or analog clock sources	None	ON
	WFE	Wakeup event			
Stop	PDDS and LPDS bits + SLEEPDEEP bit + WFI or WFE	Any EXTI line (configured in the EXTI registers)	All 1.8V domain clocks OFF	HSI and HSE oscillators OFF	ON or in low-power mode (depends on *Power control register (PWR_CR)*)
Standby	PDDS bit + SLEEPDEEP bit + WFI or WFE	WKUP pin rising edge, RTC alarm, external reset in NRST pin, IWDG reset			OFF

Backup Registers (BKP)

- Backup registers are ten 16-bit registers in low and medium density devices for storing 20 bytes of user application data
- They are implemented in the backup domain that remains powered on by VBAT when the VDD power is switched off.
  - They are not reset when the device wakes up from Standby mode or by a system reset or power reset
- In addition, the BKP control registers are used to manage the Tamper detection feature and RTC calibration.
- After reset, access to the Backup registers and RTC is disabled and the Backup domain (BKP) is protected against possible parasitic write access
- To enable access to the Backup registers and the RTC, proceed as follows:
  - Enable the power and backup interface clocks by setting the PWREN and BKPEN bits in the RCC_APB1ENR register
  - Set the DBP bit the Power Control Register (PWR_CR) to enable access to the Backup registers and RTC.

Reset

- There are three types of reset:
  - System Reset
  - Power Reset
  - Backup domain reset



System Reset

- A system reset sets all registers to their reset values except the reset flags in the clock controller CSR register and the registers in the Backup domain
- A system reset is generated when one of the following events occurs:
  - Low level on the NRST pin (external reset)
  - Window watchdog end of count condition (WWDG reset)
  - Independent watchdog end of count condition (IWDG reset)
  - Software reset (SW reset)
  - Low-power management reset

System Reset

- Software reset
  - The SYSRESETREQ bit in Cortex™-M3 Application Interrupt and Reset Control Register must be set to force a software reset on the device
- Low-power management reset:
  - Method1: *Reset generated when entering Standby mode*: This type of reset is enabled by resetting nRST_STDBY bit in User Option Bytes. In this case, whenever a Standby mode entry sequence is successfully executed, the device is reset instead of entering Standby mode.
  - Method 2: *Reset when entering Stop mode*: This type of reset is enabled by resetting NRST_STOP bit in User Option Bytes. In this case, whenever a Stop mode entry sequence is successfully executed, the device is reset instead of entering Stop mode.

Power Reset

- A power reset is generated when one of the following occurs:
  - Power-on/power-down reset (POR/PDR reset)
  - When exiting Standby mode
- A power reset sets all registers to their reset values except the Backup domain
  - These sources act on the NRST pin and it is always kept low during the delay phase. The
  - RESET vector is fixed at address 0x00000004 in the memory map

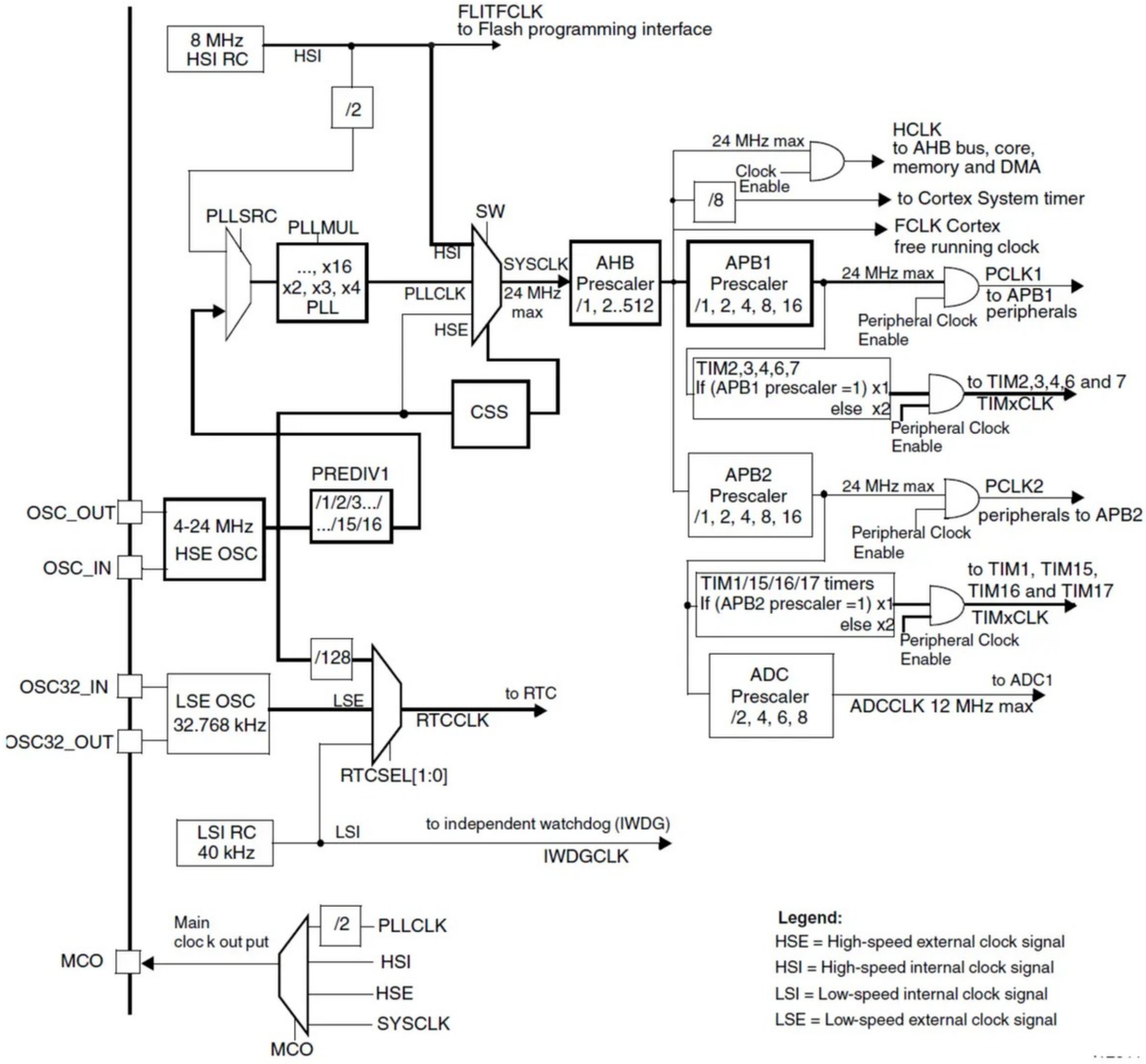
Backup Domain Reset

- Backup domain has two specific resets that affect only the backup domain
- backup domain reset is generated when one of the following events occurs:
  - Software reset, triggered by setting the BDRST bit in the *Backup domain control register (RCC_BDCR)*
  - VDD or VBAT power on, if both have previously been powered off

Clocks

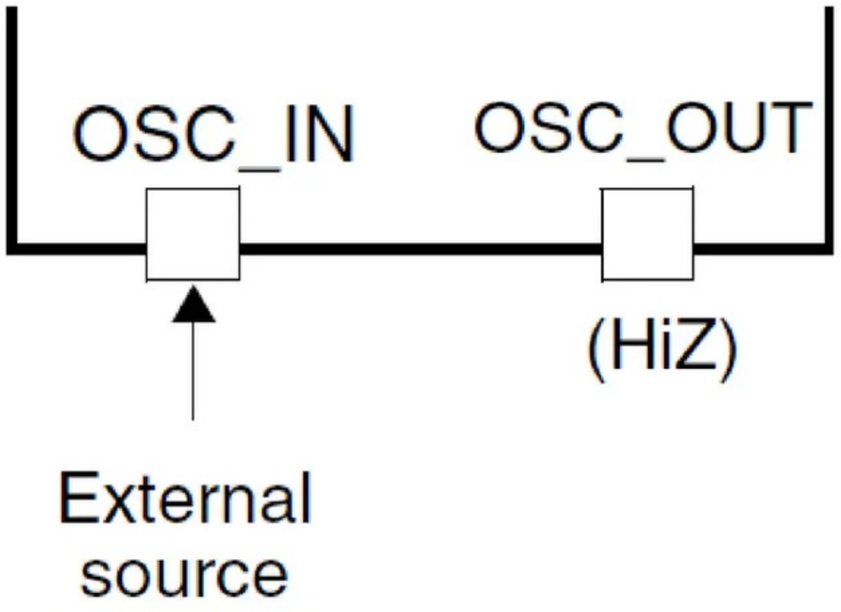
- Three different clock sources can be used to drive the system clock (SYSCLK):
  - HSI oscillator clock
  - HSE oscillator clock
  - PLL clock
- The devices have the following two secondary clock sources:
  - 40 kHz low speed internal RC (LSI RC) which drives the independent watchdog and optionally the RTC used for Auto-wakeup from Stop/Standby mode
  - 32.768 kHz low speed external crystal (LSE crystal) which optionally drives the real-time clock (RTCCLK)
- Each clock source can be switched on or off independently when it is not used, to optimize power consumption

Clock Tree

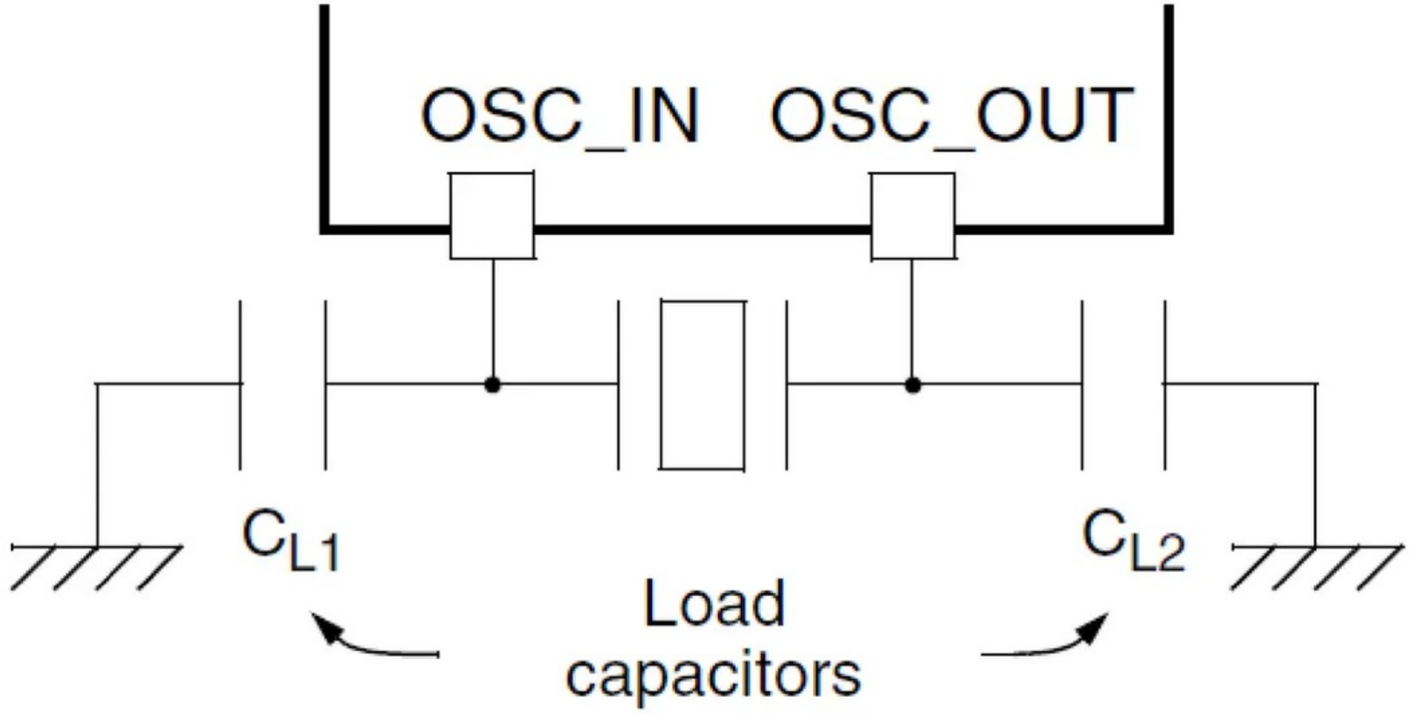

8 MHz
HSI RC
HSI
FLITFCLK
to Flash programming interface
/2
PLLSRC
PLLMUL
..., x16
x2, x3, x4
PLL
SW
HSI
PLLCLK
HSE
SYSCLK
24 MHz
max
AHB
Prescaler
/1, 2..512
CSS
24 MHz max
Clock
Enable
/8
HCLK
to AHB bus, core,
memory and DMA
to Cortex System timer
FCLK Cortex
free running clock
APB1
Prescaler
/1, 2, 4, 8, 16
24 MHz max
PCLK1
to APB1
peripherals
Peripheral Clock
Enable
TIM2,3,4,6,7
If (APB1 prescaler =1) x1
else x2
to TIM2,3,4,6 and 7
TIMxCLK
Peripheral Clock
Enable
APB2
Prescaler
/1, 2, 4, 8, 16
24 MHz max
PCLK2
peripherals to APB2
Peripheral Clock
Enable
TIM1/15/16/17 timers
If (APB2 prescaler =1) x1
else x2
to TIM1, TIM15,
TIM16 and TIM17
TIMxCLK
Peripheral Clock
Enable
ADC
Prescaler
/2, 4, 6, 8
to ADC1
ADCCLK 12 MHz max
PREDIV1
/1/2/3.../
.../15/16
OSC_OUT
OSC_IN
4-24 MHz
HSE OSC
/128
OSC32_IN
OSC32_OUT
LSE OSC
32.768 kHz
LSE
to RTC
RTCCLK
RTCSEL[1:0]
LSI RC
40 kHz
LSI
to independent watchdog (IWDG)
IWDGCLK
Main
cloc k out put
/2
PLLCLK
MCO
HSI
HSE
SYSCLK
MCO
Legend:
HSE = High-speed external clock signal
HSI = High-speed internal clock signal
LSI = Low-speed internal clock signal
LSE = Low-speed external clock signal

HSE Clock

- The high speed external clock signal (HSE) can be generated from two possible clock sources:
  - HSE external crystal/ceramic resonator
  - HSE user external clock



External clock



Crystal/Ceramic resonators

HSI Clock

- The High Speed Internal (HIS) clock signal is generated from an internal 8 MHz RC Oscillator and can be used directly as a system clock or divided by 2 to be used as PLL input
- The HSI RC oscillator has the advantage of providing a clock source at low cost (no external components)
- It also has a faster startup time than the HSE crystal oscillator however, even with calibration the frequency is less accurate than an external crystal oscillator or ceramic resonator
- RC oscillator frequencies can vary from one chip to another due to manufacturing process variations
  - Each device is factory calibrated by ST for 1% accuracy at TA=25°C.
  - After reset, the factory calibration value is loaded in the HSICAL[7:0] bits in the Clock control register (RCC_CR)
  - You can trim the HSI frequency in the application using the HSITRIM[4:0] bits in the Clock control register (RCC_CR)

PLL

- The internal PLL can be used to multiply the HSI RC output or HSE oscillator divided by 1:16 output clock frequency
- The PLL configuration (selection of HSI oscillator divided by 2 or HSE oscillator for PLL input clock, and multiplication factor) must be done before enabling the PLL
  - Once the PLL enabled, these parameters cannot be changed
- The PLL output frequency must be in the range of 16-24 MHz

Low Speed Clocks

- The LSE crystal is a 32.768 kHz Low Speed External crystal or ceramic resonator
  - It has the advantage providing a low-power but highly accurate clock source to the real-time clock peripheral (RTC) for clock/calendar or other timing functions
- The LSI RC acts as an low-power clock source that can be kept running in Stop and Standby mode for the independent watchdog (IWDG) and Auto-wakeup unit (AWU)
  - The clock frequency is around 40 kHz.

System Clock (SYSCLK) Selection

- After a system reset, the HSI oscillator is selected as system clock
- When a clock source is used directly or through the PLL as system clock, it is not possible to stop it
- A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked)
  - If a clock source which is not yet ready is selected, the switch will occur when the clock source will be ready
  - Status bits in the Clock control register (RCC_CR) indicate which clock(s) is (are) ready and which clock is currently used as system clock

Clock Security System

- Clock Security System can be activated by software. In this case, the clock detector is enabled after the HSE oscillator startup delay, and disabled when this oscillator is stopped.
- If a failure is detected on the HSE clock, the HSE oscillator is automatically disabled, a clock failure event is sent to the break input of the advanced-control timers (TIM1) and an interrupt is generated to inform the software about the failure (Clock Security System Interrupt CSSI), allowing the MCU to perform rescue operations.

RTC and Watchdog Clocks

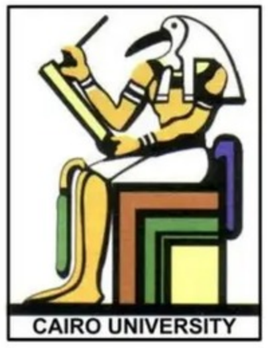
- The RTCCLK clock source can be either the HSE/128, LSE or LSI clocks. This is selected by programming the RTCSEL[1:0] bits in the *Backup domain control register (RCC_BDCR)*
- This selection cannot be modified without resetting the Backup domain.
- If the Independent watchdog (IWDG) is started by either hardware option or software access, the LSI oscillator is forced ON and cannot be disabled
  - After the LSI oscillator temporization, the clock is provided to the IWDG

Clock-Out Capability

- The microcontroller clock output (MCO) capability allows the clock to be output onto the external MCO pin.
  - The configuration registers of the corresponding GPIO port must be programmed in alternate function mode
- One of 4 clock signals can be selected as the MCO clock:
  - SYSCLK
  - HSI
  - HSE
  - PLL clock divided by 2
- The selection is controlled by the MCO[2:0] bits of the *Clock configuration register (RCC_CFGR)*

Assignments

- ARM Project #2



STM32 MICROCONTROLLER

Lecture 4

Prof. Yasser Mostafa Kadah

Nested Vectored Interrupt Controller

- The NVIC supports up to 56 maskable interrupt channels with 16 programmable priority levels
  - Not including the sixteen Cortex™-M3 interrupt lines
- NVIC and the processor core interface are closely coupled, which enables low latency interrupt processing and efficient processing of late arriving interrupts.
- The NVIC maintains knowledge of the stacked (nested) interrupts to enable tail-chaining of interrupts.

SysTick Calibration Value Register

- The SysTick calibration value is set to 9000, which gives a reference time base of 3 ms with the SysTick clock set to 3 MHz (max HCLK/8)

Interrupt and Exception Vectors

Table 50. Vector table for STM32F100xx devices

Position	Priority	Type of priority	Acronym	Description	Address
	-	-	-	Reserved	0x0000_0000
	-3	fixed	Reset	Reset	0x0000_0004
	-2	fixed	NMI_Handler	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
	-1	fixed	HardFault_Handler	All class of fault	0x0000_000C

Interrupt and Exception Vectors

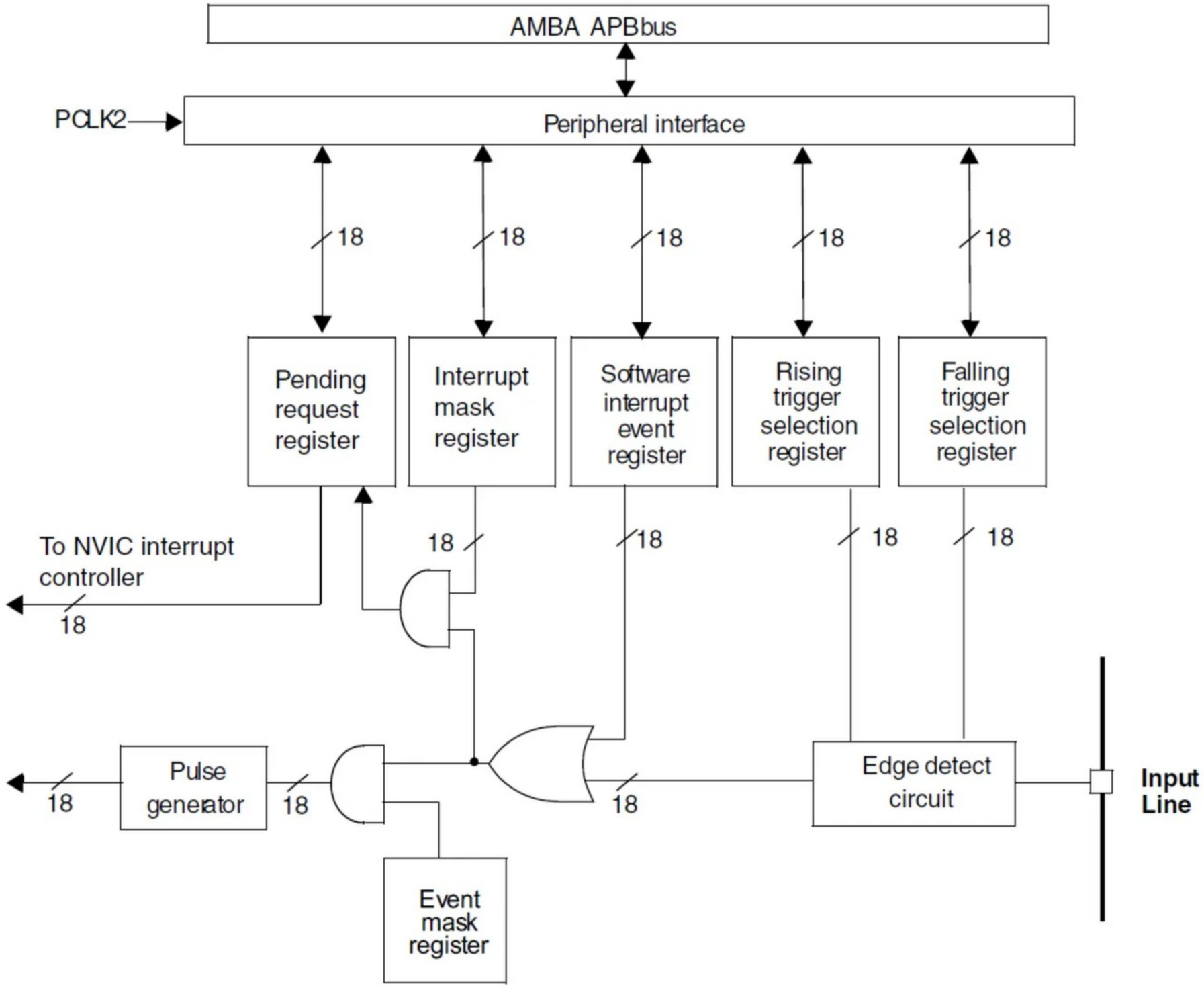
Table 50. Vector table for STM32F100xx devices (continued)

Position	Priority	Type of priority	Acronym	Description	Address
	0	settable	MemManage_Handler	Memory management	0x0000_0010
	1	settable	BusFault_Handler	Pre-fetch fault, memory access fault	0x0000_0014
	2	settable	UsageFault_Handler	Undefined instruction or illegal state	0x0000_0018
	-	-	-	Reserved	0x0000_001C - 0x0000_002B
	3	settable	SVC_Handler	System service call via SWI instruction	0x0000_002C
	4	settable	DebugMon_Handler	Debug Monitor	0x0000_0030
	-	-	-	Reserved	0x0000_0034
	5	settable	PendSV_Handler	Pendable request for system service	0x0000_0038
	6	settable	SysTick_Handler	System tick timer	0x0000_003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040

External Interrupt/Event Controller (EXTI)

- Consists of up to 18 edge detectors for generating requests
- Each input line can be independently configured
  - Select the type (pulse or pending)
  - Select corresponding trigger event (rising or falling or both)
  - Each line can also be masked independently
- Pending register maintains the status line of interrupt requests
- EXTI controller main features are the following:
  - Independent trigger and mask on each interrupt/event line
  - Dedicated status bit for each interrupt line
  - Generation of up to 18 software event/interrupt requests

External Interrupt/Event Controller (EXTI)



NVIC Functional Description

- To generate the interrupt, the interrupt line should be configured and enabled.
- This is done by programming the two trigger registers with the desired edge detection and by enabling the interrupt request by writing a '1' to the corresponding bit in the interrupt mask register.
- When the selected edge occurs on the external interrupt line, an interrupt request is generated.
- The pending bit corresponding to the interrupt line is also set. This request is reset by writing a '1' in the pending register.

NVIC Functional Description

- To generate the event, the event line should be configured and enabled
- This is done by programming the two trigger registers with the desired edge detection and by enabling the event request by writing a ‘1’ to the corresponding bit in the event mask register
- When the selected edge occurs on the event line, an event pulse is generated and the pending bit corresponding to the event line is not set
- An interrupt/event request can also be generated by software by writing a ‘1’ in the software interrupt/event register

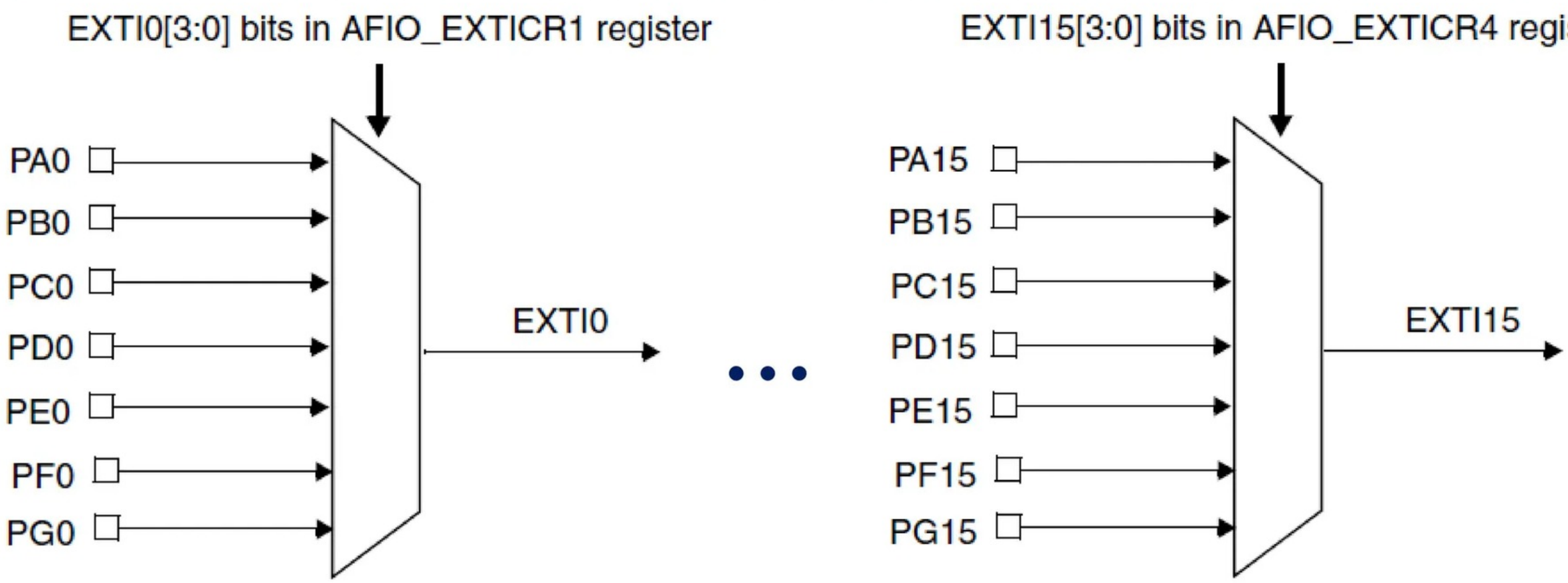
Hardware Interrupt/Event Selection

- To configure the 18 lines as interrupt sources, use the following procedure:
  - Configure the mask bits of the 18 Interrupt lines (EXTI_IMR)
  - Configure the Trigger Selection bits of the Interrupt lines (EXTI_RTSR and EXTI_FTSR)
  - Configure the enable and mask bits that control the NVIC IRQ channel mapped to the External Interrupt Controller (EXTI) so that an interrupt coming from one of the 18 lines can be correctly acknowledged
- To configure the 18 lines as event sources, use the following procedure:
  - Configure the mask bits of the 18 Event lines (EXTI_EMR)
  - Configure the Trigger Selection bits of the Event lines (EXTI_RTSR and EXTI_FTSR)

Software Interrupt/Event Selection

- The 18 lines can be configured as software interrupt lines as follows:
  - Configure the mask bits of the 18 Interrupt/Event lines (EXTI_IMR, EXTI_EMR)
  - Set the required bit of the software interrupt register (EXTI_SWIER)

External Interrupt/Event GPIO Mapping



- The two other EXTI lines are connected as follows:
  - EXTI line 16 is connected to the PVD output
  - EXTI line 17 is connected to the RTC Alarm event

Interrupt Mask Register (EXTI_IMR)

Address offset: 0x00
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														MR17	MR16
														rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:18 Reserved, must be kept at reset value (0).

Bits 17:0 **MRx:** Interrupt Mask on line x

0: Interrupt request from Line x is masked
1: Interrupt request from Line x is not masked

Event Mask Register (EXTI_EMR)

Address offset: 0x04
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														MR17	MR16
														rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:18 Reserved, must be kept at reset value (0).

Bits 17:0 **MRx:** Event mask on line x

0: Event request from Line x is masked
1: Event request from Line x is not masked

Rising trigger selection register (EXTI_RTSR)

Address offset: 0x08
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														TR17	TR16
														rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:18 Reserved, must be kept at reset value (0).

Bits 17:0 **TRx:** Rising trigger event configuration bit of line x

0: Rising trigger disabled (for Event and Interrupt) for input line
1: Rising trigger enabled (for Event and Interrupt) for input line

Falling trigger selection register (EXTI_FTSR)

Address offset: 0x0C
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														TR17	TR16
														rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:18 Reserved, must be kept at reset value (0).

Bits 17:0 **TRx:** Falling trigger event configuration bit of line x

0: Falling trigger disabled (for Event and Interrupt) for input line
1: Falling trigger enabled (for Event and Interrupt) for input line

Software Interrupt Event Register (EXTI_SWIER)

Address offset: 0x10
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														SWIER 17	SWIER 16
														rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER 15	SWIER 14	SWIER 13	SWIER 12	SWIER 11	SWIER 10	SWIER 9	SWIER 8	SWIER 7	SWIER 6	SWIER 5	SWIER 4	SWIER 3	SWIER 2	SWIER 1	SWIER 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:18 Reserved, must be kept at reset value (0).

Bits 17:0 **SWIERx:** Software interrupt on line x

Writing a 1 to this bit when it is at 0 sets the corresponding pending bit in EXTI_PR. If the interrupt is enabled on this line on the EXTI_IMR and EXTI_EMR, an interrupt request is generated.

This bit is cleared by clearing the corresponding bit of EXTI_PR (by writing a 1 into the bit)

Pending Register (EXTI_PR)

Address offset: 0x14
Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														PR17	PR16
														rc_w1	rc_w1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:18 Reserved, must be kept at reset value (0).

Bits 17:0 **PRx:** Pending bit

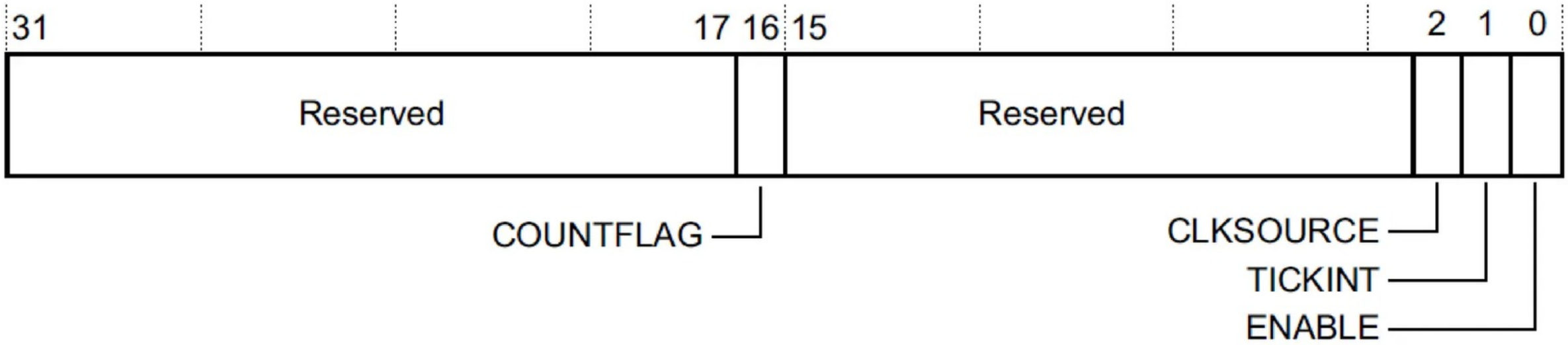
0: No trigger request occurred
1: selected trigger request occurred
This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a 1 into the bit or by changing the sensitivity of the edge detector.

SysTick Control and Status Register

Address 0xE000E010

Access Read/write

Reset state 0x00000000

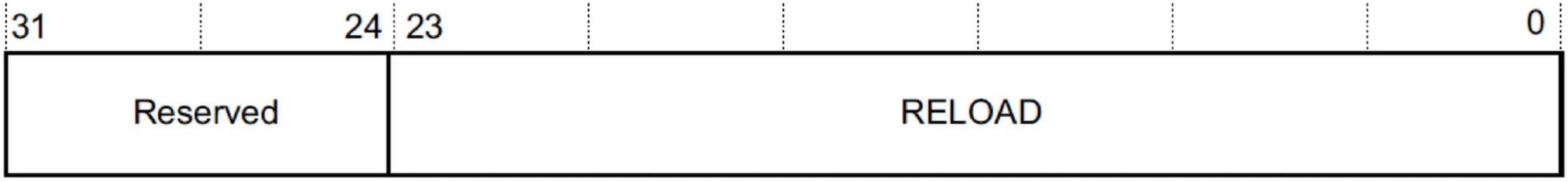


SysTick Reload Value Register

Address `0xE000E014`

Access Read/write

Reset state Unpredictable



SysTick Current Value Register

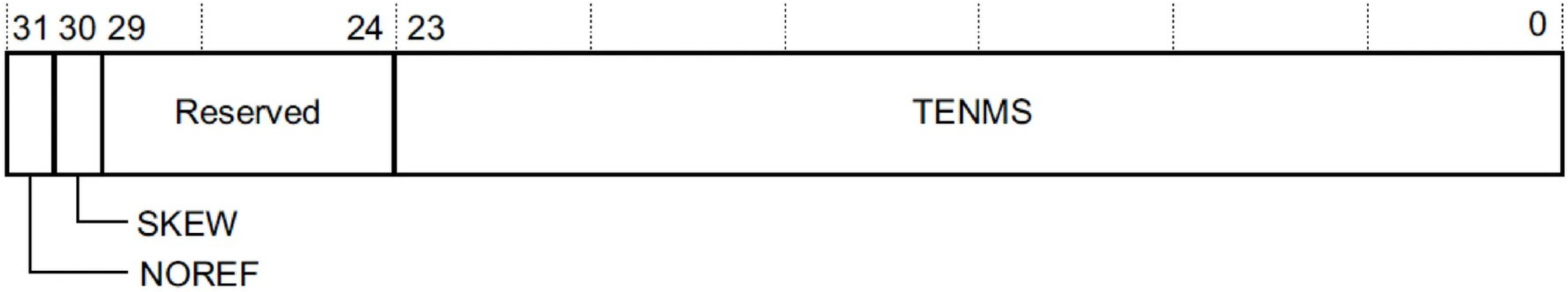
Address	`0xE000E018`
Access	Read/write clear
Reset state	Unpredictable

SysTick Calibration Value Register

Address `0xE000E01C`

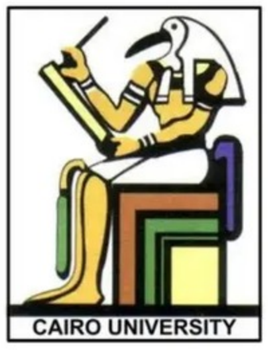
Access Read

Reset state STCALIB



Assignments

- ARM Project #3



STM32 MICROCONTROLLER: GENERAL-PURPOSE TIMERS (TIM2-TIM5)

Lecture 5

Prof. Yasser Mostafa Kadah

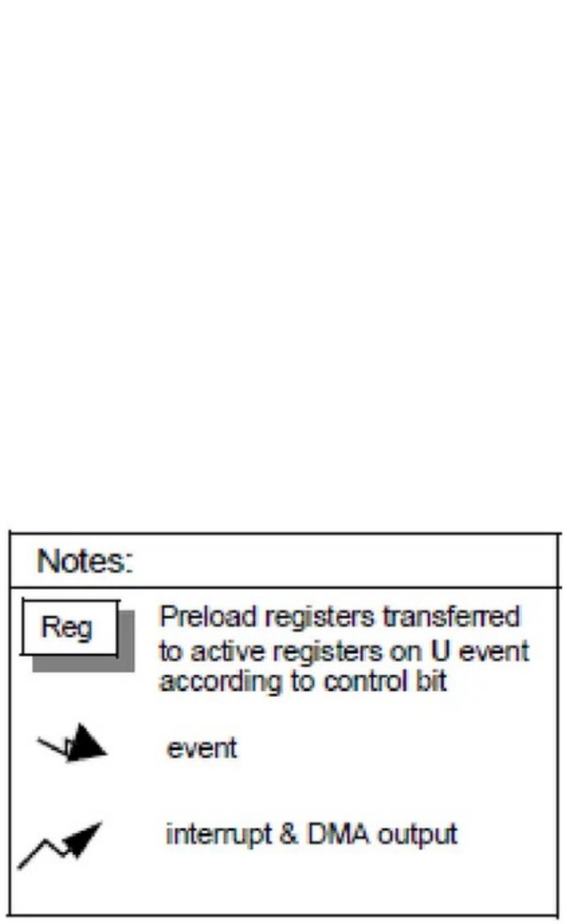
TIM2-TIM5 Introduction

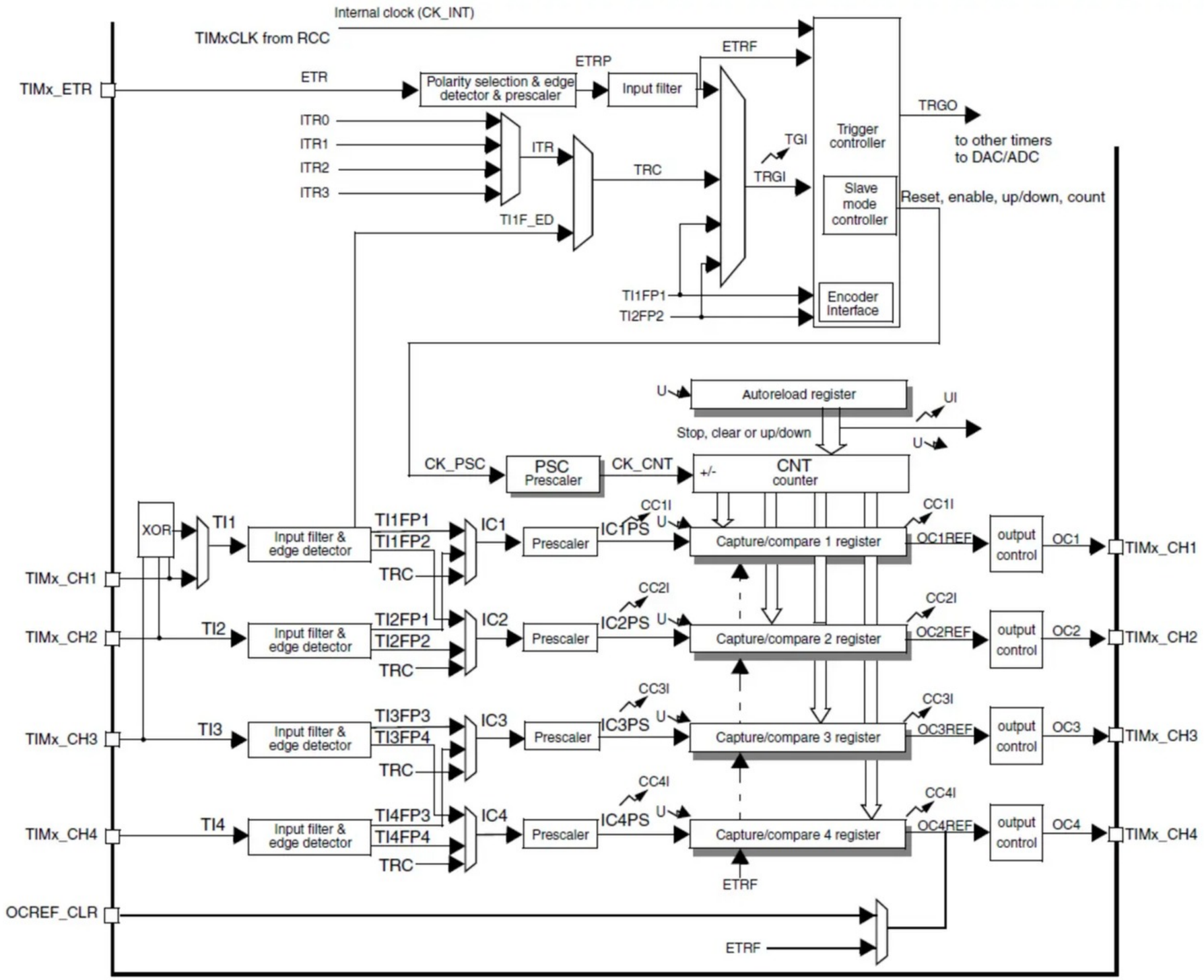
- The general-purpose timers consist of a 16-bit auto-reload counter driven by a programmable prescaler.
  - Measuring the pulse lengths of input signals (input capture)
  - Generating output waveforms (output compare, PWM)
- Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers
- General-purpose (TIMx) timers are completely independent, and do not share any resources
  - They can still be synchronized together

TIM2-TIM5 Main Features

- 16-bit up, down, up/down auto-reload counter
- 16-bit programmable prescaler allowing dividing (also “on the fly”) the counter clock frequency either by any factor between 1 and 65535.
- Up to 4 independent channels for:
  - Input Capture
  - Output Compare
  - PWM generation (Edge and Center-aligned Mode)
  - One-pulse mode output
- Synchronization circuit to control timer with external signals and to interconnect several timers together
- Interrupt/DMA generation based on several events

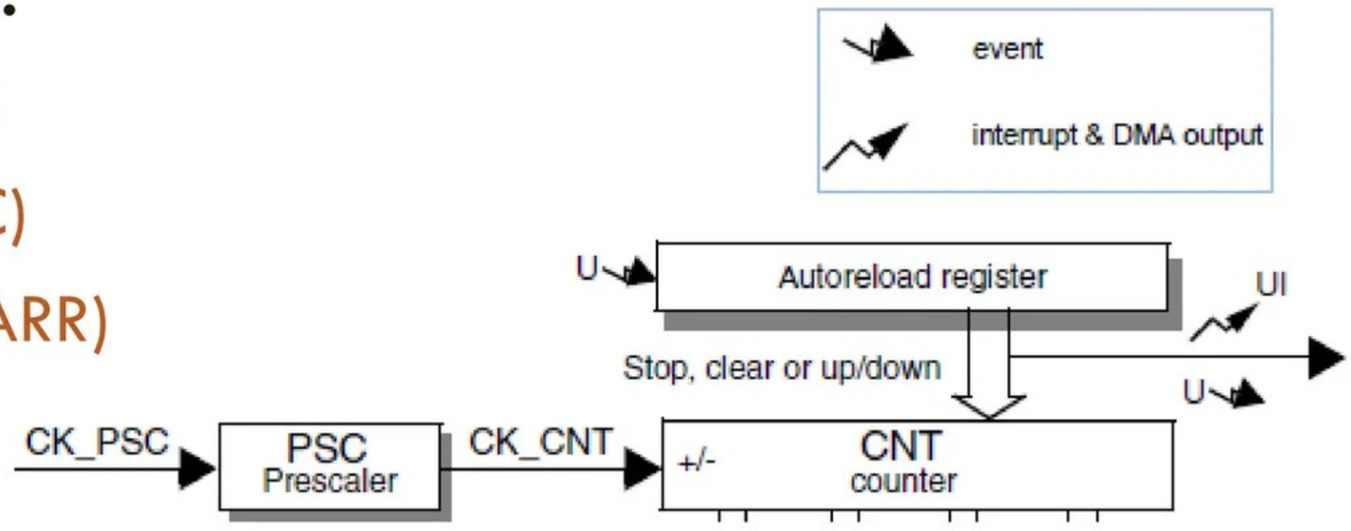
TIM2-TIM5 Block Diagram





Time-Base Unit

- The main block of the programmable timer is a 16-bit counter with its related auto-reload register
  - The counter can count up, down or both up and down
  - The counter clock can be divided by a prescaler.
- The counter, the auto-reload register and the prescaler register can be written or read by software
  - This is true even when the counter is running
- The time-base unit includes:
  - Counter register (TIMx_CNT)
  - Prescaler register (TIMx_PSC)
  - Auto-reload register (TIMx_ARR)



Time-Base Unit

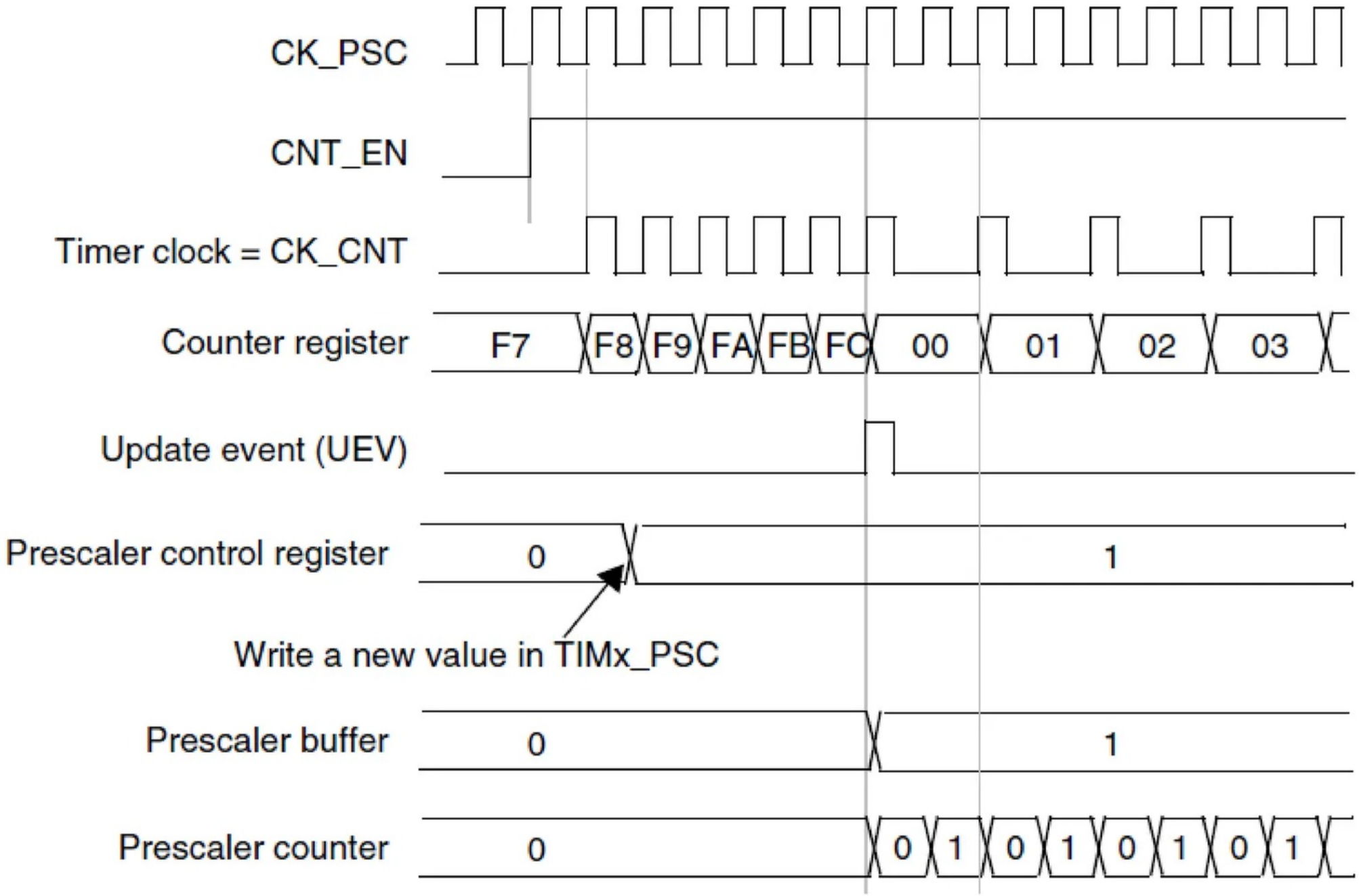
- Auto-reload register is preloaded
- Writing to or reading from the auto-reload register accesses the preload register
- Contents of preload register are transferred into the shadow register permanently or at each update event (UEV), depending on the auto-reload preload enable bit (ARPE) in TIMx_CR1 register
- Update event is sent when counter reaches overflow or underflow and if the UDIS bit equals 0 in TIMx_CR1 register
- Update event can also be generated by software
- Counter is clocked by prescaler output CK_CNT, which is enabled only when counter enable bit (CEN) in TIMx_CR1 register is set
  - actual counter enable signal CNT_EN is set 1 clock cycle after CEN

Prescaler

- Prescaler can divide the counter clock frequency by any factor between 1 and 65536
- Based on a 16-bit counter controlled through a 16-bit register (in the TIMx_PSC register)
- It can be changed on the fly as this control register is buffered
  - New prescaler ratio is taken into account at the next update event

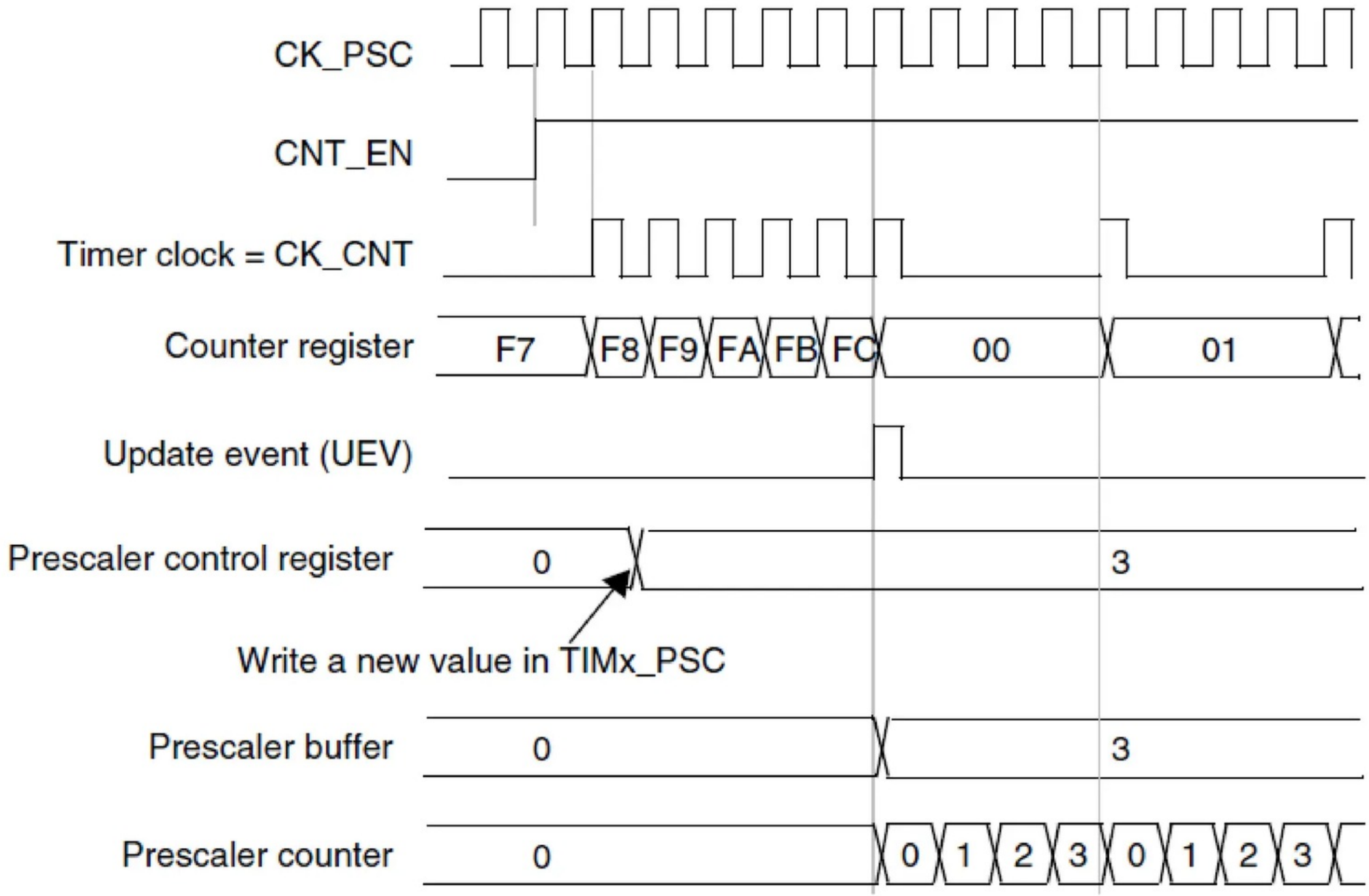
Prescaler

- Counter timing diagram with prescaler division change from 1 to 2



Prescaler

- Counter timing diagram with prescaler division change from 1 to 4

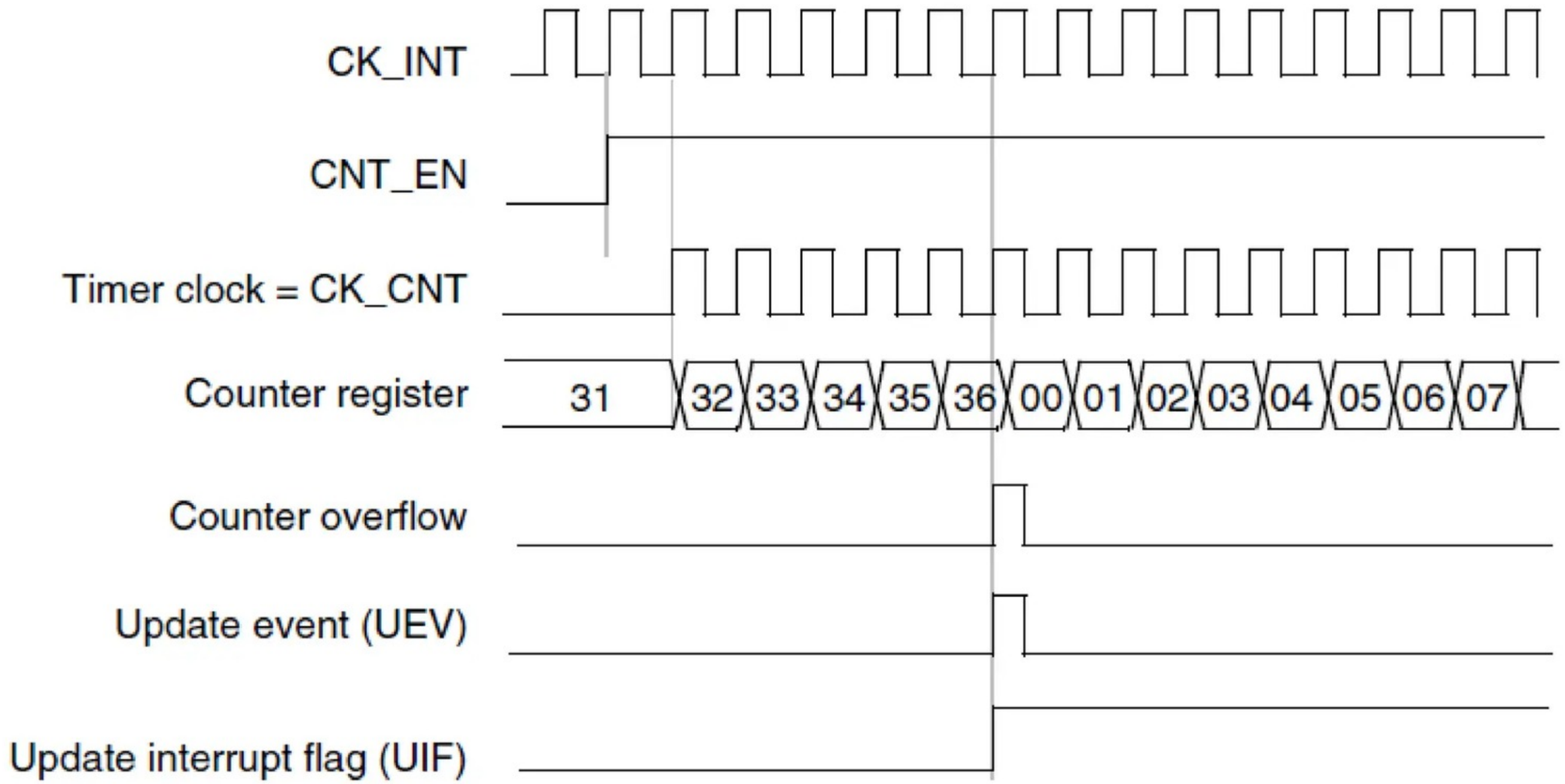


Counter Modes: Upcounting Mode

- Counter counts from 0 to the auto-reload value (content of the TIMx_ARR register), then restarts from 0 and generates a counter overflow event
- An Update event can be generated at each counter overflow or by setting the UG bit in the TIMx_EGR register
- When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):
  - The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register)
  - The auto-reload shadow register is updated with the preload value (TIMx_ARR)

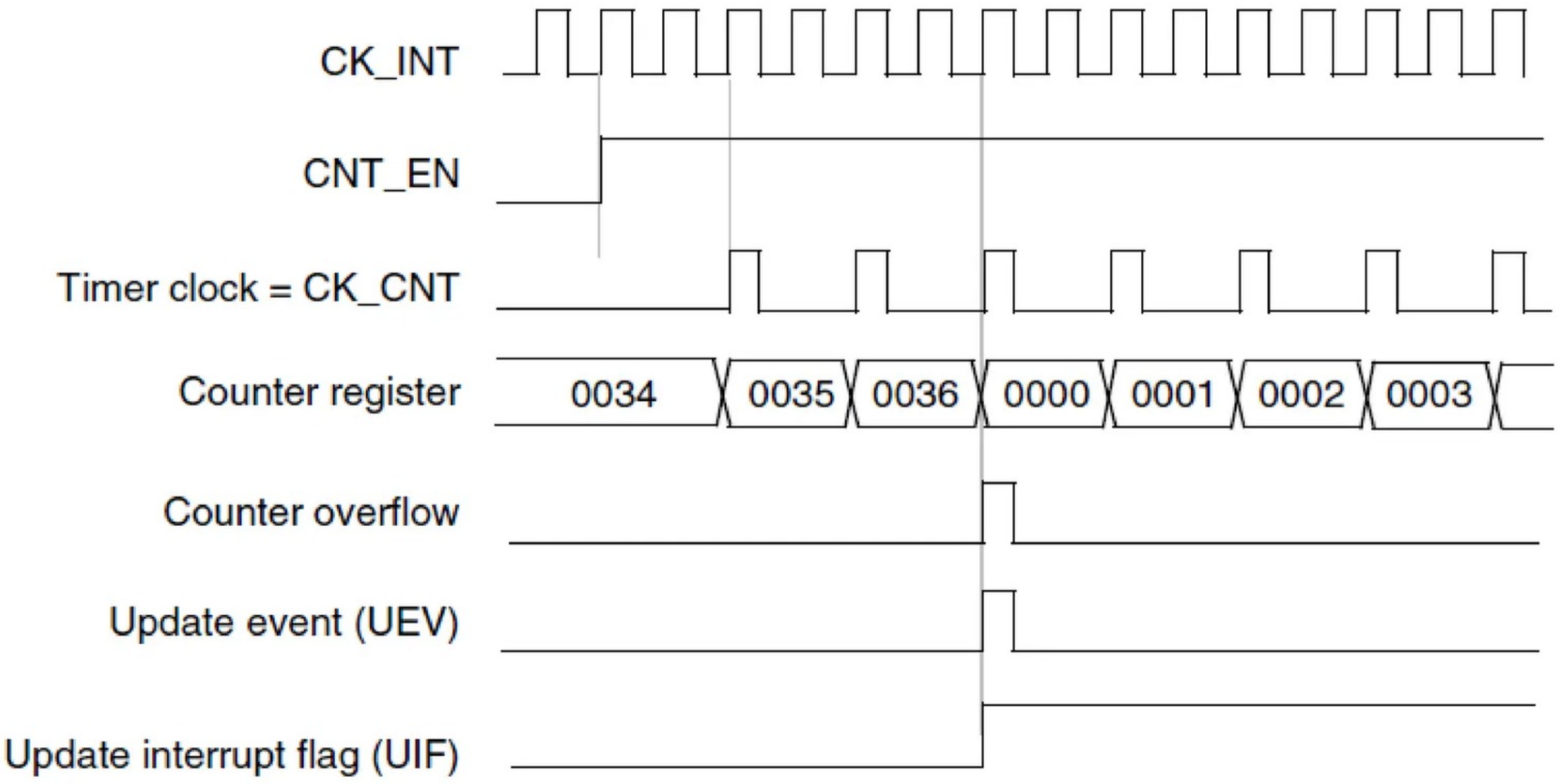
Upcounting Mode Example

- TIMx_ARR=0x36 , internal clock divided by 1



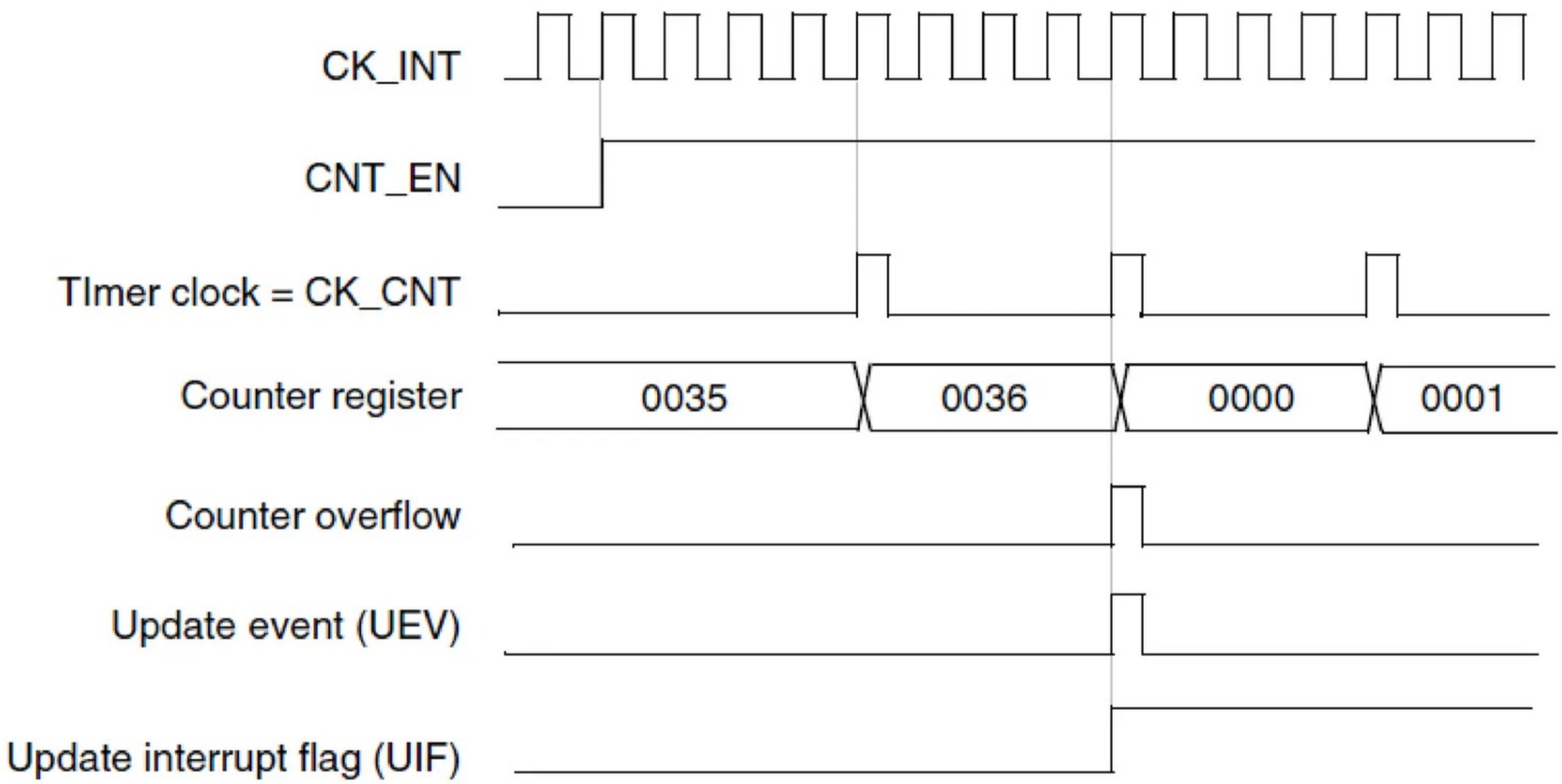
Upcounting Mode Example

- TIMx_ARR=0x36 , internal clock divided by 2



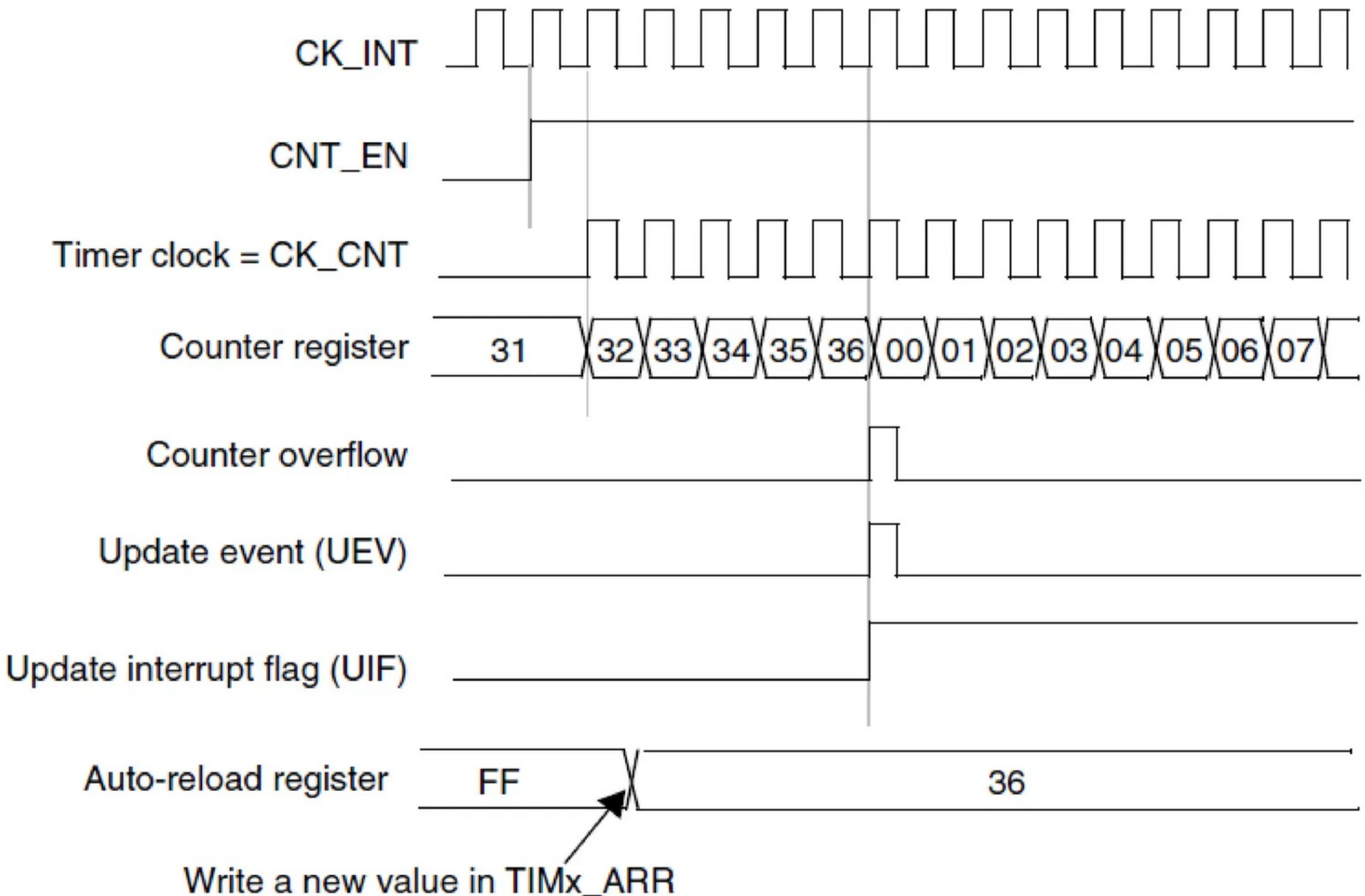
Upcounting Mode Example

- TIMx_ARR=0x36 , internal clock divided by 4



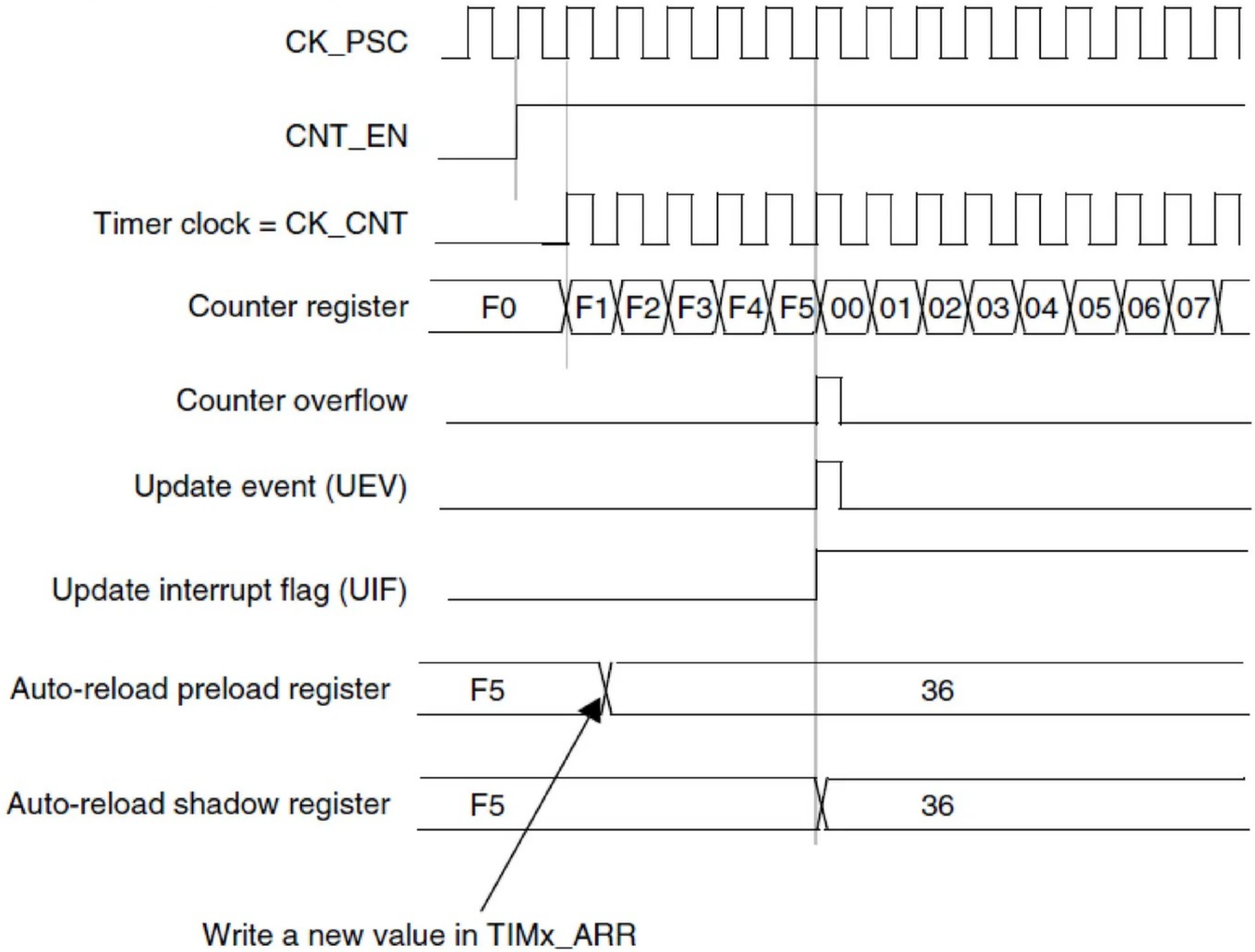
Upcounting Mode Example

- TIMx_ARR=0x36 , Update event when ARPE=0 (TIMx_ARR not preloaded)



Upcounting Mode Example

- TIMx_ARR=0x36 , Update event when ARPE=1 (TIMx_ARR preloaded)

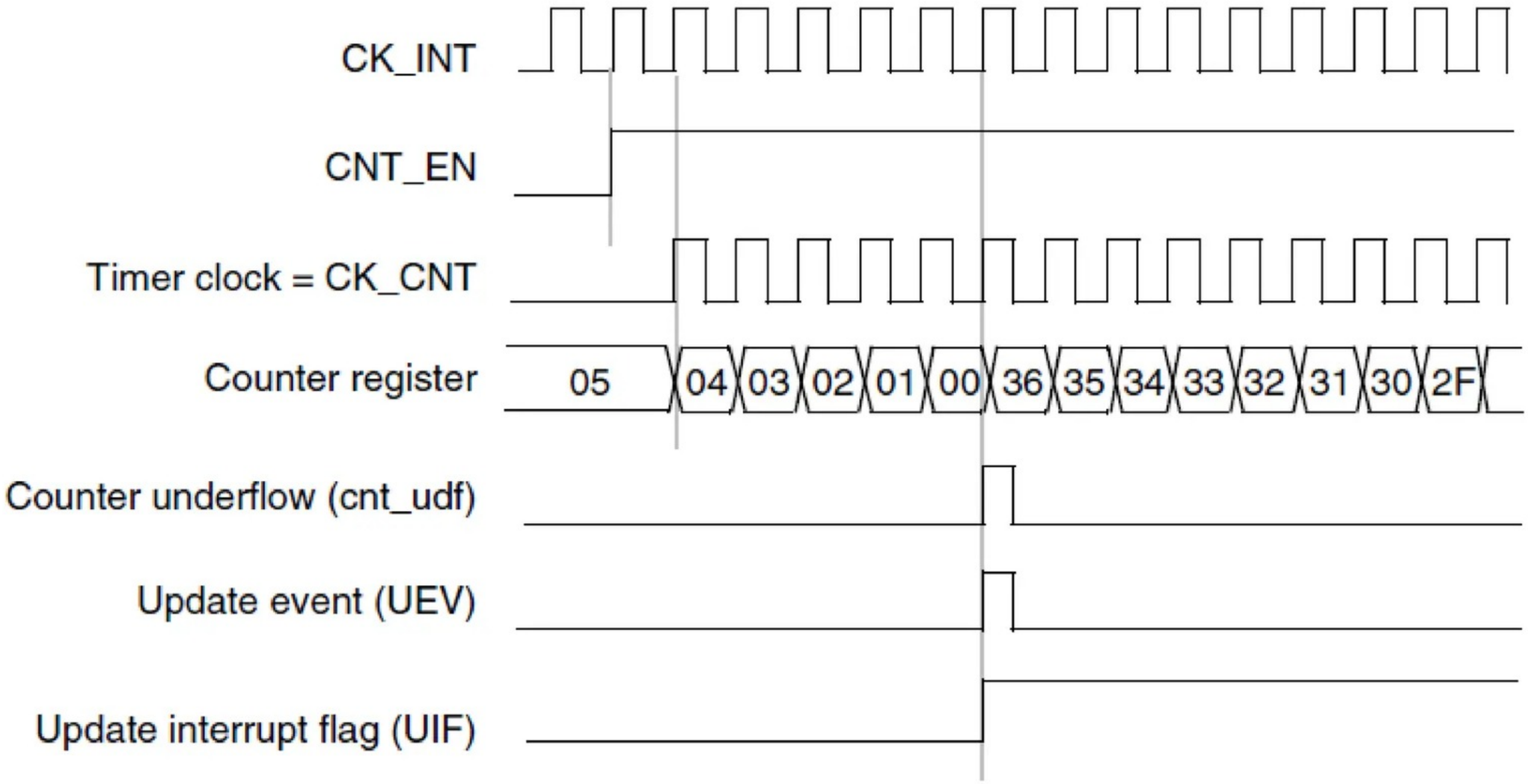


Counter Modes: Downcounting Mode

- Counter counts from the auto-reload value (content of the
- TIMx_ARR register) down to 0, then restarts from the auto-reload value and generates a counter underflow event
- An Update event can be generate at each counter underflow or by setting the UG bit in the TIMx_EGR register
- When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):
  - The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register)
  - The auto-reload shadow register is updated with the preload value (TIMx_ARR)

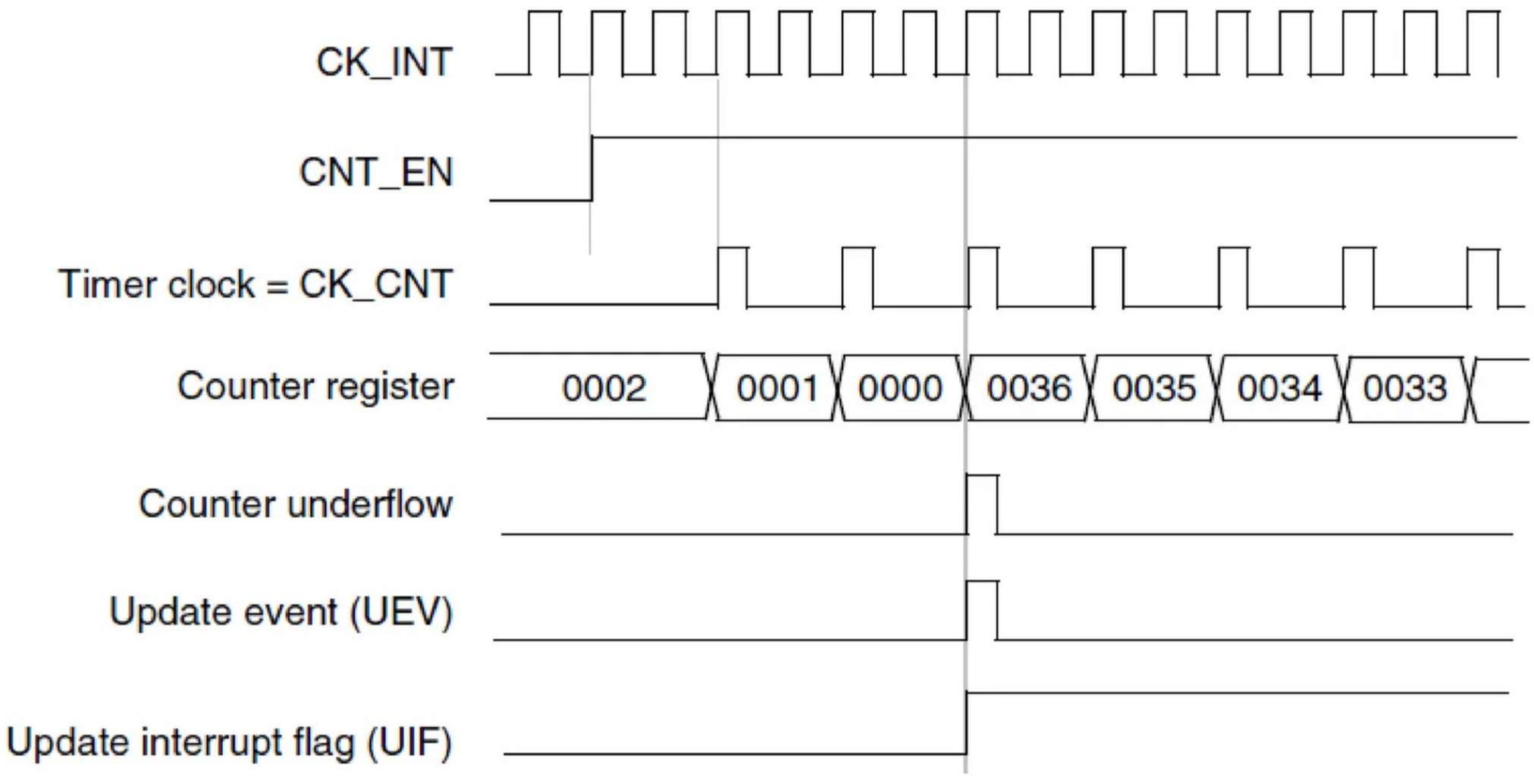
Downcounting Mode Example

- TIMx_ARR=0x36 , internal clock divided by 1



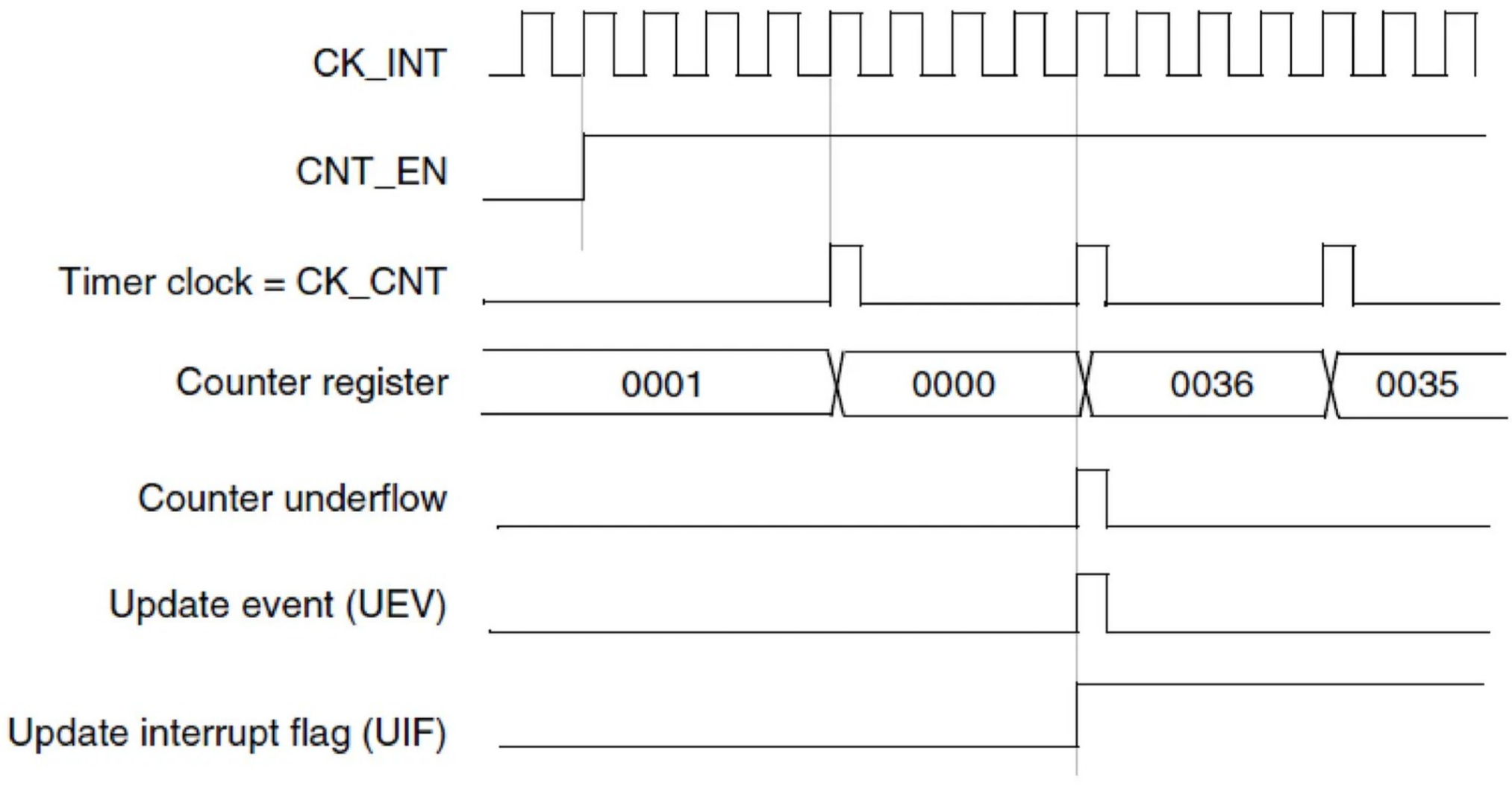
Downcounting Mode Example

- TIMx_ARR=0x36 , internal clock divided by 2



Downcounting Mode Example

- TIMx_ARR=0x36 , internal clock divided by 4

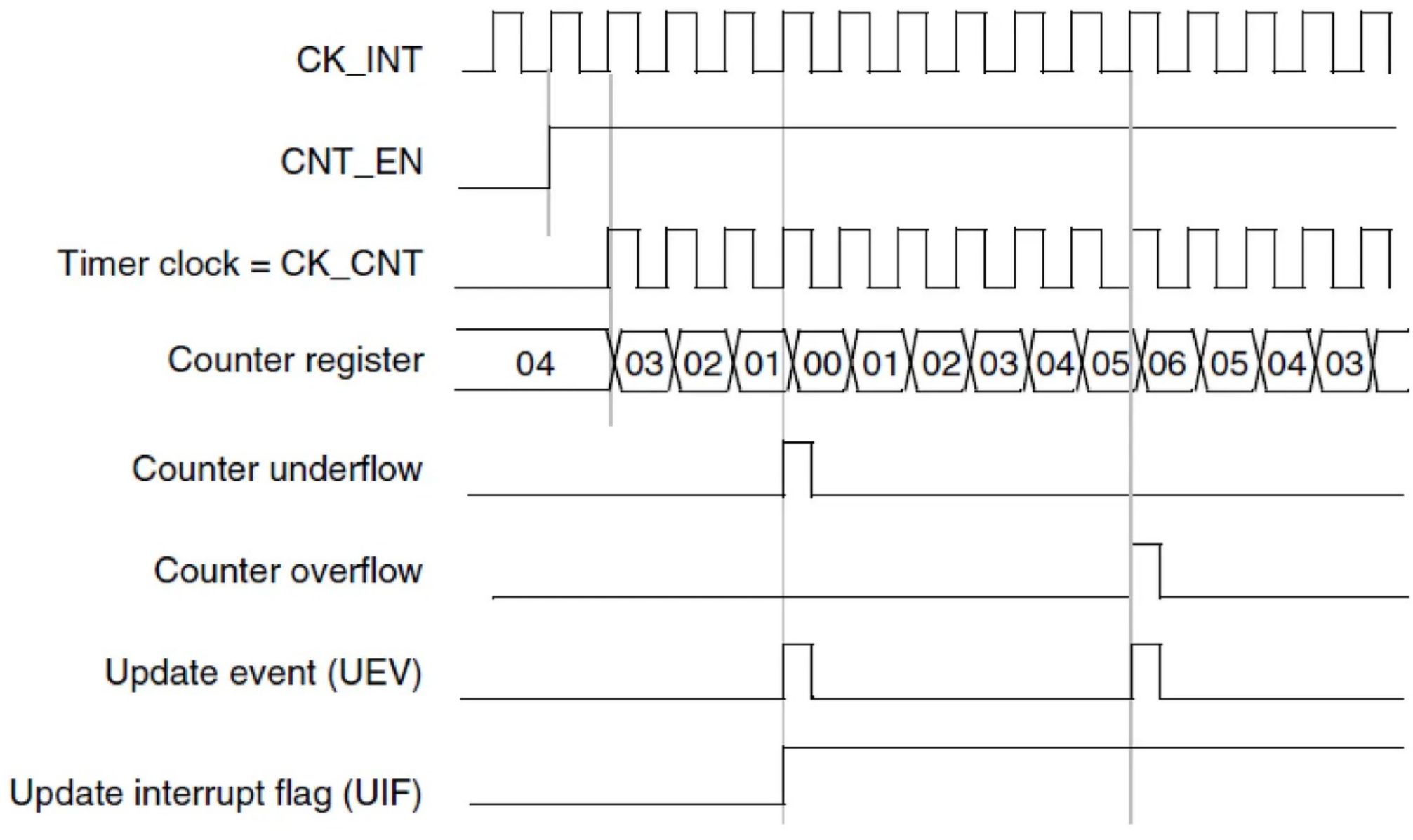


Counter Modes: Center-Aligned Mode (Up/Down Counting)

- Counter counts from 0 to the auto-reload value (content of the TIMx_ARR register) – 1, generates a counter overflow event, then counts from the autoreload value down to 1 and generates a counter underflow event. Then it restarts counting from 0
- Center-aligned mode is active when CMS bits in TIMx_CR1 register are not equal to '00'. The Output compare interrupt flag of channels configured in output is set when: the counter counts down (Center aligned mode 1, CMS = "01"), the counter counts up (Center aligned mode 2, CMS = "10") the counter counts up and down (Center aligned mode 3, CMS = "11")
- In this mode, the direction bit (DIR from TIMx_CR1 register) cannot be written
  - Updated by hardware and gives the current direction of the counter

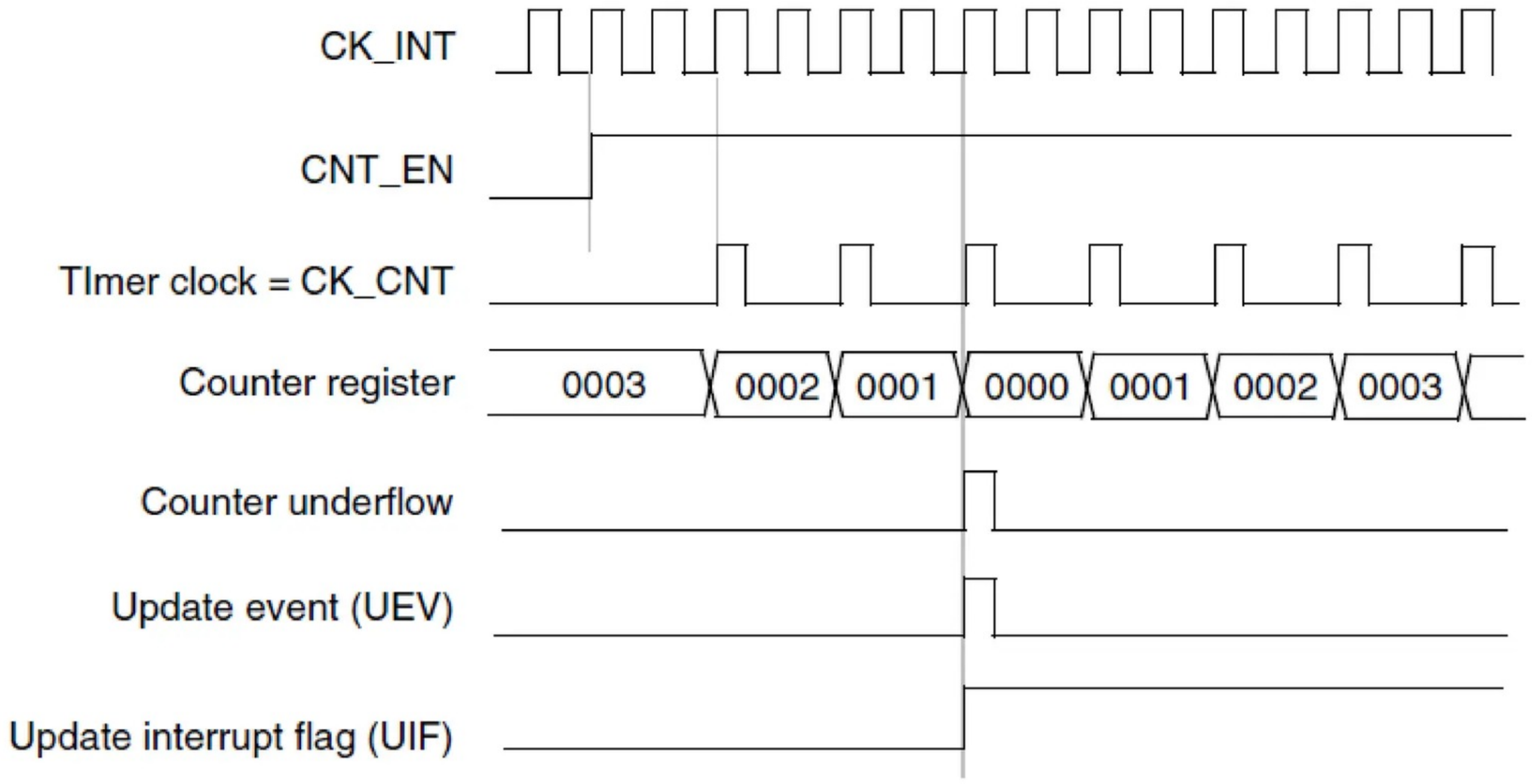
Up/Down Counting Example

- Internal clock divided by 1, TIMx_ARR=0x6



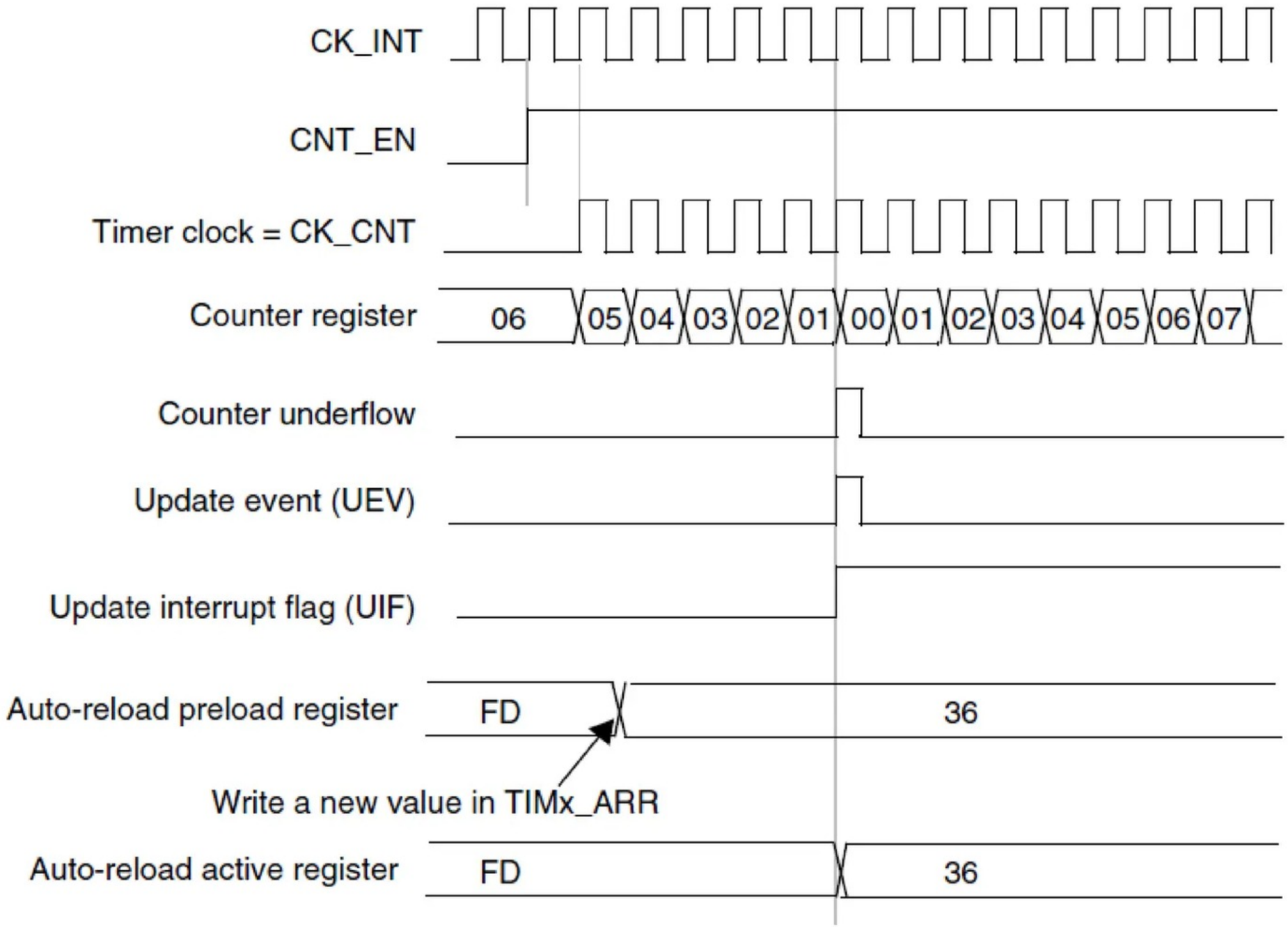
Up/Down Counting Example

- Internal clock divided by 2, TIMx_ARR=0x6



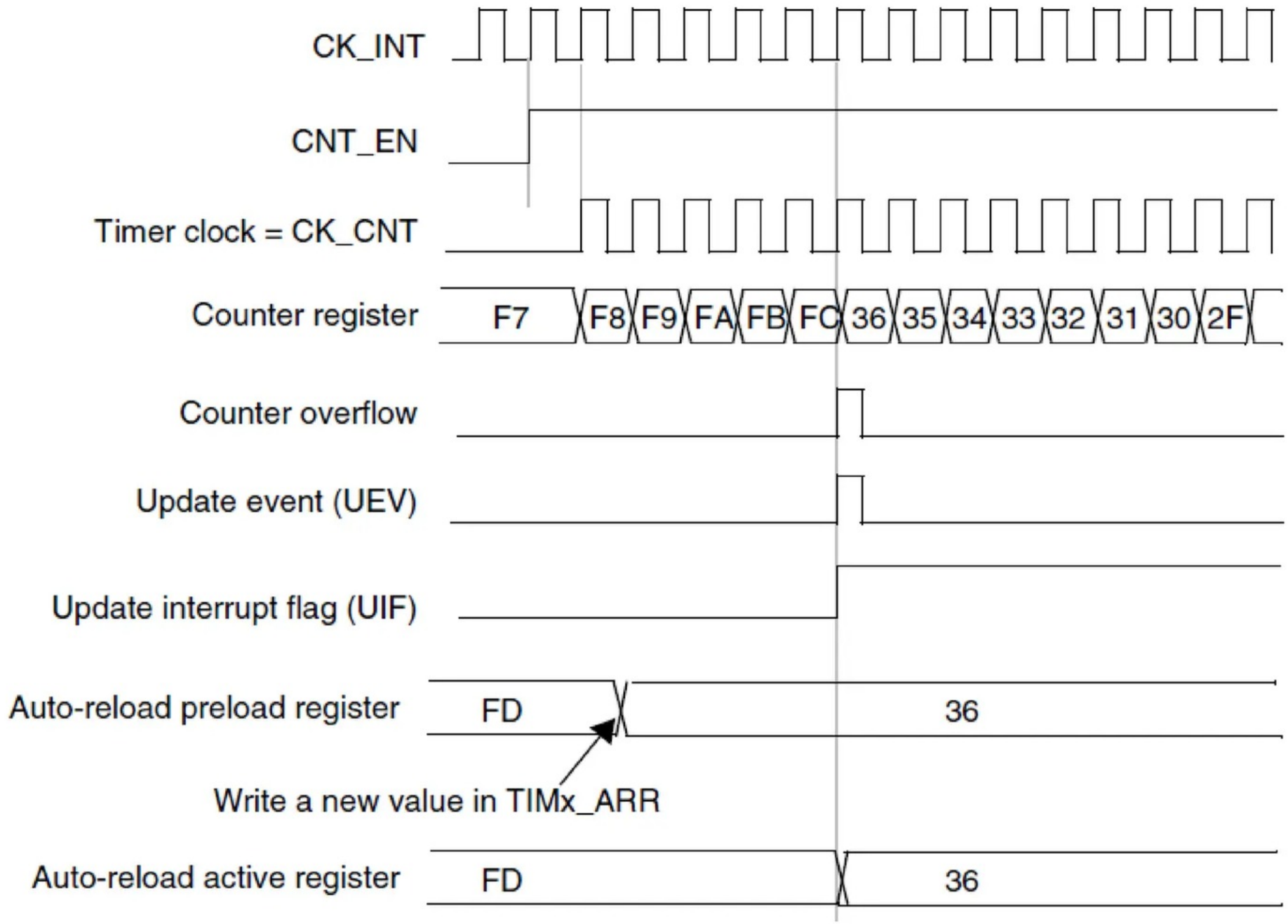
Up/Down Counting Example

- Counter timing diagram, Update event with ARPE=1 (counter underflow)



Up/Down Counting Example

- Counter timing diagram, Update event with ARPE=1 (counter overflow)

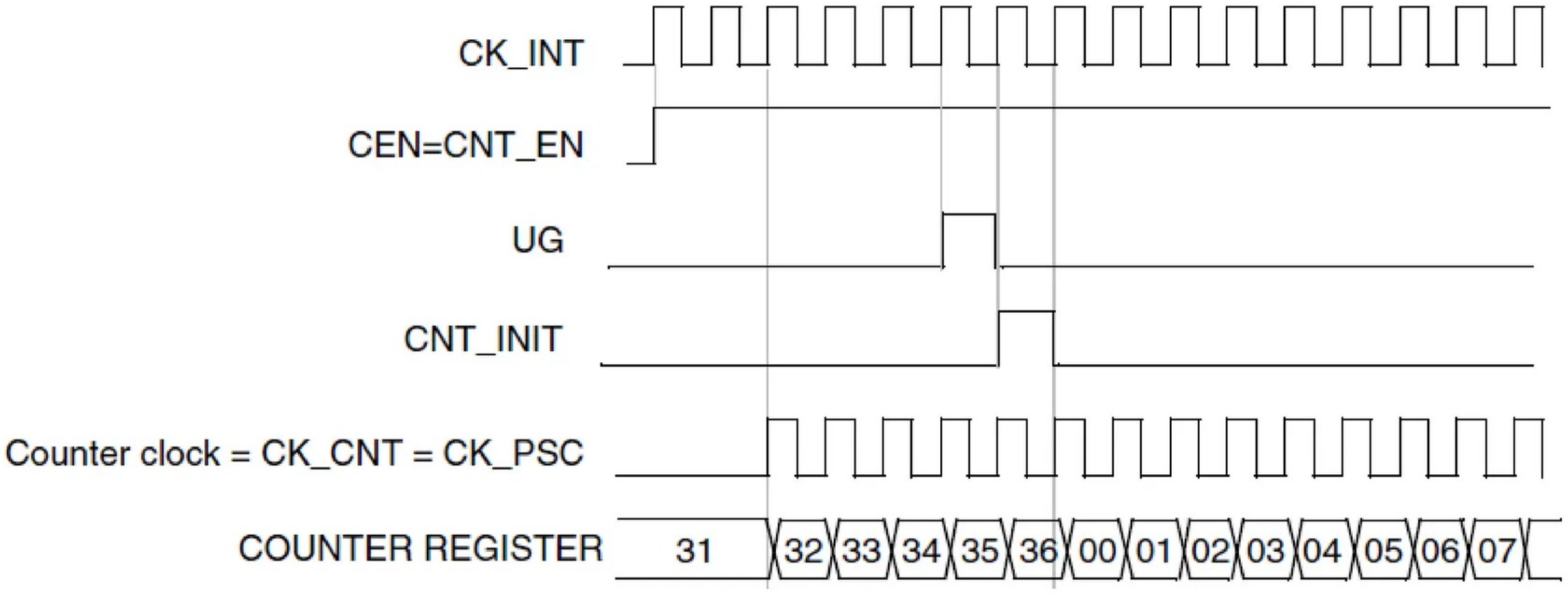


Clock Selection

- The counter clock can be provided by the following clock sources:
  - **Internal clock (CK_INT) (Our focus in this part)**
  - External clock mode1: external input pin (TIx)
  - External clock mode2: external trigger input (ETR)
  - Internal trigger inputs (ITRx): using one timer as prescaler for another timer, for example, you can configure Timer 1 to act as a prescaler for Timer 2

Internal Clock Source (CK_INT)

- If the slave mode controller is disabled (SMS=000 in the TIMx_SMCR register), then the CEN, DIR (in the TIMx_CR1 register) and UG bits (in the TIMx_EGR register) are actual control bits and can be changed only by software (except UG which remains cleared automatically)
- As soon as the CEN bit is written to 1, the prescaler is clocked by the internal clock CK_INT.



TIMx Control Register 1 (TIMx_CR1)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reset value: 0x0000

Bits 9:8 **CKD**: Clock division

This bit-field indicates the division ratio between the timer clock (CK_INT) frequency and sampling clock used by the digital filters (ETR, TIx),

00: $t_{DTS} = t_{CK_INT}$
01: $t_{DTS} = 2 \times t_{CK_INT}$
10: $t_{DTS} = 4 \times t_{CK_INT}$
11: Reserved

Bit 7 **ARPE**: Auto-reload preload enable

0: TIMx_ARR register is not buffered
1: TIMx_ARR register is buffered

Bits 6:5 **CMS**: Center-aligned mode selection

00: Edge-aligned mode. The counter counts up or down depending on the direction bit (DIR).
01: Center-aligned mode 1. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting down.
10: Center-aligned mode 2. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting up.
11: Center-aligned mode 3. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set both when the counter is counting up or down.

Note: It is not allowed to switch from edge-aligned mode to center-aligned mode as long as the counter is enabled (CEN=1)

Bit 4 **DIR**: Direction

0: Counter used as upcounter
1: Counter used as downcounter

Note: This bit is read only when the timer is configured in Center-aligned mode or Encoder mode.

Bit 3 **OPM**: One-pulse mode

0: Counter is not stopped at update event
1: Counter stops counting at the next update event (clearing the bit CEN)

Bit 2 **URS**: Update request source

This bit is set and cleared by software to select the UEV event sources.
0: Any of the following events generate an update interrupt or DMA request if enabled. These events can be:

- Counter overflow/underflow
- Setting the UG bit
- Update generation through the slave mode controller

1: Only counter overflow/underflow generates an update interrupt or DMA request if enabled.

Bit 1 **UDIS**: Update disable

This bit is set and cleared by software to enable/disable UEV event generation.
0: UEV enabled. The Update (UEV) event is generated by one of the following events:

- Counter overflow/underflow
- Setting the UG bit
- Update generation through the slave mode controller

Buffered registers are then loaded with their preload values.
1: UEV disabled. The Update event is not generated, shadow registers keep their value (ARR, PSC, CCRx). However the counter and the prescaler are reinitialized if the UG bit is set or if a hardware reset is received from the slave mode controller.

Bit 0 **CEN**: Counter enable

0: Counter disabled
1: Counter enabled

Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.

CEN is cleared automatically in one-pulse mode, when an update event occurs.

TIMx Slave Mode Control Register (TIMx_SMCR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]		ETF[3:0]				MSM	TS[2:0]			OCCS	SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reset value: 0x0000

Bits 2:0 **SMS:** Slave mode selection

When external signals are selected the active edge of the trigger signal (TRGI) is linked to the polarity selected on the external input (see Input Control register and Control Register description.

000: Slave mode disabled - if CEN = ‘1 then the prescaler is clocked directly by the internal clock.

TIMx Event Generation Register (TIMx_EGR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									TG	Res.	CC4G	CC3G	CC2G	CC1G	UG
									w		w	w	w	w	w

Reset value: 0x0000

Bit 6 **TG**: Trigger generation

This bit is set by software in order to generate an event, it is automatically cleared by hardware.

0: No action

1: The TIF flag is set in TIMx_SR register. Related interrupt or DMA transfer can occur if enabled.

Bit 0 **UG**: Update generation

This bit can be set by software, it is automatically cleared by hardware.

0: No action

1: Re-initialize the counter and generates an update of the registers. Note that the prescaler counter is cleared too (anyway the prescaler ratio is not affected). The counter is cleared if the center-aligned mode is selected or if DIR=0 (upcounting), else it takes the auto-reload value (TIMx_ARR) if DIR=1 (downcounting).

TIMx Counter (TIMx_CNT)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reset value: 0x0000

Bits 15:0 **CNT[15:0]**: Counter value

TIMx Prescaler (TIMx_PSC)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reset value: 0x0000

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to f_{CK_PSC} / (PSC[15:0] + 1).

PSC contains the value to be loaded in the active prescaler register at each update event.

TIMx Auto-Reload Register (TIMx_ARR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reset value: 0x0000

Bits 15:0 **ARR[15:0]**: Low Auto-reload value

ARR is the value to be loaded in the actual auto-reload register.

The counter is blocked while the auto-reload value is null.

TIMx DMA/Interrupt Enable Register (TIMx_DIER)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDE	Res	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res.	TIE	Res	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw		rw	rw	rw	rw	rw		rw		rw	rw	rw	rw	rw

Reset value: 0x0000

Bit 0 **UIE**: Update interrupt enable

0: Update interrupt disabled

1: Update interrupt enabled

TIMx Status Register (TIMx_SR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved		TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Reset value: 0x0000

Bit 0 **UIF**: Update interrupt flag

This bit is set by hardware on an update event. It is cleared by software.

0: No update occurred

1: Update interrupt pending. This bit is set by hardware when the registers are updated:

Vector Table for STM32F100xx Devices

Position	Priority	Type of priority	Acronym	Description	Address
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8
50	57	settable	TIM5	TIM5 global interrupt	0x0000_0108

Timer Standard Driver

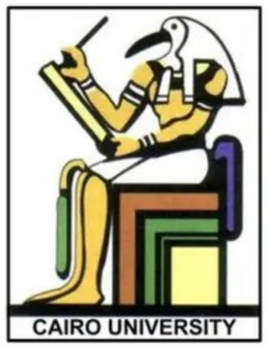
- Standard interface to all STM32 timers

- `TIM_CounterModeConfig`
- `TIM_SetCounter`
- `TIM_SetAutoreload`
- `TIM_PrescalerConfig`
- `TIM_ITConfig`
- `TIM_Cmd`
- `TIM_ClearITPendingBit`

Note: *Modify only files in the "User" group of your project and never change the standard peripheral drivers since they will affect other programs not just the one you are working on at the time. Modification of such files will result in deducting points off your project grade.*

Assignments

- ARM Project #4



STM32 MICROCONTROLLER: DIGITAL-TO-ANALOG CONVERTER (DAC)

Lecture 6

Prof. Yasser Mostafa Kadah

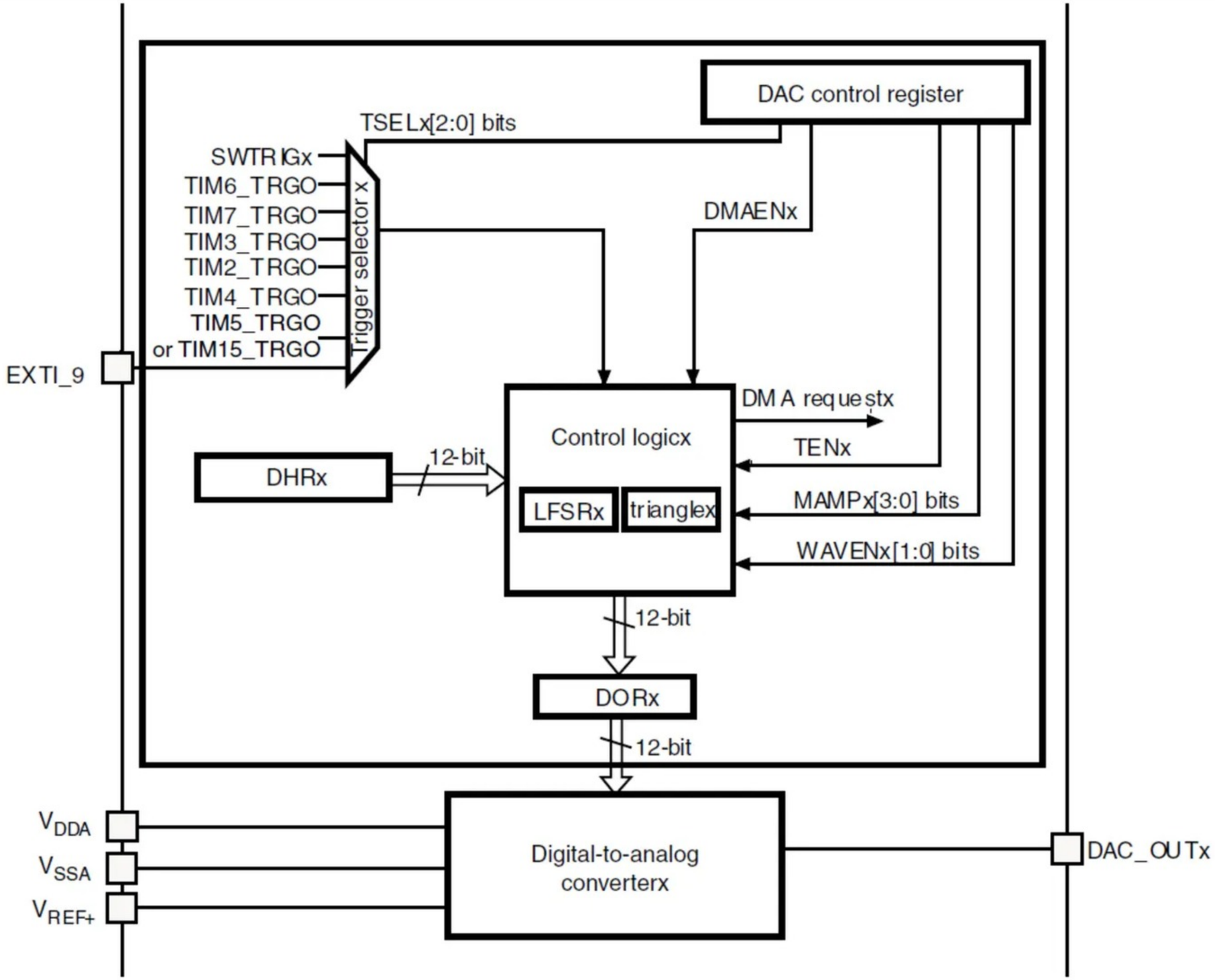
DAC Introduction

- DAC module is a 12-bit, voltage output digital-to-analog converter
- DAC can be configured in 8- or 12-bit mode and may be used in conjunction with the DMA controller
  - In 12-bit mode, the data could be left- or right-aligned
- DAC has two output channels, each with its own converter
  - In dual DAC channel mode, conversions could be done independently or simultaneously when both channels are grouped together for synchronous update operations
- Input reference pin, VREF+ (shared with ADC) is available for better resolution

DAC Main Features

- Two DAC converters: one output channel each
- Left or right data alignment in 12-bit mode
- Synchronized update capability
- Noise-wave generation
- Triangular-wave generation
- Dual DAC channel for independent or simultaneous conversions
- DMA capability for each channel
- DMA underrun error detection
- External triggers for conversion
- Input voltage reference, VREF+

DAC Channel Block Diagram



DAC Pins

- Once the DAC channelx is enabled, the corresponding GPIO pin (PA4 or PA5) is automatically connected to the analog converter output (DAC_OUTx)
- In order to avoid parasitic consumption, the PA4 or PA5 pin should first be configured to analog (AIN)

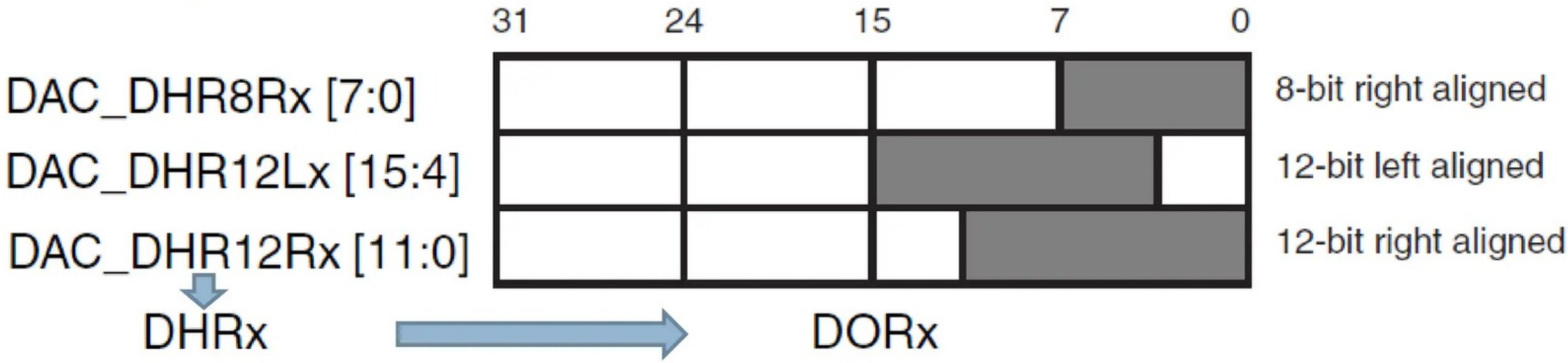
Name	Signal type	Remarks
V_{REF+}	Input, analog reference positive	The higher/positive reference voltage for the DAC, $2.4\ V \leq V_{REF+} \leq V_{DDA}$
V_{DDA}	Input, analog supply	Analog power supply
V_{SSA}	Input, analog supply ground	Ground for analog power supply
DAC_OUTx	Analog output signal	DAC channelx analog output

DAC Functional Description

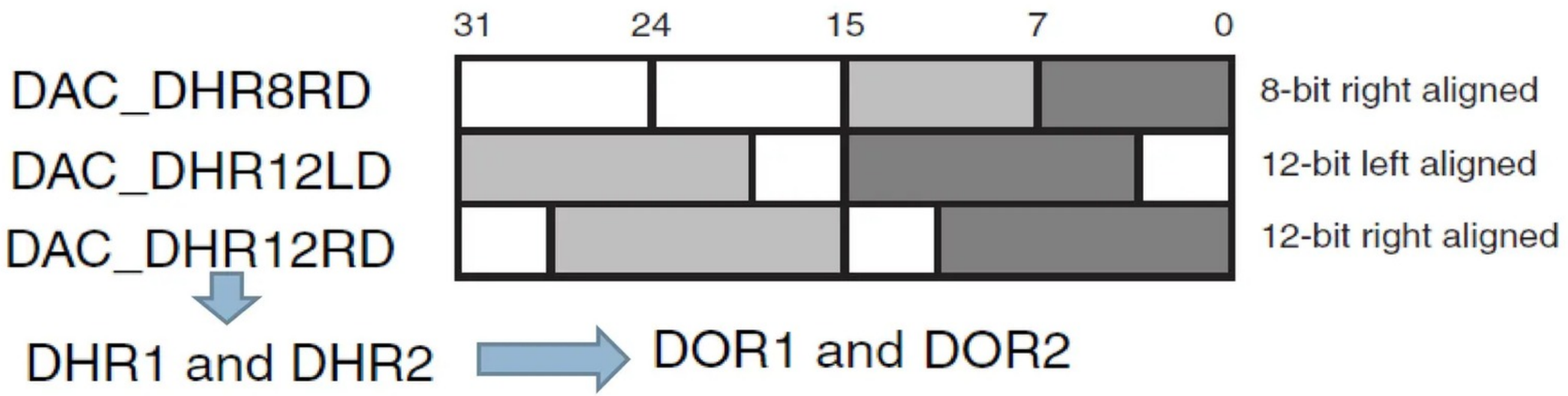
- DAC channel enable
  - Setting its corresponding ENx bit in the DAC_CR register
  - DAC channel is then enabled after a startup time t_{WAKEUP}
- DAC output buffer enable
  - DAC integrates two output buffers to reduce output impedance, and to drive external loads directly without having to add an external op amp
  - Enabled using the corresponding BOFFx bit in the DAC_CR register

DAC Functional Description

- DAC data format
  - Single DAC channelx



  - Dual DAC channels



DAC Functional Description

- DAC output voltage

$$\text{DACoutput} = V_{REF} \times \frac{DOR}{4095}$$

- DAC trigger selection
  - Rising edge for HW triggers: conversion after three APB1 clock cycles
  - If the software trigger is selected, the conversion starts once the SWTRIG bit is set. SWTRIG is reset by hardware once the DAC_DORx register has been loaded with the DAC_DHRx register contents

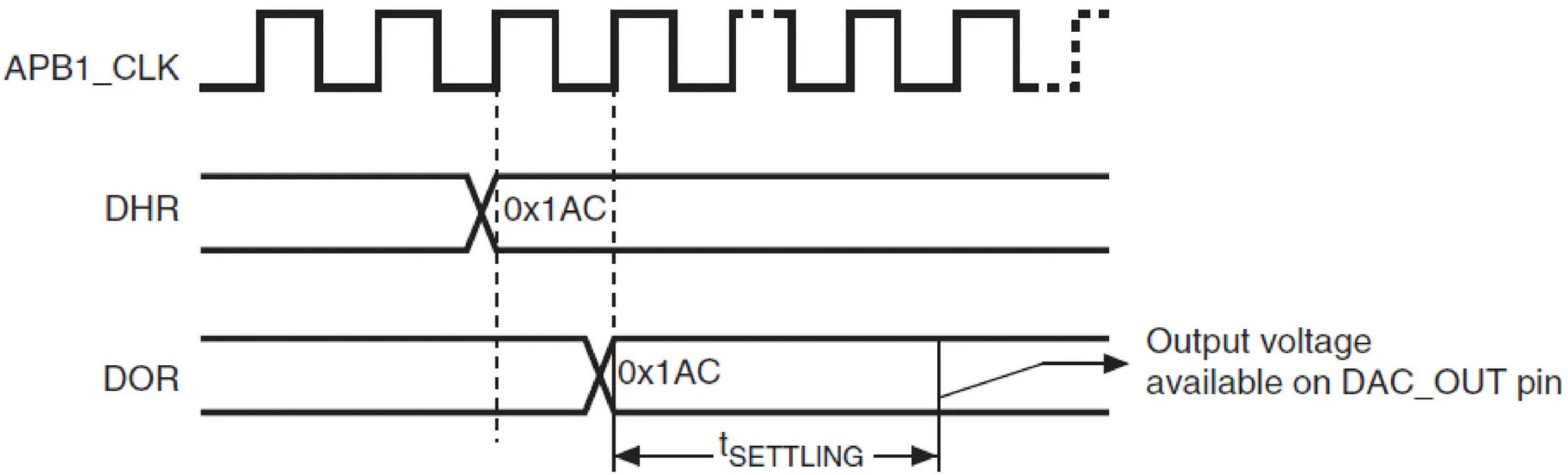
Source	Type	TSEL[2:0]
Timer 6 TRGO event	Internal signal from on-chip timers	000
Timer 3 TRGO event		001
Timer 7 TRGO event		010
Timer 5 or Timer 15 TRGO event		011
Timer 2 TRGO event		100
Timer 4 TRGO event		101
EXTI line9	External pin	110
SWTRIG	Software control bit	111

DAC Conversion

- DAC_DORx cannot be written directly and any data transfer to the DAC channelx must be performed by loading the DAC_DHRx register
- Data stored in the DAC_DHRx register are automatically transferred to the DAC_DORx register after one APB1 clock cycle, if no hardware trigger is selected (TENx bit in DAC_CR register is reset)
- When hardware trigger is selected (TENx bit in DAC_CR register is set) and a trigger occurs, transfer is performed three APB1 clock cycles later
- When DAC_DORx is loaded with the DAC_DHRx contents, the analog output voltage becomes available after a time $t_{SETTLING}$ that depends on power supply voltage and analog output load

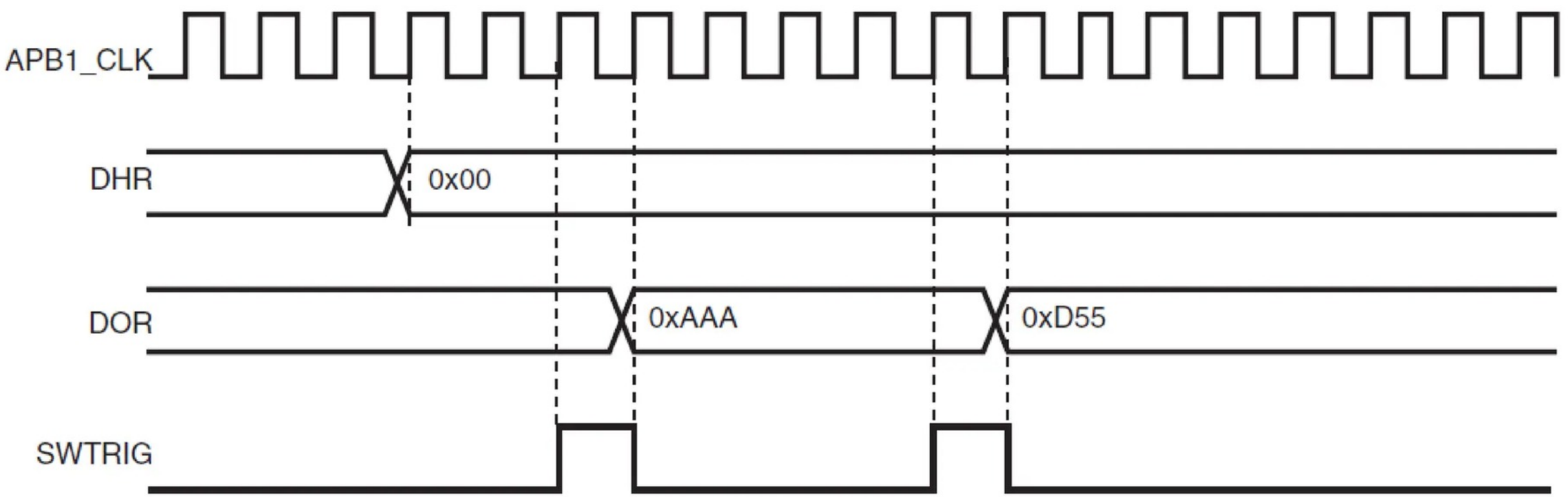
DAC Conversion

- Timing diagram for conversion with trigger disabled TEN = 0



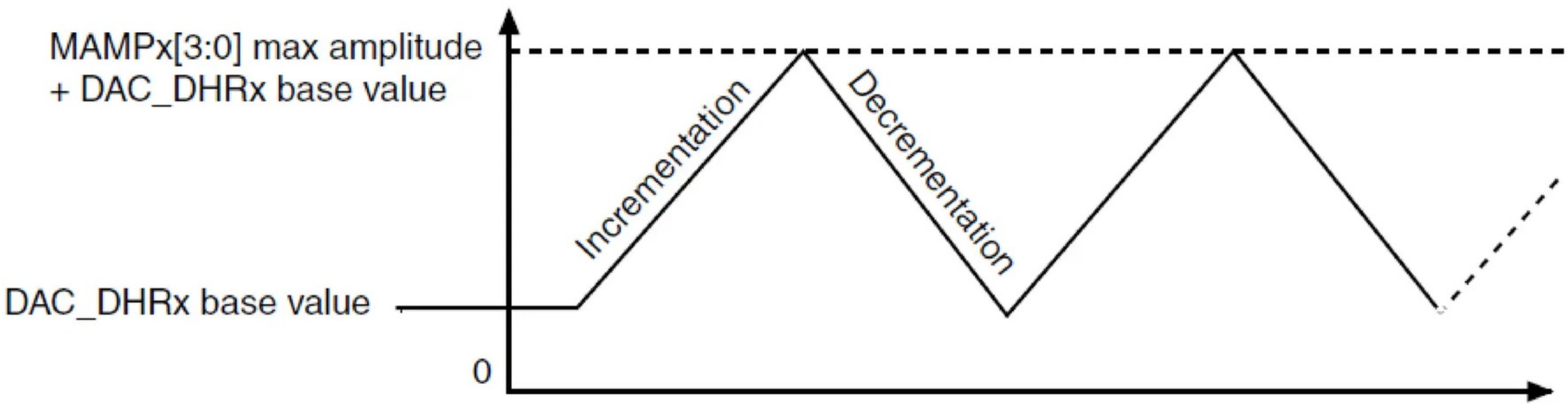
Noise Generation

- In order to generate a variable-amplitude pseudonoise, an LFSR (linear feedback shift register) is available
- Noise generation is selected by setting WAVEx[1:0] to “01”
- The preloaded value in LFSR is 0xAAA and is updated after each trigger event, following a specific calculation algorithm



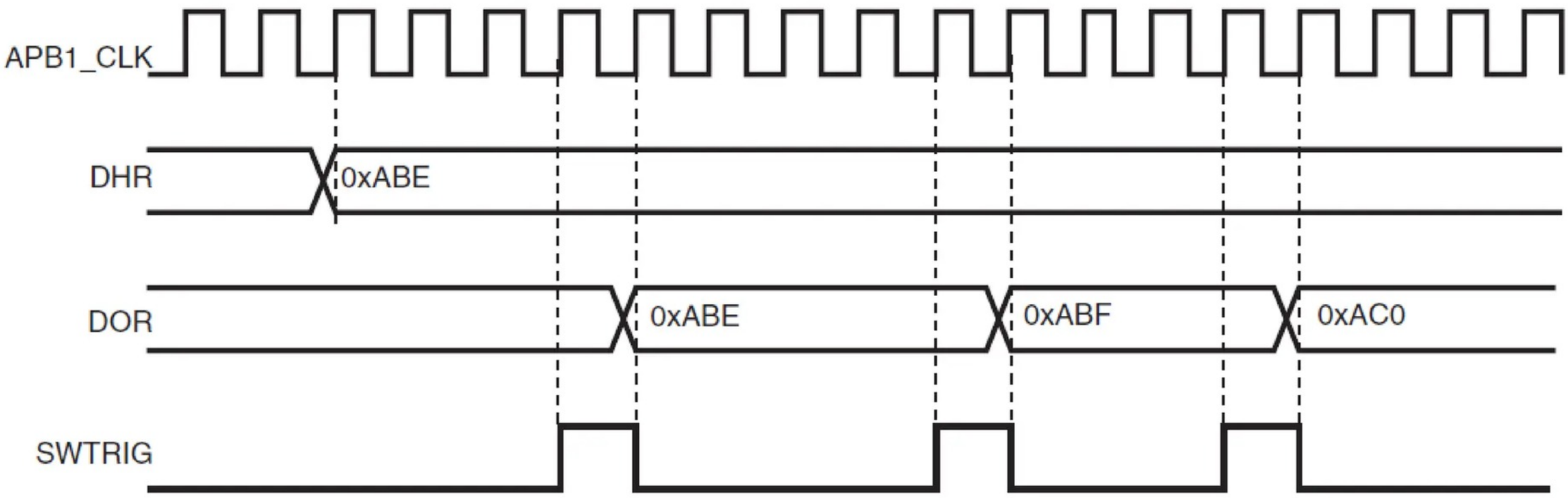
Triangle-Wave Generation

- It is possible to add a small-amplitude triangular waveform on a DC or slowly varying signal.
- DAC triangle-wave generation is selected by setting WAVEx[1:0] to “10”
  - Amplitude is configured through MAMPx[3:0] bits in DAC_CR register
- It is possible to reset triangle wave generation by resetting the WAVEx[1:0] bits



Triangle-Wave Generation

- DAC conversion (SW trigger enabled) with triangle wave generation
  - DAC trigger must be enabled by setting the TENx bit in DAC_CR register
  - The MAMPx[3:0] bits must be configured before enabling the DAC, otherwise they cannot be changed



DAC Control Register (DAC_CR)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DMAUDRIE2	DMAEN2	MAMP2[3:0]				WAVE2[1:0]		TSEL2[2:0]			TEN2	BOFF2	EN2
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		DMAUDRIE1	DMAEN1	MAMP1[3:0]				WAVE1[1:0]		TSEL1[2:0]			TEN1	BOFF1	EN1
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 11:8 **MAMP1[3:0]**: DAC channel1 mask/amplitude selector

These bits are written by software to select mask in wave generation mode or amplitude ir triangle generation mode.

0000: Unmask bit0 of LFSR/ triangle amplitude equal to 1

0001: Unmask bits[1:0] of LFSR/ triangle amplitude equal to 3

0010: Unmask bits[2:0] of LFSR/ triangle amplitude equal to 7

0011: Unmask bits[3:0] of LFSR/ triangle amplitude equal to 15

0100: Unmask bits[4:0] of LFSR/ triangle amplitude equal to 31

0101: Unmask bits[5:0] of LFSR/ triangle amplitude equal to 63

0110: Unmask bits[6:0] of LFSR/ triangle amplitude equal to 127

0111: Unmask bits[7:0] of LFSR/ triangle amplitude equal to 255

1000: Unmask bits[8:0] of LFSR/ triangle amplitude equal to 511

1001: Unmask bits[9:0] of LFSR/ triangle amplitude equal to 1023

1010: Unmask bits[10:0] of LFSR/ triangle amplitude equal to 2047

≥ 1011: Unmask bits[11:0] of LFSR/ triangle amplitude equal to 4095

Bits 7:6 **WAVE1[1:0]**: DAC channel1 noise/triangle wave generation enable

These bits are set and cleared by software.

00: wave generation disabled

01: Noise wave generation enabled

1x: Triangle wave generation enabled

Bits 5:3 **TSEL1[2:0]**: DAC channel1 trigger selection

Bit 2 **TEN1**: DAC channel1 trigger enable

Bit 1 **BOFF1**: DAC channel1 output buffer disable

Bit 0 **EN1**: DAC channel1 enable

DAC Software Trigger Register (DAC_SWTRIGR)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														SWTRIG2	SWTRIG1
														w	w

Bit 1 **SWTRIG2**: DAC channel2 software trigger

This bit is set and cleared by software to enable/disable the software trigger.

0: Software trigger disabled

1: Software trigger enabled

Note: This bit is cleared by hardware (one APB1 clock cycle later) once the DAC_DHR2 register value has been loaded into the DAC_DOR2 register.

Bit 0 **SWTRIG1**: DAC channel1 software trigger

This bit is set and cleared by software to enable/disable the software trigger.

0: Software trigger disabled

1: Software trigger enabled

Note: This bit is cleared by hardware (one APB1 clock cycle later) once the DAC_DHR1 register value has been loaded into the DAC_DOR1 register.

DAC Channel 1 Data Registers

- DAC channel1 12-bit right-aligned data holding register (DAC_DHR12R1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC1DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- DAC channel1 12-bit left aligned data holding register (DAC_DHR12L1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC1DHR[11:0]												Reserved			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				

- DAC channel1 8-bit right aligned data holding register (DAC_DHR8R1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DACC1DHR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

DAC Channel1 Data Output Register (DAC_DOR1)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC1DOR[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:12 Reserved.

Bit 11:0 **DACC1DOR[11:0]**: DAC channel1 data output

These bits are read-only, they contain data output for DAC channel1.

TIMx Control Register 2 (TIMx_CR2)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TI1S	MMS[2:0]			CCDS	Reserved		
								rw	rw	rw	rw	rw			

Bits 6:4 **MMS**: Master mode selection

These bits allow to select the information to be sent in master mode to slave timers for synchronization (TRGO). The combination is as follows:

000: **Reset** - the UG bit from the TIMx_EGR register is used as trigger output (TRGO). If the reset is generated by the trigger input (slave mode controller configured in reset mode) then the signal on TRGO is delayed compared to the actual reset.

001: **Enable** - the Counter enable signal, CNT_EN, is used as trigger output (TRGO). It is useful to start several timers at the same time or to control a window in which a slave timer is enabled. The Counter Enable signal is generated by a logic OR between CEN control bit and the trigger input when configured in gated mode.

When the Counter Enable signal is controlled by the trigger input, there is a delay on TRGO, except if the master/slave mode is selected (see the MSM bit description in TIMx_SMCR register).

010: **Update** - The update event is selected as trigger output (TRGO). For instance a master timer can then be used as a prescaler for a slave timer.

011: **Compare Pulse** - The trigger output send a positive pulse when the CC1IF flag is to be set (even if it was already high), as soon as a capture or a compare match occurred. (TRGO)

100: **Compare** - OC1REF signal is used as trigger output (TRGO)

101: **Compare** - OC2REF signal is used as trigger output (TRGO)

110: **Compare** - OC3REF signal is used as trigger output (TRGO)

111: **Compare** - OC4REF signal is used as trigger output (TRGO)

Assignments

- ARM Project #5